



نسخه 1.4.0 · راهنمای کامل کاربر

Tehran Stock Exchange Data & Analytics Library

کتابخانه داده و تحلیل بازار سرمایه ایران

Stocks · Live Market · Options · ETFs · Bonds · Currency

سهام · بازار زنده · اختیار معامله · صندوق · اوراق بدهی · ارز و سکه

Python 3.8–3.14

12 September 2026

algotik.com

Mohsen Alipour · @algotik

چطور از این دفترچه استفاده کنید؟

این راهنما دو لایه دارد: ابتدا مسیرهای سریع و توابعی که بیشتر کاربران هر روز نیاز دارند؛ سپس مرجع کامل ورودی‌ها، گزینه‌ها، خروجی‌ها و نکات رفتاری همهٔ API‌های عمومی پکیج.

۱

شروع سریع

نصب، import و اولین دریافت تاریخچهٔ قیمت در چند دقیقه.

۲

توابع پرکاربرد

قیمت، حقیقی/حقوقی، بازار زنده، سفارش، ارز، صندوق، اخزا و اختیار.

۳

قراردادهای داده

نوع خروجی، تاریخ، freshness، ردیف امروز، CSV و مرز منابع داده.

۴

مرجع تفصیلی

signatureها، تمام ورودی‌ها، مقادیرهای مجاز، پیش‌فرض‌ها و خروجی هر خانوادهٔ تابع.

نکتهٔ معاملاتی: پیش از استفاده در تصمیم‌گیری یا اجرای الگوریتم، زمان snapshot، ویژگی‌های `DataFrame.attrs`، وضعیت `partial` و freshness داده را بررسی کنید. این پکیج توصیهٔ سرمایه‌گذاری ارائه نمی‌کند.

A Python toolkit for historical, live and analytical data from Iran's capital market.

Fetch TSETMC prices, client type, trades, order books, market calendars, index impact, instrument reference data, funds, bonds, options and official subscriber TOP with Jalali date support, then use the built-in analytics for research and algorithmic trading.

IR معرفی فارسی

برای دریافت و تحلیل داده‌های بورس و فرابورس ایران ساخته شده است. با نام فارسی نماد می‌توانید تاریخچه `algotik-tse` قیمت، حقیقی/حقوقی، معاملات ریز، سفارش‌ها و اطلاعات لحظه‌ای بازار را بگیرید؛ شاخص‌های صنعت و اعضای دقیق آن‌ها را تحلیل کنید؛ فهرست صندوق‌ها، اوراق و اختیارها را بسازید؛ و تحلیل‌های تخصصی اخزا و اختیار معامله را روی همان داده‌ها انجام دهید.

بیشتر خروجی‌های جدولی به صورت **Pandas DataFrame** ارائه می‌شوند و تاریخ شمسی، تاریخ میلادی و داده‌های چندنمادی پشتیبانی می‌شوند. خروجی‌های ساختاریافته‌ای مانند snapshot بازار، زنجیره اختیار و منحنی بازده در بخش **نوع خروجی** توضیح داده شده‌اند.

ویژگی‌ها

- دریافت تاریخچه قیمت و حقیقی/حقوقی با نماد فارسی، تاریخ شمسی/میلادی، تعدیل قیمت، بازده و خروجی چندنمادی
- افزودن کنترل‌شده ردیف امروز با `include_today=True` و metadata مربوط به freshness و partial بودن داده
- نمای زنده کل بازار یا یک نماد، قدرت خریدار، جریان پول، spread و imbalance پنج سطح سفارش
- معاملات ریز، order book و صف خرید/فروش به صورت زنده و تاریخی
- تقویم معاملاتی بورس/فرابورس، ارزش بازار تاریخی، آمار فعالیت روزانه و اثر هر نماد بر شاخص
- دریافت یکجای خلاصه روزانه همه ابزارها و فهرست مرجع کامل TSETMC با تشخیص تغییرات بر پایه `InsCode`
- قیمت تئوریک گشایش رسمی (TOP) برای مشترکان وب‌سرویس TSETMC و محاسبه شفاف عدم تعادل پیش‌گشایش
- SQLite جریان صنایع و ذخیره اختیاری تاریخچه روی breadth، بازار، پیام‌ها، تغییر وضعیت watcher
- ۴۵ شاخص صنعت، اعضای رسمی، snapshot تحلیلی، تاریخچه اعضا، کندل درون‌روزی، مقایسه و هم‌بستگی صنایع
- `InsCode` رخدادهای تعدیل قیمت و حل دقیق هویت ابزار با P/E و EPS
- صندوق‌های ثبت‌شده و بورسی با طبقه‌بندی چندمحوره، پوشش ابزارهای ۳۰۵ و ۳۸۰ و فیلتر مستقیم طلا، نقره، کالایی، اهرمی، بخشی و شاخصی
- تحلیل اخزا شامل DV01، duration، convexity و YTM و منحنی بازده زنده و تاریخی
- تحلیل اختیار معامله شامل PCR، put-call parity، Greeks، IV، Black-Scholes و نقدشوندگی
- ارز و سکه، اینترادی، سهامداران، شاخص‌ها، ETFها، صندوق‌ها و اوراق بدهی
- و محدودسازی منبع داده retry، rate limiting، به صورت پیش‌فرض TLS و اعتبارسنجی HTTPS

وبسایت: algotik.com | تلگرام: t.me/algotik

این کتابخانه توصیه سرمایه‌گذاری نیست. برای استفاده معاملاتی، زمان snapshot، freshness، partial بودن داده و `DataFrame.attrs` را بررسی کنید.

API در یک نگاه

اگر اولین بار است از پکیج استفاده می‌کنید، معمولاً همین توابع نیاز شما را پوشش می‌دهند:

تابع	چه کاری انجام می‌دهد؟	خروجی
★ <code>get_history()</code>	تاریخچه OHLCV یک یا چند نماد	<code>DataFrame</code>
★ <code>get_client_type()</code>	تاریخچه خریدوفروش حقیقی/حقوقی	<code>DataFrame</code>
★ <code>get_live_symbol()</code>	نمای زنده و تحلیلی یک نماد	یک ردیفی <code>DataFrame</code>
★ <code>get_live_market()</code>	نمای زنده و تحلیلی کل بازار	<code>DataFrame</code>
★ <code>get_market_snapshot()</code>	خام قیمت و پنج سطح سفارش snapshot کل بازار	<code>dict</code>
★ <code>get_intraday()</code>	تیک یا کندل اینترادی	در <code>None</code> یا <code>DataFrame</code> قدیمی API
★ <code>get_symbols()</code>	فهرست نمادها بر اساس بازار و نوع ابزار	یا فهرست <code>DataFrame</code>
★ <code>get_currency()</code>	تاریخچه ارز و سکه	<code>DataFrame</code>
★ <code>get_live_trades()</code>	معاملات ریز امروز یک نماد	<code>DataFrame</code>
★ <code>get_order_book()</code>	پنج سطح سفارش زنده	<code>DataFrame</code>
★ <code>get_trading_calendar()</code>	تقویم معاملاتی رسمی بورس و فرابورس	<code>DataFrame</code>
★ <code>get_market_activity()</code>	تعداد، حجم و ارزش معاملات روزانه کل بازار	<code>DataFrame</code>
★ <code>get_market_value_history()</code>	تاریخچه ارزش بازار بورس و فرابورس	<code>DataFrame</code>
★ <code>get_index_impact()</code>	نمادهای اثرگذار مثبت و منفی روی شاخص	<code>DataFrame</code>
★ <code>get_market_trades()</code>	خلاصه روزانه همه ابزارها در یک درخواست	<code>DataFrame</code>
★ <code>get_industry_snapshot()</code>	بازده، breadth، جریان پول و صف یک یا همه صنایع	<code>DataFrame</code>
★ <code>compare_industries()</code>	مقایسه سری زمانی چند شاخص صنعت	<code>DataFrame</code>
★ <code>get_industry_relative_strength()</code>	محاسبه برتری نسبی به مقابل شاخص مبنا	<code>DataFrame</code>
★ <code>get_industry_correlation()</code>	ماتریس همبستگی بازدهی روزانه صنایع	<code>DataFrame</code>

تابع	چه کاری انجام می‌دهد؟	خروجی
★ <code>get_industry_membership_overlap()</code>	اندازه‌گیری همپوشانی رسمی اعضای صنایع	<code>DataFrame</code>
★ <code>get_industry_membership_events()</code>	ردیف‌های <code>added</code> / <code>dropped</code> عضویت رسمی صنعت	<code>DataFrame</code>
★ <code>get_industry_membership_churn()</code>	شاخص تغییرپذیری اعضا بین جلسات اخیر	<code>DataFrame</code>
★ <code>get_industry_concentration()</code>	شاخص تمرکز و متمرکزشدن سرمایه صنعت	<code>DataFrame</code>
★ <code>get_industry_momentum_profile()</code>	پروفایل مومنتوم چنددوره‌ای برای صنایع	<code>DataFrame</code>
★ <code>get_industry_correlation_neighborhood()</code>	همبستگی همسایگی یک صنعت با نزدیک‌ترین‌ها	<code>DataFrame</code>
★ <code>get_industry_health_score()</code>	نمره سلامت ترکیبی برای رتبه‌بندی صنایع	<code>DataFrame</code>
★ <code>rank_industries()</code>	رتبه‌بندی صنایع با معیار انتخابی	<code>DataFrame</code>
★ <code>list_etfs()</code>	NAV ها همراه قیمت و ETF	<code>DataFrame</code>
★ <code>list_funds()</code>	صندوق‌ها همراه NAV، بازده و ترکیب دارایی	<code>DataFrame</code>
★ <code>list_listed_funds()</code>	صندوق‌های بورسی و فیلتر نوع دارایی، استراتژی و کالای پایه	<code>DataFrame</code>

نقشه کامل توابع کاربری

جدول‌های زیر API‌های canonical را نشان می‌دهند. نام‌های قدیمی مانند `stock()` و `stock_RI()` همچنان کار می‌کنند، اما برای کد جدید نام‌های `get_*` پیشنهاد می‌شوند.

قیمت، حقیقی/حقوقی و اطلاعات نماد

تابع	کاربرد
<code>get_history()</code>	تاریخچه قیمت تعدیل‌شده/خام، بازده و داده چندنمادی
<code>get_client_type()</code>	تاریخچه حقیقی/حقوقی و قدرت خریدار
<code>get_intraday()</code>	تیک و کندل‌های ۱ دقیقه تا ۱۲ ساعت
<code>get_trades()</code> , <code>get_live_trades()</code>	معاملات ریز تاریخی و امروز
<code>get_detail()</code> , <code>get_info()</code> , <code>get_stats()</code>	جزئیات، اطلاعات و آمار TSETMC نماد
<code>get_shareholders()</code>	سهامداران عمده فعلی یا تاریخی
<code>get_capital_increase()</code>	تاریخچه افزایش سرمایه
<code>get_price_adjustments()</code> , <code>get_latest_price_adjustment()</code>	رخداد‌های تعدیل قیمت
<code>get_introduction()</code>	فقط سازگاری قدیمی؛ همیشه <code>UnsupportedDataSourceError</code>

داده زنده، سفارش و تحلیل بازار

تابع	کاربرد
<code>get_market_snapshot()</code>	خام و اتمیک کل بازار snapshot
<code>get_market_client_type()</code>	حقیقی/حقوقی bulk کل بازار
<code>get_live_market()</code> , <code>get_live_symbol()</code>	نمای زنده ادغام شده بازار یا یک نماد
<code>get_order_book()</code> , <code>get_queue()</code>	پنج سطح سفارش و صف زنده
<code>get_order_book_history()</code> , <code>get_queue_history()</code>	تاریخچه بازسازی شده سفارش و صف
<code>get_market_messages()</code>	پیام های ناظر بازار
<code>get_instrument_state_changes()</code>	تغییر وضعیت ابزارها
<code>get_market_overview()</code>	نمای کلی رسمی بازار
<code>get_market_breadth()</code>	و شمار نمادهای مثبت/منفی A/D, breadth
<code>get_sector_flow()</code>	و جریان پول به تفکیک صنعت breadth
<code>watch_market()</code> , <code>MarketWatcher</code>	پایش افزایشی بازار و تولید <code>MarketEvent</code>
<code>get_trading_calendar()</code>	جلسات معاملاتی منتشر شده بورس و فرابورس
<code>get_market_activity()</code>	تجمیع روزانه تعداد ابزار، معامله، حجم و ارزش
<code>get_market_value_history()</code>	تاریخچه رسمی ارزش بازار با تغییر روزانه
<code>get_index_impact()</code>	اثر نمادها بر شاخص کل هر بازار attribution
<code>get_market_trades()</code>	خلاصه bulk روزانه ابزارها؛ معاملات ریز نیست
<code>get_theoretical_opening_price()</code>	TSETMC رسمی برای مشترکان وب سرویس TOP
<code>get_preopen_imbalance()</code>	عدم تعادل حجم خرید و فروش روی خروجی TOP

شاخص ها و تحلیل صنایع

تابع	کاربرد
<code>list_industry_indices()</code>	فهرست ۴۵ شاخص صنعت و وضعیت فعلی آنها
<code>get_industry_members()</code>	اعضای رسمی یک شاخص با اتصال دقیق <code>InsCode</code> به داده زنده
<code>get_industry_snapshot()</code>	یک ردیف تحلیلی برای هر صنعت: شاخص، breadth، معامله، حقیقی و صف
<code>get_industry_history()</code>	تاریخچه روزانه خود شاخص صنعت بدون حجم ساختگی
<code>get_industry_members_history()</code>	تاریخچه کوتاه همه اعضای فعلی صنعت در قالب long-form
<code>get_industry_intraday()</code>	مشاهدات خام یا کندل های درون روزی شاخص صنعت
<code>compare_industries()</code>	مقایسه سری زمانی شاخص ها و متریک های قیمت/بازده

تابع	کاربرد
<code>get_industry_relative_strength()</code>	محاسبه بازده تجمعی نسبی هر شاخص در برابر benchmark
<code>get_industry_correlation()</code>	ماتریس همبستگی بازده روزانه بین چند صنعت
<code>get_industry_membership_overlap()</code>	همپوشانی رسمی اعضای چند شاخص صنعت را pairwise محاسبه می‌کند
<code>get_industry_membership_events()</code>	رویدادهای added / dropped تغییرات رسمی اعضا
<code>get_industry_membership_churn()</code>	اعضای رسمی صنعت طی چند جلسه اخیر churn
<code>get_industry_concentration()</code>	تمرکز روی چند عضو بزرگ و وزن سهمی، HHI
<code>get_industry_momentum_profile()</code>	پروفایل مومنتوم چند پنجره‌ای برای هر صنعت
<code>get_industry_correlation_neighborhood()</code>	نزدیک‌ترین همبستگی‌ها برای یک صنعت هدف
<code>get_industry_health_score()</code>	نمره سلامت ترکیبی breadth, momentum و concentration
<code>rank_industries()</code>	رتبه‌بندی صنایع بر اساس بازده، breadth، ارزش یا جریان پول

تاریخچه محلی و فاندامنتال

تابع	کاربرد
<code>save_market_snapshot()</code> , <code>load_market_snapshots()</code>	ذخیره و خواندن snapshotهای SQLite
<code>get_live_market_history()</code>	تاریخچه نمای زنده ذخیره‌شده
<code>get_market_overview_history()</code>	تاریخچه overview رسمی
<code>get_market_snapshot_summary_history()</code>	خلاصه snapshotهای ذخیره‌شده
<code>get_market_breadth_history()</code> , <code>get_sector_flow_history()</code>	تاریخچه تحلیل بازار و صنایع
<code>record_market_event()</code> , <code>get_market_event_history()</code>	ثبت و replay رخدادها watcher
<code>archive_market_records()</code>	آرشیو batch رکوردهای مستقل
<code>get_market_messages_history()</code>	پیام‌های آرشیو شده بازار
<code>get_instrument_state_changes_history()</code>	تغییر وضعیت‌های آرشیو شده
<code>check_market_history()</code>	بررسی سلامت و سازگاری فایل SQLite
<code>get_market_fundamentals()</code>	بازار P/E و EPS snapshot
<code>get_market_fundamentals_history()</code>	تاریخچه EPS/P/E بدون look-ahead

فهرست ابزارها و بازارها

تابع	کاربرد
<code>get_symbols()</code>	فهرست سهام، حق تقدم، صندوق، اوراق و اختیار

تابع	کاربرد
<code>get_instrument_master()</code>	جدول مرجع کامل ابزارها با ISIN و <code>InsCode</code>
<code>get_instrument_changes()</code>	مقایسه دو snapshot مرجع و گزارش افزوده/حذف/تغییر
<code>list_options()</code>	فهرست قراردادهای اختیار فعال
<code>get_options_chain()</code>	زنجیره call/put یک دارایی پایه
<code>list_etfs()</code>	مسیر قدیمی صندوق‌های بورسی عمومی نوع ۳۰۵ همراه NAV و discount/premium
<code>list_funds()</code>	صندوق‌های ثبتی با NAV، بازده، ترکیب دارایی و دسته/استراتژی قابل فیلتر
<code>list_listed_funds()</code>	ابزارهای بورسی دقیق، شامل صندوق‌های عمومی و کالایی با فیلتر طلا/نقره
<code>list_bonds()</code>	اوراق بدهی همراه سررسید
<code>list_indices()</code> , <code>get_index_companies()</code>	های بالا پیشنهاد می‌شوند API قدیمی شاخص‌ها و اعضای شاخص؛ برای صنعت API
<code>get_currency()</code>	ارز و سکه از API قدیمی TGJU

اخزا و درآمد ثابت

تابع	کاربرد
<code>parse_treasury_maturity()</code>	استخراج سررسید از نماد اخزا
<code>day_count_fraction()</code>	محاسبه فاصله زمانی با day-count convention
<code>treasury_yield()</code>	بازده اخزا از قیمت و سررسید
<code>bond_price()</code> , <code>yield_to_maturity()</code>	قیمت اوراق و حل YTM
<code>bond_analytics()</code>	duration, convexity و DV01
<code>build_yield_curve()</code>	ساخت شیء <code>YieldCurve</code> از nodeها
<code>get_ifb_yield_table()</code>	جدول مرجع YTM فرابورس
<code>get_treasury_yields()</code>	اخزا و تحلیل بازده snapshot
<code>get_treasury_yield_history()</code>	تاریخچه YTM اخزا
<code>get_yield_curve()</code> , <code>get_yield_curve_history()</code>	منحنی بازده زنده و تاریخی

اختیار معامله

تابع	کاربرد
<code>black_scholes_price()</code>	قیمت بلک-شولز اروپایی
<code>black_scholes_greeks()</code>	Delta, Gamma, Vega, Theta و Rho
<code>option_price_bounds()</code>	کران‌های بدون آربیتراژ

تابع	کاربرد
<code>implied_volatility()</code>	حل نوسان ضمنی
<code>get_option_market()</code>	اتمیک بازار اختیار snapshot
<code>analyze_option_chain()</code>	و نقدشوندگی IV، Greeks، parity
<code>option_put_call_ratios()</code>	حجم، ارزش و موقعیت باز PCR
<code>get_option_history()</code>	تاریخچه قرارداد اختیار
<code>save_option_snapshot()</code> , <code>load_option_snapshots()</code>	ذخیره و خواندن snapshot های JSON

هویت ابزار و ابزارهای کمکی

تابع	کاربرد
<code>resolve_instrument()</code>	تبدیل نماد یا <code>InsCode</code> به <code>InstrumentRef</code> دقیق
<code>validate_ins_code()</code>	اعتبارسنجی شناسه ابزار
<code>normalize_instrument_text()</code>	یکسان سازی ی/ک و فاصله های فارسی

برای signature و همه ورودی های هر تابع به مرجع تفصیلی همه توابع مراجعه کنید.

فهرست مطالب

- نصب و به روزرسانی
- شروع سریع
- راهنمای توابع پرکاربرد
- قراردادهای مهم داده
- حل دقیق هویت نماد
- قیمت و حقیقی/حقوقی
- معاملات ریز
- سفارش و صف
- و تحلیل کل بازار Watcher
- تقویم، نمای کلان، مستر ابزار و پیش گشایش
- شاخص ها و تحلیل صنایع
- تاریخچه محلی SQLite
- فاندمنتال، صندوق و تعدیل قیمت
- اخزا و درآمد ثابت

- [اختیار معامله](#)
- [سایر API های بازار](#)
- [تنظیمات و خطاها](#)
- [مرجع تفصیلی همهٔ توابع](#)
- [مثال های کاربردی](#)
- [تست و مشارکت](#)
- [تاریخچهٔ تغییرات](#)

نصب و به روزرسانی

```
pip install algotik-tse
```

برای ارتقا به آخرین نسخه:

```
python -m pip install --upgrade algotik-tse
```

نیازمندی نسخهٔ فعلی: **3.8** Python تا **3.14**.

برای نصب نسخهٔ توسعه:

```
git clone https://github.com/mohsenalipour/algotik_tse.git
cd algotik_tse
python -m pip install -e "[dev]"
```

شروع سریع

```
import algotik_tse as att
```

نیاز شما	تابع پیشنهادی
تاریخچهٔ قیمت تعدیل شده	<code>att.get_history("شتران")</code>
حقیقی/حقوقی یک نماد	<code>att.get_client_type("شتران")</code>
اطلاعات زندهٔ یک نماد	<code>att.get_live_symbol("شتران")</code>

نیاز شما	تابع پیشنهادی
لحظه‌ای کل بازار snapshot	<code>att.get_market_snapshot()</code>
کندل‌های اینترادی	<code>att.get_intraday("شتران", interval="5min")</code>
معاملات ریز امروز	<code>att.get_live_trades("شتران")</code>
پنج سطح سفارش	<code>att.get_order_book("شتران")</code>
تقویم روزهای معاملاتی	<code>att.get_trading_calendar(market="tse")</code>
ارزش و فعالیت تاریخی بازار	<code>att.get_market_value_history()</code> و <code>att.get_market_activity()</code>
اثرگذارترین نمادهای شاخص	<code>att.get_index_impact(top=10)</code>
خلاصه همه ابزارهای یک روز	<code>att.get_market_trades("1405-06-18")</code>
تصویر تحلیلی صنایع	<code>att.get_industry_snapshot()</code>
رتبه‌بندی صنایع	<code>att.rank_industries(metric="breadth")</code>
فهرست نمادها و ابزارها	<code>att.get_symbols()</code>
قیمت ارز و سکه	<code>att.get_currency("dollar")</code>
صندوق‌های ETF با NAV	<code>att.list_etfs()</code>
اخزا و YTM	<code>att.get_treasury_yields()</code>
زنجیره و تحلیل اختیار	<code>att.get_option_market()</code> و <code>att.analyze_option_chain()</code>

اولین دریافت: تاریخچه قیمت

```
prices = att.get_history(
    "شتران",
    start="1403-01-01",
    end="1403-03-31",
    progress=False,
)
print(prices.tail())
```

خروجی نماینده:

J-Date	Open	High	Low	Close	Volume
1403-03-26	2630	2680	2605	2668	82471422
1403-03-27	2670	2715	2641	2692	70938510
1403-03-28	2695	2734	2670	2718	93612045

قیمت‌ها در حالت پیش‌فرض تعدیل می‌شوند. برای داده خام از `auto_adjust=False` و برای ستون‌های کامل‌تر از `output_type="full"` استفاده کنید.

چند کاربرد رایج در یک نگاه

```
# حقیقی/حقوقی ۳۰ روز اخیر
client_type = att.get_client_type("فملی", limit=30, progress=False)

# است opt-in امروز؛ این رفتار observation تاریخچه به همراه
prices_today = att.get_history(
    "فملی", limit=20, include_today=True, progress=False
)

# نمای زنده نماد و قدرت خریدار
live = att.get_live_symbol("فملی", fallback="none")
print(live[["Symbol", "Last", "Close", "IndividualPower"]])

# پنج سطح سفارش، صف و معاملات امروز
book = att.get_order_book("فملی")
queue = att.get_queue("فملی", side="both", strict=True)
today_trades = att.get_live_trades("فملی")

# ایزارهای بازار
etfs = att.list_etfs()
funds = att.list_funds(fund_type="fixed_income")

# اخزا و اختیار معامله
treasuries = att.get_treasury_yields(min_volume=1)
options = att.get_option_market(underlying="خودرو")
analytics = att.analyze_option_chain(options, risk_free_rate=0.30)
```

خروجی نماینده `get_live_symbol()` در زمان بازار:

Symbol	Last	Close	IndividualPower
0	1.31	73950	74200 فملی

مقادیر مثال‌های متصل به شبکه «خروجی نماینده» هستند و با زمان بازار تغییر می‌کنند. مثال‌های ریاضی deterministic هستند و در تست‌های آفلاین کنترل می‌شوند. ستون‌های `Last` و `Close` در feed زنده به ترتیب «آخرین معامله» و «قیمت پایانی» هستند؛ تفاوت نام‌گذاری live و history در بخش بعد آمده است.

سند مرجع واحد پروژه است. اگر تازه شروع کرده‌اید، بخش راهنمای توابع پرکاربرد را بخوانید؛ [مرجع تفصیلی همه README.md](#) ورودی، خروجی و قرارداد هر تابع است و لازم نیست از ابتدا تا انتها خوانده شود، signature دقیق lookup توابع برای

راهنمای توابع پرکاربرد

این بخش برای استفاده روزمره است. ورودی‌های هر تابع کنار همان تابع توضیح داده شده‌اند؛ پس از آن، مرجع تخصصی تمام API‌ها قرار دارد.

get_history() — تاریخچه قیمت

```
get_history(
    symbol="", start=None, end=None, limit=0,
    raw=False, auto_adjust=True, output_type="standard",
    date_format="jalali", progress=True, save_to_file=False,
    dropna=True, adjust_volume=False, return_type=None,
    ascending=True, save_path=None, include_today=False,
    *, ins_code=None, asset_type="auto", **kwargs,
)
```

ورودی	گزینه‌ها و معنی
symbol	نماد فارسی یا فهرست نمادها؛ مانند "فملی" یا ["فملی", "شتران"] .
start , end	ابتدا و انتهای شامل بازه؛ تاریخ شمسی 1403-01-01 یا میلادی 2024-03-20 .
limit	تعداد آخرین ردیف‌های معاملاتی؛ 0 یعنی همه تاریخچه موجود.
raw	با True نام ستون‌ها به فرمت نزدیک TSETMC برمی‌گردد.
auto_adjust	پیش‌فرض True ؛ OHLC را با سری تعدیل‌شده بازسازی می‌کند.
output_type	و اطلاعات بیشتر Final , No. , Value برای "full" یا OHLCV برای "standard"
date_format	"both" یا "gregorian" ، "jalali"
progress	نمایش پیام پیشرفت؛ روی داده خروجی اثری ندارد.
save_to_file	ذخیره CSV را فعال می‌کند.
save_path	پوشه مقصد CSV؛ نام فایل از نماد ساخته می‌شود.
dropna	در خروجی چندنمادی ستون‌های کاملاً تهی را حذف می‌کند.
adjust_volume	حجم را نیز متناسب با ضریب افزایش سرمایه تعدیل می‌کند.
return_type	["simple", "Close", 5] یا فرم سفارشی مانند "both" ، "log" ، "simple"
ascending	جدید به قدیمی False قدیمی به جدید؛ True
include_today	با observation True معتبر امروز را به صورت opt-in اضافه/جایگزین می‌کند.
ins_code	شناسه دقیق ابزار؛ برای نمادهای تکراری و production پیشنهاد می‌شود.
asset_type	"equity" ، "index" ، "bond" نوع ابزار مانند hint یا "auto"
**kwargs	فقط aliasهای قدیمی مستند مانند stock , values , tse_format ؛ کلید ناشناخته API عمومی نیست.

خروجی: DataFrame تک‌نمادی یا DataFrame با ستون‌های MultiIndex برای چند نماد.

get_client_type() – حقیقی/حقوقی

```
get_client_type(
    symbol="", start=None, end=None, limit=0, raw=False,
    output_type="standard", date_format="jalali", progress=True,
    save_to_file=False, dropna=True, ascending=True, save_path=None,
    include_today=False, *, ins_code=None, asset_type="auto", **kwargs,
)
```

ورودی	گزینه‌ها و معنی
<code>symbol</code> , <code>ins_code</code> , <code>asset_type</code>	انتخاب نماد؛ قواعد آن‌ها مانند <code>get_history()</code> است.
<code>start</code> , <code>end</code> , <code>limit</code>	بازه یا تعداد آخرین روزهای معاملاتی.
<code>raw</code>	خروجی نزدیک به schema خام provider.
<code>output_type</code>	سرانه و قدرت را نیز نگه می‌دارد، value "full" ؛ حالت کامل یا "standard"
<code>date_format</code>	"both" یا "gregorian" ، "jalali"
<code>include_today</code>	ردیف حقیقی/حقوقی امروز را با provenance و برچسب برآوردی بودن اضافه می‌کند.
<code>progress</code> , <code>dropna</code> , <code>ascending</code>	نمایش پیشرفت، حذف ستون تهی چندنامی و ترتیب زمانی.
<code>save_to_file</code> , <code>save_path</code>	ذخیره CSV و پوشه مقصد.
<code>**kwargs</code>	limit=30 به جای values=30 های قدیمی مانند alias

خروجی: `DataFrame` روزانه تعداد/حجم/ارزش خریدوفروش حقیقی و حقوقی و شاخص‌های مشتق‌شده.

get_live_symbol() و get_live_market() – داده زنده

```
get_live_symbol(symbol=None, *, ins_code=None, fallback="none")
get_live_market(symbol=None, *, strict=False)
```

تابع/ورودی	گزینه‌ها و معنی
<code>get_live_symbol.symbol</code>	یک نماد فارسی یا <code>InsCode</code> .
<code>get_live_symbol.ins_code</code>	شناسه دقیق keyword-only؛ بر selector مبهم ترجیح دارد.
<code>fallback</code>	نقطه‌ای استفاده می‌کند endpoint در نبود نماد از "point" ؛ MarketWatch فقط "none"
<code>get_live_market.symbol</code>	برای کل بازار، یک نماد یا فهرست نمادها None
<code>strict</code>	با False نماد گم‌شده در attrs['missing_selectors'] ثبت می‌شود؛ با True خطا می‌دهد.

خروجی: `get_live_symbol()` یک `DataFrame` یک‌ردیفی و `get_live_market()` یک `DataFrame` فیلترشده یا کل بازار می‌دهد. ستون‌های مهم شامل `Last` , `Close` , `IndividualPower` , `EstimatedNetIndividualFlow` , `SpreadBps` و `L5Imbalance` هستند.

`get_market_client_type()` و `feed` – خام `bulk`

```
get_market_snapshot()
get_market_client_type()
```

این دو تابع ورودی کاربری ندارند. `get_market_snapshot()` یک `dict` شامل `stocks` , `order_book` , `metadata` و زمان بازار و `get_market_client_type()` یک `bulk DataFrame` حقیقی/حقوقی کل بازار برمی‌گرداند. برای مصرف معمول، `get_live_market()` نسخهٔ ادغام‌شده و راحت‌تر است.

`get_intraday()` – تیک و کندل

```
get_intraday(
    symbol="شتران", interval="1min",
    start=None, end=None, progress=True, **kwargs,
)
```

ورودی	گزینه‌ها و معنی
<code>symbol</code>	نماد فارسی.
<code>interval</code>	"1m" , هایی مانند "12h" , "1h" , "4h" , "30min" , "15min" , "5min" , "1min" , "tick" , نیز پذیرفته می‌شوند "60min" , "240m" , "720"
<code>start</code> , <code>end</code>	بدون مقدار، دادهٔ امروز؛ با تاریخ، snapshot های تاریخی بازه.
<code>progress</code>	نمایش پیشرفت.
<code>**kwargs</code>	فقط سازگاری نام‌های قدیمی.

خروجی: در حالت tick دادهٔ معامله و در حالت candle ستون‌های `Open` , `High` , `Low` , `Close` , `Volume` , `TradeCount` . این API قدیمی برای ورودی نامعتبر ممکن است پیام چاپ کند و `None` بدهد.

`get_symbols()` – فهرست ابزارها

```
get_symbols(
    bourse=True, farabourse=True, payeh=True,
    haghe_taqadom=False, sandogh=False, bonds=False,
    options=False, mortgage=False, commodity=False, energy=False,
    payeh_color=None, output="dataframe", progress=True, **kwargs,
)
```

ورودی	گزینه‌ها و معنی
<code>bourse</code> , <code>farabourse</code> , <code>payeh</code>	بازارهای سهام که به صورت پیش فرض فعال اند.
<code>haghe_taqadom</code>	افزودن حق تقدم‌ها.
<code>sandogh</code>	افزودن صندوق‌ها.
<code>bonds</code>	افزودن اوراق بدهی.
<code>options</code>	افزودن اختیار معامله‌ها.
<code>mortgage</code>	افزودن اوراق تسهیلات مسکن.
<code>commodity</code>	افزودن گواهی‌های کالایی.
<code>energy</code>	افزودن ابزارهای انرژی.
<code>payeh_color</code>	"قرمز" , "نارنجی" , "زرد" یا فهرستی از آن‌ها.
<code>output</code>	یا خروجی فهرستی قدیمی <code>"dataframe"</code>
<code>progress</code>	نمایش پیشرفت.
<code>**kwargs</code>	نام‌های انگلیسی قدیمی فیلترها.

خروجی: فهرست ابزارها با هویت، نماد، نام، بازار و نوع دارایی.

get_currency() — ارز و سکه

```
get_currency(
    name="", start=None, end=None, limit=0,
    output_type="standard", date_format="jalali", progress=True,
    save_to_file=False, dropna=True, return_type=None,
    ascending=True, save_path=None, **kwargs,
)
```

ورودی	گزینه‌ها و معنی
<code>name</code>	یک نام یا فهرست؛ مانند <code>"نیم سکه"</code> , <code>"seke"</code> , <code>"euro"</code> , <code>"dollar"</code> .
<code>start</code> , <code>end</code> , <code>limit</code>	بازه یا آخرین تعداد ردیف.
<code>output_type</code>	قدیمی API یا حالت کامل پشتیبانی شده <code>"standard"</code>
<code>date_format</code>	<code>"jalali"</code> , <code>"gregorian"</code> , <code>"both"</code> .
<code>return_type</code>	محاسبه بازده ساده/لگاریتمی.
<code>progress</code> , <code>dropna</code> , <code>ascending</code>	پیشرفت، خروجی چندارزی و ترتیب زمانی.
<code>save_to_file</code> , <code>save_path</code>	ذخیره CSV و پوشه مقصد.
<code>**kwargs</code>	های سازگاری قدیمی <code>alias</code> .

خروجی: `DataFrame[Open, High, Low, Close]`؛ برای چند ارز ستون‌ها MultiIndex می‌شوند.

get_live_trades() و get_trades() — معاملات ریز

```
get_trades(
    symbol=None, *, ins_code=None, start=None, end=None,
    include_canceled=False, max_requests=None, raw=False, progress=True,
)
get_live_trades(
    symbol=None, *, ins_code=None,
    include_canceled=False, max_requests=None, raw=False, progress=True,
)
```

ورودی	گزینه‌ها و معنی
<code>symbol</code> , <code>ins_code</code>	انتخاب نماد یا نام یا شناسه دقیق.
<code>start</code> , <code>end</code>	بازه تاریخی شامل دو سر؛ فقط در <code>get_trades()</code> .
<code>include_canceled</code>	نگه‌داشتن معاملات ابطالی.
<code>max_requests</code>	سقف سخت تعداد درخواست‌ها؛ برای بازه‌های بزرگ حتماً تعیین کنید.
<code>raw</code>	افزودن ستون‌های audit نزدیک provider.
<code>progress</code>	نمایش پیشرفت.

خروجی: `DataFrame` معاملات با `TradeNo`, `Timestamp`, `Price`, `Volume`, `Value`, `Canceled`, `Source`.

get_order_book() و get_queue() — سفارش و صف زنده

```
get_order_book(symbol=None, *, selector_strict=False)
get_queue(symbol=None, side="both", strict=True, *, selector_strict=False)
```

ورودی	گزینه‌ها و معنی
<code>symbol</code>	برای کل بازار، یک نماد یا فهرست نمادها <code>None</code> .
<code>selector_strict</code>	گم‌شده یا مبهم را به خطا تبدیل می‌کند <code>selector</code> .
<code>side</code>	در <code>get_queue()</code> : <code>"buy"</code> , <code>"sell"</code> , <code>"both"</code> .
<code>strict</code>	فقط صف قطعی مبتنی بر قیمت مجاز و level یک را نگه می‌دارد.

خروجی: order book به شکل long با پنج level؛ queue با سمت، قیمت، حجم، تعداد سفارش و ارزش صف.

و اختیاریها `list_etfs()` , `list_funds()` , `list_bonds()`

```
list_etfs(progress=True)
list_funds(fund_type=None, progress=True, *, listed_only=False,
           strategy=None, commodity_underlying=None,
           classification_status=None)
list_listed_funds(progress=True, *, fund_category=None, strategy=None,
                  commodity_underlying=None, classification_status=None,
                  include_unknown=True)
list_bonds(progress=True)
list_options(underlying=None, progress=True)
get_options_chain(underlying, fetch_oi=False, progress=True)
```

تابع/ورودی	گزینه‌ها و معنی
<code>progress</code>	در همهٔ این توابع فقط نمایش پیشرفت را کنترل می‌کند.
<code>fund_type</code>	<code>fixed_income</code> , <code>commodity</code> , <code>equity</code> , <code>mixed</code> , <code>market_maker</code> , <code>venture_capital</code> , <code>project</code> , <code>private</code> , <code>fund_of_funds</code> , <code>real_estate</code> , <code>sector</code> , <code>leveraged</code> , <code>index</code> , <code>capital_guaranteed</code> , <code>supplementary_retirement</code> . نیز پذیرفته می‌شود <code>venture</code> نام قدیمی.
<code>listed_only</code>	در <code>list_funds()</code> فقط ابزارهای قابل معامله را برمی‌گرداند؛ هم‌زمان با <code>fund_type</code> مجاز نیست.
<code>fund_category</code>	در <code>list_listed_funds()</code> دستهٔ اثبات‌شده، مانند <code>"commodity"</code> .
<code>strategy</code>	تگ دقیق مانند <code>"index_tracking"</code> , <code>"leveraged"</code> , <code>"sector"</code> یا <code>"market_making"</code> .
<code>commodity_underlying</code>	کالای پایهٔ اثبات‌شده: <code>"gold"</code> , <code>"silver"</code> یا <code>"saffron"</code> ؛ نام عمومی حدس زده نمی‌شود.
<code>classification_status</code>	فیلتر بر اساس <code>authoritative_field</code> , <code>verified_exact_mapping</code> یا <code>explicit_official_name</code> . <code>unknown</code> .
<code>include_unknown</code>	در فهرست بورسی، با <code>False</code> ردیف‌های فاقد طبقه‌بندی کافی را حذف می‌کند؛ پیش‌فرض <code>True</code> همه را نگه می‌دارد.
<code>underlying</code>	در <code>list_options()</code> فیلتر اختیاری و در <code>get_options_chain()</code> دارایی پایهٔ اجباری.
<code>fetch_oi</code>	در زنجیرهٔ اختیار، دریافت Open Interest و metadata تکمیلی را فعال می‌کند و ممکن است درخواست‌های بیشتری بسازد.

خروجی: همه `DataFrame` هستند، به جز `get_options_chain()` که `dict` شامل `calls` , `puts` , `underlying_price` , `expiry_dates` و `market_time` می‌دهد.

قراردادهای مهم داده

نوع خروجی

همه خروجی‌ها DataFrame نیستند:

خانواده	نوع خروجی
قیمت، حقیقی/حقوقی، trades، order book، fundamentals و فهرست ابزارها	<code>None</code> گاهی legacy های API یا در <code>pandas.DataFrame</code> هنگام خطا
<code>market_watch()</code> / <code>get_market_snapshot()</code>	و زمان <code>metadata</code> ، <code>order_book</code> ، <code>stocks</code> شامل <code>dict</code> مشاهده
<code>get_options_chain()</code>	<code>calls</code> و <code>puts</code> شامل <code>dict</code>
<code>resolve_instrument()</code>	شیء <code>ImmutableRef</code> از نوع <code>InstrumentRef</code>
<code>watch_market()</code> / <code>MarketWatcher</code>	<code>MarketEvent</code> از iterator
<code>get_yield_curve()</code> / <code>build_yield_curve()</code>	شیء <code>YieldCurve</code>
توابع Black-Scholes	عدد، tuple یا <code>dict</code> مطابق تابع

برای DataFrame‌های جدید، خروجی خالی همان `columns` و `dtypes` خروجی غیرخالی را نگه می‌دارد. `metadata`‌های مهم مانند منبع، `freshness`، پوشش، `partial` بودن و بودجه درخواست در `df.attrs` قرار می‌گیرند:

```
df = att.get_live_market("فملی")
print(df.attrs)
```

```
{
  'trade_date': datetime.date(...),
  'exchange_time': '12:28:41',
  'fetched_at': Timestamp(..., tz='Asia/Tehran'),
  'is_realtime_fresh': True,
  'is_partial': False,
  'missing_selectors': []
}
```

قیمت پایانی و آخرین معامله

- در feed زنده: `Last` آخرین قیمت معامله و `Close` قیمت پایانی TSETMC است.
- در تاریخچه سهام: `Close` آخرین قیمت معامله روز است؛ `Final` قیمت پایانی است و فقط در `output_type="full"` دیده می‌شود.
- با `auto_adjust=True` (پیش‌فرض)، `OHLC` و `Final` در صورت حضور، تعدیل شده‌اند.
- با `Close`، `auto_adjust=False` و `Final` خام‌اند و `Adj Close` نیز ارائه می‌شود.

- هنگام **Last** ، **include_today=True** زنده به **Close** تاریخچه و **Close** زنده به **Final** تاریخچه نگاشت می‌شود. بنابراین صرفاً بر اساس نام ستون بین **live** و **history join** ننزید.

تاریخ، ردیف امروز و freshness

- می‌پذیرند (2024-07-22) یا میلادی (1403-05-01) شمسی ISO شامل دو سر بازه‌اند و تاریخ **end** و **start** رفتار قبلی حفظ شده است: **include_today=False** هیچ درخواست زنده اضافه‌ای انجام نمی‌دهد.
- قدیمی snapshot . می‌کند **append/replace** معتبر همان روز معاملاتی را **observation** فقط **include_today=True** خواهد داشت **live_is_realtime_fresh=False** همان روز ممکن است برای تکمیل تاریخچه پذیرفته شود ولی
- توقف نماد، نبود معامله، نبود هویت قطعی یا شکست **live** باعث جعل ردیف امروز نمی‌شود؛ تاریخچه موفق برمی‌گردد و هشدار/attrs دلیل را نشان می‌دهند.
- مانند **server-side** ادعا نمی‌کند. تاریخچه **backfill** محلی فقط از زمان ضبط شما پوشش دارد و **history** قرارداد جداگانه دارد **price/trades/order-book**.

تقارن API زنده/تاریخی:

داده	زنده	تاریخی
قیمت و نمای نماد	<code>get_live_symbol</code> , <code>get_live_market</code>	<code>get_history(include_today=...)</code>
حقیقی/حقوقی	<code>get_live_market</code> , <code>get_market_client_type</code>	<code>get_client_type(include_today=...)</code>
معاملات ریز	<code>get_live_trades</code>	<code>get_trades</code>
پنج سطح سفارش	<code>get_order_book</code>	<code>get_order_book_history</code>
صف	<code>get_queue</code>	<code>get_queue_history</code>
فاندامنتال snapshot	<code>get_market_fundamentals</code>	<code>get_market_fundamentals_history</code>
اخزا و YTM	<code>get_treasury_yields</code>	<code>get_treasury_yield_history</code>
منحنی بازده	<code>get_yield_curve</code>	<code>get_yield_curve_history</code>
اختیار	<code>get_option_market</code>	<code>get_option_history</code> و snapshot های <code>save_option_snapshot</code> / <code>load_option_snapshots</code>
overview/breadth/sector/message/state	متناظر live های helper	یا <code>archive_to</code> پس از <code>*_history</code> های helper snapshot

تقارن به معنی یکسان بودن منبع نیست: بعضی **history** ها **server-side** هستند و بعضی فقط **observation** های ذخیره شده کاربر را می‌خوانند. **NoBackfill** , **Source** و **coverage** را بررسی کنید.

ذخیره در CSV

در API های تاریخی **save_path** , legacy مسیر پوشه است، نه نام فایل. نام فایل از نماد/دارایی ساخته می‌شود:

```
att.get_history(
    "فملی", save_to_file=True, save_path="exports", progress=False
)
# exports/فملی.csv
```

source boundary

درخواست‌های runtime فقط به providerهای پشتیبانی‌شده TSETMC، صفحه مرجع YTM فرابورس ایران (ifb.ir) و API قدیمی ارز/سکه TGJU محدودند. redirect به origin دیگر پیش از درخواست دوم رد می‌شود. هیچ API کدال در این پکیج وجود ندارد.

قطعی و قابل استناد منتشر نمی‌کند. اختلاف قیمت تعدیل‌شده و تعدیل‌نشده **معادل DPS**، برای رخداد تعدیل قیمت TSETMC. استنتاج نمی‌کند DPS **سود نقدی نیست** و کتابخانه از آن

حل دقیق هویت نماد

نمادهای تکراری، ابزار منقضی، حق تقدم، اختیار و اوراق می‌توانند نام مشابه داشته باشند. APIهای جدید از **InstrumentRef** و **InsCode** استفاده می‌کنند:

```
ref = att.resolve_instrument("فملی", asset_type="equity")
print(ref)
```

خروجی نماینده:

```
InstrumentRef(
    ins_code='35425587644337450', symbol='فملی',
    name='ملی صنایع مس ایران', asset_type='equity',
    is_active=True, provenance='tsetmc_search_exact', selector='فملی'
)
```

برای اجرای قطعی در **ins_code**، production، بدهید:

```
prices = att.get_history(
    ins_code="35425587644337450", asset_type="equity", progress=False
)
live = att.get_live_symbol(ins_code="35425587644337450")
```

را می‌پذیرد و نتیجه را به صورت رشته برمی‌گرداند؛ ASCII عدد صحیح مثبت دقیق یا رشته ۱ تا ۲۰ رقم **validate_ins_code()** بیش از یک نتیجه معتبر داشته باشد، selector رقم فارسی/عربی، صفر و مقدار مبهم پذیرفته نمی‌شوند. اگر، **bool**، **float** کتابخانه حدس نمی‌زند:

```
try:
    att.resolve_instrument("نماد تکراری")
except att.AmbiguousSymbolError as exc:
    print(exc)
```

AmbiguousSymbolError: selector matched more than one instrument; pass ins_code

گزینه‌های اصلی:

```
att.resolve_instrument(
    selector=None,
    ins_code=None,
    asset_type="auto",
    snapshot=None,
    require_active=True,
)
```

- دوباره را می‌دهد fetch بدون resolve امکان `snapshot`
- برای تاریخچه ابزار منقضی مناسب است `require_active=False`
- هویتی fuzzy join و فاصله‌های متداول را یکسان می‌کند، اما `ک/ک`، `ی/ی` اختلاف `normalize_instrument_text()` انجام نمی‌دهد.

قیمت و حقیقی/حقوقی؛ تاریخچه و زنده

`get_history()`

```
att.get_history(
    symbol="فملی", start=None, end=None, limit=0,
    raw=False, auto_adjust=True, output_type="standard",
    date_format="jalali", progress=True, save_to_file=False,
    dropna=True, adjust_volume=False, return_type=None,
    ascending=True, save_path=None, include_today=False,
    ins_code=None, asset_type="auto",
)
```

```
history = att.get_history(
    "فملی", limit=10, include_today=True,
    output_type="full", progress=False,
)
print(history.tail(2))
print(history.attrs["include_today_appended"])
```

خروجی نماینده:

	Open	High	Low	Close	Final	Volume	No.	Value
Date								
1405-06-02	73100	74600	72800	74200	73950	1820043	3912	1.35e+11
1405-06-03	74000	74900	73600	74700	74420	814220	1830	6.06e+10
['فملی']								

در `standard` ستون‌ها `Open, High, Low, Close, Volume` هستند. `full` ستون‌های `Final, No., Value` و اطلاعات تقویم/Ticker را نیز اضافه می‌کند. `raw=True` فرمت TSE را برمی‌گرداند. `return_type` یکی از `simple`، `log`، `both` یا فرم سفارشی مانند `['simple', 'Close', 5]` است. برای چند نماد، `DataFrame` با `MultilIndex` ستونی برمی‌گردد.

`get_client_type()`

```
client = att.get_client_type(
    "فملی", limit=30, include_today=True,
    output_type="full", progress=False,
)
print(client.tail(1))
```

خروجی نماینده ردیف امروز:

	No_buy_retail	Vol_buy_retail	No_sell_retail	Vol_sell_retail	Power_retail	Is_partial
Date						
1403-05-07	1284	920000	991	701000	1.24	True

ردیف امروز `volume/count` را از `ClientTypeAll` می‌گیرد. `value`های امروز در صورت نیاز با `VWAP` بازار برآورد می‌شوند و ستون‌های `Value_source` / `Is_estimated` در خروجی `opt-in` مشخص می‌کنند که مقدار رسمی یا برآوردی است. رفتار پیش‌فرض و `schema` قدیمی بدون `include_today` تغییر نکرده است.

live کل بازار و یک نماد

```
snapshot = att.get_market_snapshot()      # dict سازگار با market_watch()
live_all = att.get_live_market()          # DataFrame ادغام شده
live_one = att.get_live_market("فملی")
point = att.get_live_symbol("فملی", fallback="none")
```

می‌کند. ستون‌های کلیدی `join` و بهترین سفارش‌ها را `client type`، قیمت `get_live_market()`

```
InsCode, Symbol, Name, Last, Close, PreviousClose, Volume, Value,
IndividualBuyVolume, IndividualSellVolume, LegalBuyVolume, LegalSellVolume,
IndividualPower, NetIndividualVolume, EstimatedNetIndividualFlow,
BidPrice1, AskPrice1, Spread, SpreadBps, L1Imbalance, L5Imbalance,
trade_date, exchange_time, fetched_at, is_realtime_fresh, is_partial
```

freshness در `get_live_market()` attr عمومی است، نه row-wise ستون audit schema/selection واقعی برای attrs هستند:

```
print(live_all.attrs.keys())
print(live_all.attrs["missing_selectors"])
```

```
dict_keys(['field_validity', 'source_schema_presence', 'migration', 'missing_selectors'])
[]
```

می‌دهد `StockNotFoundError` در `MarketWatch` در نبود نماد در `get_live_symbol(..., fallback="none")` دقیقاً یکی از `SnapshotSource` را امتحان می‌کند؛ ستون TSETMC نقطه‌ای endpoint فقط در آن حالت `fallback="point"` است. `PresentInMarketWatch`, `FallbackReason`, `IdentityVerified`, `has_trade_today` و `PriceActionable` قابلیت استفاده قیمت را شفاف می‌کنند.

پنج‌سطحی دارد long خروجی `get_order_book()`

```
InsCode Symbol Level BidOrderCount BidVolume BidPrice AskPrice AskVolume AskOrderCount
...      31          124000      74200      74100      180000      42          1      فملی
```

ستون‌های metadata آن شامل `trade_date`, `exchange_time`, `fetched_at`, `snapshot_age_seconds` و `is_partial`, `is_realtime_fresh`, `is_stale` است.

معاملات ریز

همان قرارداد را `get_live_trades()` استفاده می‌کند و TSETMC تاریخچه معاملات endpoint برای هر روز از `get_trades()` برای روز جاری ارائه می‌دهد:

```
trades = att.get_trades(
    "فملی",
    start="1403-05-01",
    end="1403-05-03",
    include_canceled=False,
    max_requests=10,
    raw=False,
    progress=False,
)

live_trades = att.get_live_trades(
    ins_code="35425587644337450",
    include_canceled=False,
    max_requests=1,
    progress=False,
)
```

استاندارد schema:

InsCode, Symbol, GregorianDate, JalaliDate, TradeNo, Time, Timestamp,
Price, Volume, Value, Canceled, Source

خروجی نماینده:

	TradeNo	Time	Price	Volume	Value	Canceled	Source
Timestamp							
...	1	09:01:01	50000	100	5000000	False	tsetmc_trade_history_lossless
...	2	09:01:01	50010	50	2500500	False	tsetmc_trade_history_lossless

را نیز نگه `RawCanceled` و `RawDate` ، `RawInsCode` ، `iSensVarP` ، `qTitNgJ` مانند provider فیلدهای خام `raw=True` های مهم `attrs` می‌دارد.

```
request_count, trade_request_count, max_requests, request_budget_scope,
partial, failures, source_coverage, reconciliation,
resolved_ins_code, resolved_symbol
```

- ممکن است هزینه جداگانه داشته باشد و resolver است؛ trade endpoint سقف سخت درخواست‌های `max_requests` مشخص می‌کند شمارش کل شناخته شده است یا نه `attrs`.
- شکست بخشی از روزها با `partial=True` و جزئیات `failures` برمی‌گردد؛ داده موجود دور ریخته نمی‌شود.
- رد می‌شود `DataParsingError` پاسخ، تاریخ یا فیلدهای عددی با `InsCode` در mismatch
- رکورد ابطال فقط با `include_canceled=True` نگه داشته می‌شود.

سفارش و صف

سفارش زنده

```
book = att.get_order_book("فملی")
queue = att.get_queue("فملی", side="both", strict=True)
```

قیمت صف را با `strict=True` قطعی `False` است، نه `NA` صف را سه حالته گزارش می‌کند؛ نبود شواهد کافی `get_queue()` بازار تطبیق می‌دهد state دامنه مجاز و

```
InsCode Symbol Side QueuePrice QueueVolume QueueOrders QueueValue
...      فملی   buy      73500      820000      163  6.027e+10
```

```
is_queue=True, is_strict=True, is_partial=False,
threshold_source='market_watch_price_limits', book_source='market_watch_live_snapshot'
```

تاریخچه پنج سطح سفارش

```
books = att.get_order_book_history(
    "فملی",
    limit=20,
    output_type="standard", # long ردیف در هر long
    include_today=True,
    complete_only=True,
    max_requests=25,
    progress=False,
)

wide = att.get_order_book_history(
    "فملی", limit=10, output_type="wide", max_requests=25, progress=False
)
```

داده تاریخی `BestLimits` یک `delta stream` روزانه است. کتابخانه `state` هر روز را مستقل و به ترتیب `hEven/refID` بازسازی می‌کند؛ مقدار صفر پاک شدن `level` است. `schema long`:

```
InsCode, Symbol, Name, Date, GregorianDate, JalaliDate, Time, Timestamp,
DEven, hEven, refID, _sequence, Level,
BidOrderCount, BidVolume, BidPrice, AskPrice, AskVolume, AskOrderCount,
is_partial, is_complete, market_partial_status, is_reconstructed,
is_stale, record_type, source
```

خروجی نماینده:

Timestamp	Level	BidPrice	BidVolume	AskPrice	AskVolume	is_complete	source
2024-07-22 09:05:01+03:30	1	74100	180000	74200	124000	True	tsetmc_best_limits_history_reconstructed
2024-07-22 09:05:01+03:30	2	74050	95000	74250	88000	True	tsetmc_best_limits_history_reconstructed

برابر source زنده معتبر را با snapshot `include_today=True` ناقص را حذف می‌کند `complete_only=True`. سقف سخت است و `max_requests` اضافه یا جایگزین می‌کند `market_watch_live_snapshot`. پوشش ناقص را توضیح می‌دهند `attrs['request_count']` و `attrs['failed_requests']`. نام alias `get_orderbook_history` عینی `get_order_book_history` است.

تاریخچه صف

```
queues = att.get_queue_history(
    "فملی",
    limit=20,
    include_today=True,
    complete_only=True,
    side="both",
    strict=True,
    max_requests=25,
    progress=False,
)
```

schema:

```
InsCode, Symbol, Name, Date, GregorianDate, JalaliDate, Time, Timestamp,
DEven, hEven, Side, QueuePrice, QueueVolume, QueueOrders, QueueValue,
Pricelimit, is_strict, is_queue, is_partial, is_complete,
market_partial_status, book_state, is_crossed, is_preopen_or_stopped,
threshold_hEven, threshold_source, book_source, source
```

اگر static threshold تاریخی در دسترس نباشد، `threshold_source='unavailable'` و `is_queue=NA` می‌ماند. این رفتار برای backtest مهم است: نبود داده به «صف نبود» تبدیل نمی‌شود.

Watcher و تحلیل کل بازار

بازار iterator

```
for event in att.watch_market(
    symbol="فملى",
    interval=2,
    max_updates=5,
    notifications=("messages", "state"),
):
    print(event.kind, event.sequence, event.fetched_at, event.changed_inscodes)
```

شامل event. وجود ندارد `kind="update"` است. `initial` , `delta` , `heartbeat` , `resync` یکی از `MarketEvent.kind` است. `snapshot`، `persistence` و وضعیت `retry`، `cursor`، `level` تغییرکرده، `InsCode` تغییرکرده فهرست، `snapshot`،

برای کنترل کامل:

```
watcher = att.MarketWatcher(
    symbol=None,
    interval=2,
    max_updates=None,
    include_initial=True,
    emit_heartbeats=True,
    notifications=("messages", "state"),
    notification_top=50,
    error_policy="retry",
    max_consecutive_retries=5,
    max_backoff=30,
    callback_error_policy="raise",
)
```

`resync` محدود `backoff` را نگه می‌دارد و در قطع ارتباط با `cursor`، استفاده می‌کند `MarketWatchInit/Plus` از `Watcher` هستند `state` و `messages` ها فقط `notification` می‌شود.

پیام، وضعیت و نمای بازار

```
messages = att.get_market_messages(flow=0, top=20, since_id=None)
states = att.get_instrument_state_changes(top=20, since_id=None)
overview = att.get_market_overview(flow=0)
```

پیام schema:

```
message_id, date, time, timestamp, title, description, flow
```

وضعیت schema:

```
event_id, date, time, timestamp, InsCode, Symbol, Name,
state_code, state, real_time, under_supervision, state_title
```

رسمی payload برمی‌گرداند؛ مجموعه ستون‌ها به `flow` را با ستون `overview` رسمی `raw` فیلدهای `get_market_overview()` وابسته است و به تعداد ثابتی از ستون‌ها متعهد نیست `provider`.

breadth و جریان صنایع

```
breadth = att.get_market_breadth(traded_only=True)
sectors = att.get_sector_flow(traded_only=True)
```

خروجی نماینده breadth:

instrument_count	advances	declines	unchanged	ad_difference	ad_ratio	total_volume	upper_limit_count
lower_limit_count	612	318	241	53	77	1.32	8.91e+09
18							42

ستون‌های breadth علاوه بر موارد بالا شامل `no_trade`, `missing_previous`, `missing_current_price` درصد صعود/نزول، `total_value`, `trade_date`, `exchange_time`, `fetch_at`, `is_realtime_fresh` است.

همراه با پوشش حقیقی/حقوقی ارائه می‌کند `SectorCode` را برای هر breadth همین `get_sector_flow()`:

SectorCode	instrument_count	advances	declines	client_coverage	net_individual_volume
estimated_net_individual_value	27	43	25	13	0.91
5.8e+10	market_vwap	False			820000

freshness و `client_coverage`, `value_method`, `value_available` برآورد است؛ `estimated_net_individual_value` در استراتژی لحاظ کنید.

فیلترهای مشترک عبارت‌اند از `instrument_types` و `symbol`, `flow`, `sector`, `traded_only`, `include_base_market` برای محاسبه چند خروجی روی یک مشاهده، snapshot را یک‌بار دریافت و به مسیر خصوصی `_snapshot` ندهید؛ API عمومی `save_market_snapshot()` یک snapshot اتمیک می‌سازد و history derivation را هم‌زمان نگه می‌دارد.

تقویم، نمای کلان، مستر ابزار و پیش‌گشایش

برای تقویم، TSETMC های این بخش دو گروه داده را یکجا در نسخه 1.4.0 فراهم می‌کنند: داده‌های عمومی و بدون حساب API قابل دریافت است TSETMC رسمی که فقط با اشتراک وب‌سرویس TOP ارزش بازار، اثر شاخص، خلاصه روزانه و مستر ابزار؛ و

تقویم معاملاتی بورس و فرابورس

```
calendar = att.get_trading_calendar(
    start="1405-06-01",
    end="1405-06-18",
    market="all",
    include_closed=True,
)
print(calendar.tail())
```

تاریخ شمسی یا `end` و `start` است. `farabourse` / `ifb` یا `bourse` / `tse`، یکی از `all`، `market` در حالت `include_closed` آخرین جلسه هر بازار را برمی‌گرداند. پارامتر `limit`، میلادی و شامل دو مرز هستند. اگر بازه ندهید روزهای تقویمی بین دو مرز را نیز می‌سازد و `True` فقط جلسات منتشرشده شاخص مرجع را می‌آورد؛ مقدار `False` پیش‌فرض می‌گذارد `IsTradingDay=False`.

```
Market, Flow, GregorianCalendar, JalaliDate, IsTradingDay,
ReferenceIndex, ReferenceInsCode, Source
```

تعریف روز معاملاتی در این تابع «وجود مشاهده در تاریخچه شاخص کل رسمی همان بازار» است. بنابراین تعطیلی عادی یا نبود جلسه مشخص می‌شود، اما برنامه ساعات جلسه و علت تعطیلی از این منبع استخراج نمی‌شود. این معنا در `attrs['calendar_semantics']` ثبت شده است.

ارزش و فعالیت روزانه بازار

```
market_value = att.get_market_value_history(
    start="1405-01-01",
    market="all",
    limit=90,
)

activity = att.get_market_activity(
    start="1405-06-15",
    end="1405-06-18",
    market="tse",
    max_requests=10,
    progress=False,
)
```

`get_market_value_history()` برای هر بازار `MarketValue` دو ستون مشتق‌شده `Change` و `ChangePct` را به ریال و دو ستون مشتق‌شده `MarketValue` برای هر بازار `get_market_value_history()` مشخص باشد تا ۵۰۰۰ مشاهده از منبع درخواست می‌شود و سپس بازه فیلتر می‌شود؛ در حالت بدون بازه `start` می‌دهد. اگر سقف دریافت است `limit`.

```
Market, Flow, GregorianCalendar, JalaliDate, MarketValue,
Change, ChangePct, Source
```

بودجه سخت ۱ تا ۳۶۶ است و `max_requests` انجام می‌دهد bulk برای هر جلسه یک درخواست `get_market_activity()` `InstrumentCount` , `TradedInstrumentCount` , `TradeCount` , `Volume` , `Value` , `AverageTradeValue` است. این آمار از خلاصه رسمی روزانه ابزارها ساخته می‌شود؛ `AverageTradeValue = Value` و `Value` است `NaN` و در نبود معامله `/ TradeCount`

اثر نمادها بر شاخص

```
impact = att.get_index_impact(
    date="1405-06-18",
    market="tse",
    top=10,
    direction="both",
)
print(impact[["Rank", "Symbol", "Impact", "AbsSharePct"]])
```

`both` فقط `direction` . عدد صحیح مثبت تا ۱۰۰۰ است `top` . آخرین تاریخ دارای ارزش بازار را انتخاب می‌کند `date=None` , سهم قدرمطلق هر ردیف از مجموع قدرمطلق `AbsSharePct` عدد رسمی اثر و `Impact` . را می‌پذیرد `negative` یا `positive` . نیست مگر اینکه منبع تمام ردیف‌ها را برگرداند universe **ردیف‌های بازگردانده شده همان بازار است**؛ این درصد سهم از کل

خلاصه bulk همه ابزارها در یک روز

```
daily = att.get_market_trades(
    date="1405-06-18",
    market="all",
    progress=False,
)
leaders = daily.nlargest(10, "Value")
```

هر ردیف یک خلاصه روزانه یک هویت ابزار است و ستون‌های `TradeCount` , `Volume` , `Value` , `Open` , `High` , `Low` , `Close` , `ChangePct` و `Last` , `PreviousClose` , `Change` را دارد. نام قدیمی سرویس رسمی `TradeOneDayAll` است، اما این خروجی **معاملات ریز و شماره‌دار نیست**. برای معامله به معامله از `get_trades()` یا `get_live_trades()` استفاده کنید.

ردیف‌های `market='all'` فیلتر می‌شوند. در ISIN با نگاشت قطعی خانواده پیشوند `market='tse'` و `market='ifb'` و `MarketClassification` جریان‌های دیگر، بورس کالا، انرژی یا طبقه‌بندی نشده نیز حفظ می‌شوند و کیفیت اتصال در `attrs['unknown_market_rows']` اما پیشوندهایی که بازارشان به‌تنهایی قابل اثبات fuzzy دیده می‌شود. این روش `progress` . می‌مانند `unknown` یا `other` نیست عمداً فقط پیام پایان را کنترل می‌کند

مستر ابزارها و کشف تغییرات

```
master = att.get_instrument_master(
    active=None,
    market=None,
    asset_type=["stock", "fund", "bond"],
    progress=False,
)
master.to_parquet("instrument-master-1405-06-18.parquet", index=False)

changes = att.get_instrument_changes(
    previous="instrument-master-1405-06-18.parquet",
    current=None,
    progress=False,
)
```

در صورت استفاده یکی از `market` می‌خواند `InsCode` را با کلید پایدار TSETMC جدول عمومی `get_instrument_master()` از `stock`, `right`, `fund`, `bond`, `option`, `future`, `asset_type` یک رشته یا `iterable` از `stock`, `right`, `fund`, `bond`, `option`, `future`, `mortgage`, `commodity`, `energy`, `unknown` است. پارامتر `active` با مقدار `None` و `True` یا `False` درخواست زنده اضافه ندارد؛ مقدار وضعیت فعالیت را نامعلوم نگه می‌دارد و `snapshot` مستر را با `True` یا `False` درخواست زنده اضافه ندارد؛ مقدار `MarketWatch` لحظه‌ای `snapshot` مستر را با `True` یا `False` درخواست زنده اضافه ندارد؛ مقدار

InsCode, ISIN, Symbol, Name, EnglishName, CompanyCode, CompanyISIN,
Market, MarketGroup, Industry, AssetType, InstrumentStatus, Active, Source

یا مسیر `DataFrame` مقایسه می‌کند. هر ورودی می‌تواند `current` قبلی را با `snapshot` یک `get_instrument_changes()` `current=None` داشته باشد `InsCode` باشد و باید ستون CSV, Parquet, JSON/JSONL است. `Field`, `OldValue`, `NewValue` و ستون‌های `added`, `removed`, `changed` `ChangeType` های با long-form خروجی است. `backfill` این تابع تاریخچه را از سرور های خودتان را نگه دارید `snapshot` نمی‌کند؛ برای مقایسه روزهای گذشته باید `backfill` این تابع تاریخچه را از سرور

TOP رسمی و عدم تعادل پیش‌گشایش

```
top = att.get_theoretical_opening_price(
    username="TSETMC_WEB_SERVICE_USERNAME",
    password="TSETMC_WEB_SERVICE_PASSWORD",
    market="tse",
    timeout=20,
    progress=False,
)

imbalance = att.get_preopen_imbalance(top_data=top)
print(imbalance.nlargest(10, "ImbalanceRatio"))
```

کاربرد ندارد. PyPI هستند؛ حساب عادی سایت یا حساب TSETMC مشخصات اشتراک وب‌سرویس `username` و `password` به سرویس رسمی می‌فرستد و ذخیره نمی‌کند. برای امنیت، رمز را در کد HTTPS پکیج آن‌ها را فقط در همان فراخوانی از راه مهلت درخواست بر حسب ثانیه است `all` و `timeout` بازار یا `market`. قرار ندهید `repository` یا `log`، `notebook`.

```
Market, Flow, InsCode, Symbol, Name, GregorianCalendar, JalaliDate, Time,
TheoreticalOpeningPrice, TheoreticalVolume, RemainderSide, RemainderVolume,
TheoreticalBuyVolume, TheoreticalBuyPrice, TheoreticalSellPrice,
TheoreticalSellVolume, ChangePct, Source
```

اگر قبلاً TOP را گرفته‌اید، آن را با `top_data` به `get_preopen_imbalance()` بدهید تا درخواست دیگری انجام نشود. اگر `top_data=None` باشد، همان تابع با `username`, `password`, `market`, `timeout` و `progress` داده رسمی را می‌گیرد. خروجی سه ستون اضافه دارد:

```
ImbalanceVolume = TheoreticalBuyVolume - TheoreticalSellVolume
ImbalanceRatio = ImbalanceVolume / (TheoreticalBuyVolume + TheoreticalSellVolume)
ImbalanceSide = buy | sell | balanced | unavailable
```

این نسبت صرفاً عدم تعادل حجم‌های TOP رسمی است و سیگنال خرید یا فروش محسوب نمی‌شود. در مجموع حجم صفر یا داده ناقص، نسبت `NaN` و سمت `unavailable` است.

شاخص‌ها و تحلیل صنایع

صنعت در نسخه 1.2.2 دو مفهوم را از هم جدا می‌کند API:

- **عضویت رسمی شاخص:** نمادهایی که endpoint رسمی `GetIndexCompany` برای همان شاخص برمی‌گرداند. توابع این فصل به‌طور پیش‌فرض از این universe استفاده می‌کنند.
- **گروه دیده‌بان:** دسته‌بندی سریع `SectorCode` در MarketWatch که مبنای `get_sector_flow()` است و الزاماً با اعضای رسمی شاخص برابر نیست.

پارامترهای اختصاصی این خانواده عبارت‌اند از `industry`, `industries`, `include_member_count`, `include_live`, `include_client_type`, `include_orderbook`, `include_empty`, `days`, `metric`, `top`, `refresh`, `max_workers` و ورودی‌های عمومی `interval` و `progress`, `start`, `end`, `limit`, `ascending` همان معنای توضیح داده‌شده در هر تابع را دارند.

هویت عضوها فقط با `InsCode` متصل می‌شود. نام مشابه یا جست‌وجوی fuzzy برای join استفاده نمی‌شود. همچنین بعضی شاخص‌ها تجمیعی‌اند و عضویت صنایع می‌تواند هم‌پوشانی داشته باشد؛ بنابراین جمع‌زدن ردیف‌های تمام صنایع، کل بازار بدون تکرار تولید نمی‌کند.

مسیر سریع برای کاربر معمولی

```
import algotik_tse as att

# شاخص صنعت؛ سریع و تنها با یک درخواست ۴۵
indices = att.list_industry_indices(progress=False)

# اعضای رسمی یک صنعت همراه داده جاری بازار
members = att.get_industry_members("فلزات اساسی", progress=False)

# تحلیل همه صنایع همراه حقیقی/حقوقی snapshot
snapshot = att.get_industry_snapshot(progress=False)

# همپوشانی رسمی اعضا برای شناسایی همپوشانی بین صنایع
overlap = att.get_industry_membership_overlap(
    industries=["بانک", "خودرو", "فلزات"],
    progress=False,
)

# تغییرات عضویت رسمی هر روز در بازه کوتاه
membership_events = att.get_industry_membership_events(
    industries=["بانک", "خودرو"],
    days=7,
    progress=False,
)

# breadth پنج صنعت برتر از نظر
leaders = att.rank_industries(
    metric="breadth",
    top=5,
    progress=False,
)
```

فهرست شاخص‌های صنعت

```
indices = att.list_industry_indices(
    progress=False,
    include_member_count=True,
    refresh=False,
    max_workers=6,
)
```

تهی می‌مانند. با `HasMembers` و `MemberCount` پیش‌فرض و سریع است؛ در این حالت `include_member_count=False` عضویت را دور می‌زند `cache refresh=True` می‌شود `cache` عضویت رسمی همه صنایع دریافت و در حافظه، `True` درخواست‌های عضویت است `concurrency` و فقط سقف `1..16` عدد صحیح `max_workers`

schema:

```
IndustryName, IndustryNameEn, IndustryGroupCode, IndexInsCode,
IndexValue, IndexPreviousValue, DayHigh, DayLow,
IndexChange, IndexChangePct, MemberCount, HasMembers, ExchangeTime
```

نسخه‌های `list_indices()` درصد تغییر است. این نگاشت در `IndexChangePct` تغییر واحد شاخص و `IndexChange` برعکس بود و در 1.2.0 اصلاح شده است `<=1.1.3`

اعضای دقیق یک شاخص

```
members = att.get_industry_members(
    industry="بانک",
    include_live=True,
    include_client_type=True,
    include_orderbook=True,
    progress=False,
    refresh=False,
)
```

ورودی `industry` می‌تواند نام فارسی، alias شناخته‌شده مانند `بانک`، صورت کامل مانند `شاخص صنعت بانکها` یا `IndexInsCode` باشد.

- ادغام می‌کند MarketWatch جاری snapshot اعضای رسمی را با `include_live=True`
- شده قابل reconcile قدرت خریدار و جریان حقیقی/حقوقی را اضافه می‌کند؛ فقط ردیف‌های `include_client_type=True` استفاده‌اند.
- `include_live=True` و صف تخمینی پنج سطح را اضافه می‌کند و به `spread`، `imbalance` و `include_orderbook=True` نیاز دارد.
- می‌گیرد TSETMC عضویت را دوباره از `refresh=True`

ثابت خروجی schema

```
IndustryName, IndustryIndexCode, InsCode, Symbol, Name,
SectorCode, Flow, MarketCode, InstrumentType,
PreviousClose, Open, High, Low, Close, Last, Change, ChangePct,
TradeCount, Volume, Value, SharesOutstanding, EstimatedMarketCap,
IndividualPower, NetIndividualVolume, EstimatedNetIndividualFlow,
ClientDataAvailable, BidPrice1, AskPrice1, SpreadBps,
L1Imbalance, L5Imbalance,
EstimatedBuyQueueVolume, EstimatedBuyQueueValue,
EstimatedSellQueueVolume, EstimatedSellQueueValue,
TradeDate, ExchangeTime, FetchedAt, IsRealtimeFresh, IsStale
```

ستون‌های مربوط به client type یا order book وقتی درخواست نشده‌اند nullable می‌مانند. `EstimatedMarketCap` حاصل `SharesOutstanding × Close` و `EstimatedNetIndividualFlow` برآورد مبتنی بر VWAP است؛ هیچ‌کدام وزن رسمی شاخص نیستند.

در `DataFrame.attrs` مواردی مانند `universe='exact_index_members'` , `membership_is_current` , `historical_membership_available` , `cache_hit` , `client_type_requested` , `orderbook_requested` و `membership_count` ثبت می‌شوند.

snapshot تحلیلی صنعت

برای یک صنعت:

```
metals = att.get_industry_snapshot(
    industries="فلزات اساسی",
    include_client_type=True,
    include_orderbook=True,
    include_empty=False,
    progress=False,
    refresh=False,
    max_workers=6,
)
```

برای چند صنعت:

```
selected = att.get_industry_snapshot(
    industries=["بانک", "خودرو", "شیمیایی"],
    include_client_type=True,
    progress=False,
)
```

با `industries=None` تمام صنایع بررسی می‌شوند. `include_empty=False` شاخص‌هایی را که provider در آن لحظه عضو ندارند حذف می‌کند؛ نام آن‌ها در `attrs['empty_industries']` باقی می‌ماند. این تابع فقط یک MarketWatch bulk، حداکثر یک client-type bulk و عضویت‌های cache شده را مصرف می‌کند.

گروه‌های اصلی خروجی:

گروه	ستون‌ها
وضعیت شاخص	<code>IndexValue</code> , <code>IndexPreviousValue</code> , <code>IndexChange</code> , <code>IndexChangePct</code> , <code>IndexDayHigh</code> , <code>IndexDayLow</code>
breadth	<code>MemberCount</code> , <code>TradedCount</code> , <code>Advances</code> , <code>Declines</code> , <code>Unchanged</code> , <code>NoTrade</code> , <code>AdvanceDeclineRatio</code> , <code>AdvancePct</code> , <code>DeclinePct</code>
عملکرد اعضا	<code>EqualWeightReturn</code> , <code>MedianReturn</code> , <code>ReturnDispersion</code>
معاملات	<code>TotalTradeCount</code> , <code>TotalVolume</code> , <code>TotalValue</code> , <code>MarketValueSharePct</code> , <code>UpperLimitCount</code> , <code>LowerLimitCount</code>
حقیقی/حقوقی	<code>ClientCoveredCount</code> , <code>ClientCoverage</code> , <code>NetIndividualVolume</code> , <code>EstimatedNetIndividualValue</code> , <code>IndividualPower</code> , <code>FlowValueMethod</code>

گروه	ستون‌ها
سفارش و صف	<code>OrderBookCoveredCount</code> , <code>OrderBookCoverage</code> , <code>BuyQueueCount</code> , <code>BuyQueueValue</code> , <code>SellQueueCount</code> , <code>SellQueueValue</code>
زمان و تازگی	<code>TradeDate</code> , <code>ExchangeTime</code> , <code>FetchAt</code> , <code>IsRealtimeFresh</code> , <code>IsStale</code>

`IndividualPower` میانگین ساده بازده اعضای دارای قیمت معتبر است و بازده رسمی شاخص نیست `EqualWeightReturn`.
 های سازگار با client row از سرانۀ تجمیعی خرید حقیقی به سرانۀ تجمیعی فروش حقیقی ساخته می‌شود. جریان پول فقط از تاز و متعلق به روز order book را کنار آن کنترل کنید. صف‌ها فقط برای `ClientCoverage` حجم بازار جمع می‌شود؛ همیشه تهران محاسبه می‌شوند.

در `attrs`: `memberships_may_overlap=True` , `membership_is_current=True` , `historical_membership_available=False` , تعداد cache hit/request و روش تخمین جریان پول ثبت می‌شود.

همپوشانی اعضای رسمی صنعت‌ها

```
industry_overlap = att.get_industry_membership_overlap(
    industries=["بانک", "خودرو", "فلزات"],
    progress=False,
)
```

این تابع همپوشانی رسمی اعضای هر جفت صنعت را در `GetIndexCompany` محاسبه می‌کند. خروجی long-form است و برای هر جفت unordered یک ردیف دارد:

```
IndustryAName, IndustryAIndexCode, IndustryBName, IndustryBIndexCode,
CommonMembers, UnionMembers, Jaccard, OverlapA_Pct, OverlapB_Pct
```

است و union تعداد منحصر به فرد اعضای `UnionMembers`، تعداد نمادهای مشترک در دو صنعت را می‌دهد `CommonMembers`.
 همپوشانی نسبت به اندازه هر `OverlapA_Pct` و `OverlapB_Pct` است. `CommonMembers/UnionMembers` نسبت `Jaccard` صنعت را نشان می‌دهند و برای شناسایی همپوشانی‌های نامتقارن مفید هستند.

رویدادهای عضویت point-in-time برای صنایع

```
membership_events = att.get_industry_membership_events(
    industries=["بانک", "خودرو"],
    days=30,
    progress=False,
)
```

این تابع اختلاف اعضای رسمی هر صنعت را بین تاریخ‌های متوالی در `GetIndexCompany` payload نشان می‌دهد. خروجی long-form است و برای هر تغییر، ردیف با `EventType` برابر `added` یا `dropped` می‌آورد:

IndustryName, IndustryIndexCode, TradeDate, JalaliDate, PreviousTradeDate,
PreviousJalaliDate, EventType, InsCode, Symbol, Name

یعنی در تاریخ قبلی `EventType="dropped"` یعنی نماد در تاریخ فعلی ظاهر شده و در تاریخ قبلی نبود `EventType="added"` معتبر است `days=1..30` وجود داشته اما در تاریخ فعلی حذف شده است. این نگاه در حال حاضر فقط برای بازه کوتاه

تاریخچه شاخص و اعضا

تاریخچه خود شاخص:

```
history = att.get_industry_history(
    industry="شیمیایی",
    start="1404-01-01",
    end=None,
    limit=120,
    ascending=True,
    progress=False,
)
```

یعنی `limit=0` هستند. فیلتر تاریخ ابتدا اعمال می شود و `YYYY-MM-DD / YYYYMMDD` تاریخ شمسی یا میلادی `start` و `end` جلسه را نگه می دارد. خروجی N بدون محدودیت؛ مقدار مثبت آخرین

است. منبع رسمی برای `IndustryName, IndustryIndexCode, TradeDate, JalaliDate, High, Low, Close, Change, ChangePct` روزانه نمی دهد؛ ستون حجم مصنوعی ساخته نمی شود و `volume` شاخص صنعت `attrs['volume_available']=False` است.

تاریخچه کوتاه همه اعضای فعلی:

```
member_history = att.get_industry_members_history(
    industry="خودرو",
    days=30,
    ascending=True,
    progress=False,
    refresh=False,
)
```

long-form رسمی است. خروجی payload و تعداد آخرین تاریخ معاملاتی موجود در `1..30` عدد صحیح `days`

IndustryName, IndustryIndexCode, TradeDate, JalaliDate,
InsCode, Symbol, Name, Close, Last, Change, ChangePct,
TradeCount, Volume, Value

این history برای اعضای فعلی شاخص است، نه عضویت point-in-time. اگر ترکیب شاخص تغییر کرده باشد، survivorship bias محتمل است؛ به همین دلیل attrs صریح `point_in_time_membership=False` و `survivorship_bias_possible=True` دارد.

داده درون‌روزی شاخص صنعت

```
intraday = att.get_industry_intraday(
    industry="فلزات",
    interval="5min",
    progress=False,
)
```

مقادیر مجاز `interval` عبارت‌اند از `raw`، `1min`، `5min`، `15min`، `30min`، `60min`، `1h`. حالت `raw` هر مشاهده provider را به صورت یک کندل تک‌نقطه‌ای نگه می‌دارد؛ سایر حالت‌ها سطح شاخص را به OHLC تبدیل می‌کنند.

```
IndustryName, IndustryIndexCode, Timestamp, JalaliDate, Interval,
Open, High, Low, Close, Change, ChangePct
```

مصنوعی تولید نمی‌کند و فقط `turnover` یا `volume` حجم ندارد؛ تابع `endpoint` تهران است. این `timezone` دارای `Timestamp` را برمی‌گرداند provider آخرین روز موجود در

رتبه‌بندی صنایع

```
ranking = att.rank_industries(
    metric="money_flow",
    top=10,
    ascending=False,
    include_client_type=True,
    include_orderbook=False,
    progress=False,
    refresh=False,
    max_workers=6,
)
```

های پشتیبانی‌شده `alias` و `metric`:

alias های رایج	معیار canonical
<code>change_pct</code> ، <code>return</code>	<code>IndexChangePct</code>
<code>equal_weight_return</code>	<code>EqualWeightReturn</code>
<code>median_return</code>	<code>MedianReturn</code>
<code>advance_pct</code> ، <code>breadth</code>	<code>AdvancePct</code>
<code>total_value</code> ، <code>turnover</code>	<code>TotalValue</code>
<code>estimated_net_individual_value</code> ، <code>money_flow</code>	<code>EstimatedNetIndividualValue</code>
<code>individual_power</code> ، <code>buyer_power</code>	<code>IndividualPower</code>

ردیف اول را نگه می‌دارد N همه ردیف‌های دارای معیار معتبر را برمی‌گرداند؛ مقدار مثبت فقط `top=None` `ascending=False` دارد و معیار نهایی در `Rank` را همراه snapshot رتبه بزرگ‌تر به کوچک‌تر است. خروجی تمام ستون‌های ثبت می‌شود `attrs['ranking_metric']`

مقایسه و همبستگی شاخص‌های صنعت

```
compare = att.compare_industries(
    industries=["خودرو", "بانک", "فلزات"],
    metric="close",
    limit=20,
    ascending=True,
    progress=False,
)
```

در خروجی، برای هر تاریخ یک ردیف است و هر ستون یک شاخص مقایسه‌شده است. ستون‌های خروجی به این شکل هستند:

TradeDate, JalaliDate, صنعت فلزات [ID], صنعت بانک [ID], صنعت خودرو [ID]

```
relative = att.get_industry_relative_strength(
    industries=["بانک", "شیمیایی", "فلزات"],
    benchmark="خودرو",
    metric="close",
    limit=30,
    ascending=False,
    progress=False,
)
```

این خروجی long-form است و برای هر industry (به جز benchmark) ستون‌های `IndustryReturn`, `BenchmarkReturn`, `RelativeStrength` را تولید می‌کند.

TradeDate, JalaliDate, BenchmarkIndexCode, BenchmarkName,
IndustryIndexCode, IndustryName, IndustryReturn, BenchmarkReturn, RelativeStrength

```
corr = att.get_industry_correlation(
    industries=["بانک", "خودرو", "صنعت فولاد", "ساختمان"],
    limit=120,
    ascending=False,
    progress=False,
)
```

خروجی تابع ماتریس همبستگی $n \times n$ است با `index/columns` برچسب‌دار به فرمت `IndustryName [IndexCode]`.

هر یک از توابع بالا همانند زیر امضای رسمی‌شان را دارند:

```
compare_industries(industries, start=None, end=None, limit=0, metric='close', ascending=True,
progress=True, max_workers=6)
get_industry_relative_strength(industries, benchmark, start=None, end=None, limit=0, metric='close',
ascending=True, progress=True, max_workers=6)
get_industry_correlation(industries, start=None, end=None, limit=0, ascending=True, progress=True,
max_workers=6)
get_industry_membership_overlap(industries, progress=True, refresh=False, max_workers=6)
get_industry_membership_events(industries, days=30, progress=True, refresh=False, max_workers=6)
```

cache، پوشش و محدودیت داده

عضویت کامل شاخص و تاریخچه کوتاه اعضا در یک payload مشترک می‌آیند. TTL پیش‌فرض cache حافظه یک ساعت است:

```
att.settings.industry_membership_cache_ttl = 3600.0
```

مقدار cache 0 را غیرفعال می‌کند. برای دریافت اجباری عضویت تازه از `refresh=True` استفاده کنید. cache فقط در حافظه همان process است و فایلی روی دیسک نمی‌نویسد.

محدودیت‌های رسمی این نسخه:

- وزن رسمی هر عضو، ضریب سهام شناور و divisor شاخص در منبع فعلی ارائه نمی‌شود؛ contribution دقیق نماد به واحد شاخص محاسبه نمی‌شود.
- فعلی ساخته می‌شود و ممکن است در صورت تغییر ترکیب universe بر پایه `get_industry_members_history()` داشته باشد؛ برای تغییرات دقیق اعضا از `survivorship` شاخص، بایاس استفاده کنید.
- صنایع می‌توانند هم‌پوشانی داشته باشند و بعضی کدهای صنعت ممکن است در یک روز بدون عضو باشند.
- برآوردند و نام آن‌ها عمداً `EstimatedNetIndividualValue` و `EstimatedNetIndividualFlow` ، `EstimatedMarketCap` این موضوع را نشان می‌دهد.
- برای تصمیم معاملات، `TradeDate` ، `OrderBookCoverage` ، `ClientCoverage` ، `IsStale` ، `IsRealtimeFresh` و attrs را بررسی کنید.

تاریخچه محلی SQLite

ذخیره‌سازی کاملاً opt-in است؛ بدون path صریح هیچ فایلی نوشته نمی‌شود.

snapshotهای بازار

```
db = "market-history.sqlite"

snapshot_id = att.save_market_snapshot(db)
history = att.load_market_snapshots(db, symbol="فملی", limit=500)

# alias ذخیره شده live معنایی برای همان نمای
live_history = att.get_live_market_history(db, symbol="فملی", limit=500)
breadth_history = att.get_market_breadth_history(db, limit=100)
sector_history = att.get_sector_flow_history(db, limit=100)
overview_history = att.get_market_snapshot_summary_history(db, limit=100)
```

خروجی نماینده:

AsOf	InsCode	Symbol	Last Close	Volume	NoBackfill	SnapshotAtomic
2026-08-25 10:11:12+03:30	...	812200	73950	74200	فملی	True

و `CoverageStart`، `CoverageEnd`، `SnapshotFrameAttrs`، `NoBackfill`، `SnapshotAtomic` و `SourceAtomic` هایمانند `attrs` را توضیح می‌دهند `archive` محدود و کیفیت `SourceAtomic`، `application-id`، `schema version`، `SQLite` دارای `transaction` است خراب است.

record_to برای replay eventها

```
watcher = att.MarketWatcher(
    interval=2,
    max_updates=3,
    record_to=db,
    checkpoint_interval=100,
    record_max_records=10_000,
    record_retention_seconds=7 * 24 * 3600,
    storage_error_policy="raise",
)
for _ in watcher:
    pass

events = att.get_market_event_history(db, kind="delta", limit=1000)
```

اولین `event` هر `session` یک `checkpoint` کامل است؛ `delta`ها پس از آن `replay` می‌شوند و `pruning` فقط از مرز `checkpoint` معتبر انجام می‌شود. `storage_error_policy="raise"` انتخاب امن پیش فرض است. `record_market_event()` برای ثبت مستقیم `MarketEvent` موجود است.

archive_to برای رکوردهای مستقل

دو مسیر مستقل اند `record_to` و `archive_to`

```
att.get_market_overview(flow=0, archive_to=db)
att.get_market_messages(flow=0, top=20, archive_to=db)
att.get_instrument_state_changes(top=20, archive_to=db)

overview = att.get_market_overview_history(db, flow=0, limit=100)
messages = att.get_market_messages_history(db, flow=0, limit=100)
states = att.get_instrument_state_changes_history(db, symbol="فملی", limit=100)
```

های پشتیبانی شده است kind سطح پایین برای API `archive_market_records(path, kind, frame, source=...)` اعمال SQL در pagination قبل از `start/end`, `limit/offset` هستند؛ archive بخشی از هویت `source` و `selector` می شوند.

توابع مهم نگهداری:

```
info = att.check_market_history(db)
att.record_market_event(db, event, session_id="strategy-a")
```

عمومی اند migration/inspection برای `MARKET_HISTORY_SCHEMA_VERSION` و `MARKET_HISTORY_APPLICATION_ID`

فاندامنتال بازار، صندوق و تعدیل قیمت

EPS و P/E زنده

```
fundamentals = att.get_market_fundamentals(
    symbols=["فملی", "شتران"],
    pe_min=None,
    pe_max=None,
    positive_pe=False, # نگه دار audit را هم برای unavailable ردیف
    strict=False,
    allow_stale=False,
    archive_to="market-history.sqlite",
    max_requests=1,
    progress=False,
)
```

این تابع یک bulk snapshot می گیرد و EPS/P/E را از همان مشاهده می سازد؛ برای هر نماد سراغ منبع دیگری نمی رود.
schema ثابت:

```
InsCode, Symbol, Name, SectorCode, InstrumentType, Close, Last,
EPS, PE, PECalculated, EPSSource, PriceSource, PESTatus,
TradeDate, ExchangeTime, AsOf, SnapshotAgeSeconds,
IsRealtimeFresh, IsStale, Source, NoLookahead
```

خروجی نماینده:

Symbol	Close	EPS	PE	PECalculated	EPSSource	PriceSource	PEStatus	NoLookahead
10.0	7395	73950	فملى	True	market_watch	Close	ok	True
42100	شتران	<NA>	<NA>	True	missing_in_market_watch	Close	eps_missing	True

EPS صفر/خالی به مقدار جعلی تبدیل نمی‌شود. `PEStatus` یکی از `ok` , `price_missing` , `price_nonfinite` , `price_nonpositive` , `eps_missing` , `eps_nonfinite` , `eps_nonpositive` است؛ بودن در `stale` جداسازی `IsStale` snapshot. مفقود را به خطا تبدیل می‌کند `selector` مثبت را نگه می‌دارد و فقط `positive_pe=True` زنده `attrs` می‌شود `attrs['stale_rejected']=True` نیست: خروجی تهی و `exception` با `allow_stale=False` `stale` دقیقاً شامل

`request_count,max_requests,missing_selectors,strict,stale_rejected,source,price_source,eps_source,no_loo`
`oahead,no_backfill,archive_path` است.

تاریخچه fundamentals

```
history = att.get_market_fundamentals_history(
    "market-history.sqlite",
    start="1403-05-01",
    end="1403-05-07",
    symbols="فملى",
    limit=1000,
)
print(history.attrs["no_lookahead"], history.attrs["current_eps_used"])
```

AsOf	Symbol	Close	EPS	PE	EPSSource	Source	NoLookahead
2026-08-24 10:00:00+03:30	10.0	7350	73500	فملى	market_watch	local_market_snapshot	True
2026-08-25 10:00:00+03:30	10.0	7420	74200	فملى	market_watch	local_market_snapshot	True

این `history` فقط از `snapshot` های ذخیره شده شما ساخته می‌شود: `attrs['no_backfill']=True` و `attrs['current_eps_used']=False` دیگر `missing_selectors,strict,source,price_source,eps_source,no_loo`
`ahead,coverage_start,coverage_end` هستند. EPS فعلی برای گذشته `forward-fill` نمی‌شود.

صندوق‌های قابل معامله

```
listed = att.list_listed_funds(progress=False)

gold = att.list_listed_funds(
    fund_category="commodity",
    commodity_underlying="gold",
    progress=False,
)
silver = att.list_listed_funds(commodity_underlying="silver", progress=False)

sector_registry = att.list_funds(fund_type="sector", progress=False)
leveraged_registry = att.list_funds(fund_type="leveraged", progress=False)
index_registry = att.list_funds(fund_type="index", progress=False)
```

و (صندوق بورسی عمومی) `InstrumentType=305` هر دو گروه bulk call با یک `list_listed_funds()` NAV، صندوق‌ها می‌آید و اطلاعاتی مثل registry از `list_funds()` را می‌گیرد (صندوق کالایی بورسی) `InstrumentType=380` بازده، ترکیب دارایی و مدیر را برمی‌گرداند. این دو منبع شناسه مشترک قطعی ندارند و با شباهت نام به هم متصل نمی‌شوند «کالایی» مترادف «طلا» نیست و ETF نیز سازوکار معامله است، نه طبقه دارایی. طلا، نقره و زعفران فقط از نگاشت دقیق `InsCode+ISIN` یا عبارت صریح در نام رسمی تشخیص داده می‌شوند. عباراتی مثل «زر»، «گوهر» یا «سیمین» به‌تنهایی مدرک نیستند. اگر مدرک کافی وجود نداشته باشد، مقدار `unknown` برمی‌گردد و `needs_review=True` می‌شود؛ پوشش زیرنوع‌ها ممکن است کامل نباشد.

فهرست بورسی schema

```
InsCode, ISIN, Symbol, Name, InstrumentType, Flow, MarketCode,
Last, Close, Yesterday, Volume, Value, TradeCount, Low, High, NAV,
NAV_Discount, Change, ChangePct, fund_category, asset_exposure,
strategy_tags, trading_mechanism, unit_class, commodity_profile,
commodity_underlyings, primary_commodity, classification_status,
classification_source, classification_evidence, taxonomy_version, needs_review
```

Symbol	InsCode	ISIN	Last	Close	NAV	NAV_Discount	Volume
افران	...	IR03AFRZ0001	21650	21620	...	...	1250040

در خروجی registry نیز `registry_type_id`, `registry_category`, `asset_exposure`, `strategy_tags`, `classification_status`, `classification_source`, `classification_evidence`, `taxonomy_version`, `record_date` و `needs_review` در کنار ستون‌های قبلی حفظ می‌شوند. ثابت `FUND_TAXONOMY_VERSION` نسخه مرجع طبقه‌بندی را نشان می‌دهد. `attrs`های `no_fuzzy_join=True` و `registry_joined=False` قرارداد هویتی را روشن می‌کنند.

رخدادهای تعدیل قیمت

```
adjustments = att.get_price_adjustments(
    "فملی", start="1402-01-01", end="1403-12-29", progress=False
)
latest = att.get_latest_price_adjustment("فملی", progress=False)
```

typed ثابت و schema:

```
InsCode, Symbol, GregorianCalendar, JalaliDate,
AdjustedClosingPrice, UnadjustedClosingPrice, AdjustmentAmount,
CorporateTypeCode, CorporateActionType, IsConfirmedDPS,
IdentityVerified, Source, FetchedAt
```

خروجی نماینده:

GregorianCalendar	JalaliDate	AdjustedClosingPrice	UnadjustedClosingPrice	AdjustmentAmount	CorporateTypeCode
CorporateActionType	IsConfirmedDPS	IdentityVerified			
2024-01-01	1402-10-11	8000	10000	2000	7
<NA>	False	True			

قرارداد مهم:

- فقط اختلاف ریاضی قیمت‌هاست $\text{AdjustmentAmount} = \text{UnadjustedClosingPrice} - \text{AdjustedClosingPrice}$
- provider خام `CorporateTypeCode` است؛ nullable `CorporateActionType` و `False` همیشه `IsConfirmedDPS` بدون تفسیر نگه داشته می‌شود.
- ثبت می‌شود `attrs['dps_reason']` و دلیل در `attrs['dps_available'] == False`
- رکورد با `InsCode` متفاوت رد می‌شود؛ نبود `InsCode` در خود رکورد با `IdentityVerified=False` و `InsCode` حل شده گزارش می‌شود.
- می‌دهد `schema/dtypes/attrs` صفر یا یک‌ردیفی با همان `DataFrame` `get_latest_price_adjustment()`

```
assert adjustments["IsConfirmedDPS"].eq(False).all()
assert adjustments.attrs["dps_available"] is False
```

از این API برای ساخت «سری DPS» استفاده نکنید. TSETMC در این endpoint طبقه‌بندی قطعی سود نقدی ارائه نمی‌دهد.

اخزا و درآمد ثابت

قاعده تاریخ در نماد اخزا

شش رقم انتهای نماد به صورت تاریخ شمسی `YYMMDD` تفسیر می‌شود؛ دو رقم سال با `13` یا `14` تکمیل می‌شود:

```
att.parse_treasury_maturity("020322 اخزا")
```

```
{
  'maturity_jalali': '1402/03/22',
  'maturity_gregorian': datetime.date(2023, 6, 12),
  'maturity_source': 'user_confirmed_symbol_jalali_yymdd'
}
```

```
att.parse_treasury_maturity("991117 اخزا")
```

```
{
  'maturity_jalali': '1399/11/17',
  'maturity_gregorian': datetime.date(2021, 2, 5),
  'maturity_source': 'user_confirmed_symbol_jalali_yymdd'
}
```

برای نماد non-match یا تاریخ شمسی نامعتبر، تابع exception نمی‌دهد و **None** برمی‌گرداند. در صورت تعارض یا نماد غیرقابل تفسیر، **maturity_date** یا **maturity_map** صریح بدهید؛ provenance ستون **MaturitySource** را بررسی کنید.

محاسبات deterministic

```
result = att.treasury_yield(
    price=800_000,
    maturity_date="2028-01-01",
    settlement_date="2026-01-01",
    face_value=1_000_000,
)
```

خروجی واقعی این مثال:

```
EffectiveAnnualYield  0.1180339887
SimpleAnnualYield     0.125
ContinuousYield       0.1115717757
BankDiscountYield     0.0986301370
MacaulayDuration      2.0
ModifiedDuration       1.7888543820
Convexity              4.8
DV01                  143.10835056
DiscountFactor         0.8
Status                 ok
```

برای اوراق کوپنی:

```
cashflows = [
    ("2027-01-01", 80_000),
    ("2028-01-01", 80_000),
    ("2029-01-01", 1_080_000),
]
price = att.bond_price(0.08, cashflows, settlement_date="2026-01-01")
ytm = att.yield_to_maturity(price, cashflows, settlement_date="2026-01-01")
risk = att.bond_analytics(price, cashflows, settlement_date="2026-01-01")
```

`bond_price`، `day_count_fraction()` پشتیبانی می‌کند `ACT/365F`، `ACT/360`، `30/360` از قراردادهایی مانند `yield_to_maturity` و `bond_analytics` و `compounding/frequency`، `clean/dirty price` و `accrued interest` پارامترهای `sign-changing` های مبهم یا `cashflow` دارند؛

اخزای زنده

```
ytm = att.get_treasury_yields(
    symbol=None,
    settlement_date=None,
    face_value=1_000_000,
    include_stale=False,
    min_volume=1,
    price_source="auto", # auto/bid/ask/last/close
    strict=False,
    source="tsetmc", # مرجع join برای hybrid یا
)
```

ستون‌های اصلی:

```
InsCode, ISIN, Symbol, MaturityJalali, Maturity, MaturitySource,
MaturityConflict, SettlementDate, DaysToMaturity, Tenor,
Price, PriceSource, NoTrade, InstrumentPriceAsOf, IsInstrumentStale,
BidPrice, AskPrice, DiscountFactor,
EffectiveAnnualYield, ContinuousYield, SimpleAnnualYield, BankDiscountYield,
MacaulayDuration, ModifiedDuration, Convexity, DV01,
Volume, Value, TradeCount, FaceValue, FaceValueSource,
DayCount, IsStale, SnapshotAgeSeconds, FetchedAt, Status
```

مقایسه `https://ifb.ir/ytm.aspx` را با جدول مرجع مجاز فرابورس ایران در TSETMC داده معاملات `source="hybrid"` گزارش می‌شود با provenance bps جای قیمت ابزار را نمی‌گیرد و اختلاف در IFB می‌کند؛

```
ifb = att.get_ifb_yield_table(category="treasury")
print(ifb.attrs["reference_url"])
```

schema IFB:

Symbol, Price, LastTradeJalali, LastTradeDate, PublishJalali, PublishDate,
MaturityJalali, Maturity, Volume, ReferenceYTM,
ReferenceSimpleYield, ReferenceSource

تاریخچه YTM

```
history = att.get_treasury_yield_history(
    "اخز ۰۹۰۱۰۱",
    limit=20,
    include_today=True,
    price_source="close",
    max_requests=25,
    progress=False,
)
```

عینی همین تابع است. هر ردیف از قیمت همان روز ساخته می‌شود و `get_treasury_yields_history` alias است. `OHLC` مناسب نیست. ستون‌های تاریخچه شامل YTM است؛ قیمت تعدیل‌شده برای `auto_adjust=False`، `Close`، `Final`، `price_source` از همان `include_today`، سررسید/قیمت است `provenance` و `Status`، `IsPartial`، `FetchedAt`، `analytics`، استفاده می‌کند `live` متناظر `source`.

منحنی بازده

```
curve = att.get_yield_curve(
    min_nodes=3,
    interpolation="log_discount",
    duplicate_policy="volume_weighted",
    extrapolate=False,
)

df_18m = curve.discount_factor(1.5)
zero_18m = curve.zero_rate(1.5)
fwd = curve.forward_rate(1.0, 2.0)
```

`YieldCurve` شامل `settlement_date`، `maturities`، `times`، `discount_factors`، `continuous_zero_rates`، `node_metadata` و `diagnostics` است. `extrapolate=False` را می‌گیرد.

ساخت مستقیم:

```
nodes = [
    {"Maturity": "2027-01-01", "DiscountFactor": 0.90},
    {"Maturity": "2028-01-01", "DiscountFactor": 0.80},
    {"Maturity": "2029-01-01", "DiscountFactor": 0.70},
]
curve = att.build_yield_curve(nodes, settlement_date="2026-01-01")
```

و `curve.diagnostics` دقیقاً کلیدهای `input_node_count` , `node_count` , `duplicate_count` , `duplicate_policy` و `monotonic_discount_enforced` discount factor نزولی بودن `guard` و `dedupe`، ورودی `audit` برای `metadata` را دارد؛ این است.

تاریخچه منحنی:

```
curves = att.get_yield_curve_history(
    limit=10,
    min_nodes=3,
    include_today=True,
    max_requests=50,
    progress=False,
)
```

هر node دارای `CurveID` , `CurveStatus` , `CurveNodeCount` , `CurveInterpolation` و flagهای `CurvePricesNoLookahead` , `CurveUniverseNoLookahead` , `CurveNoLookahead` است. اگر `universe` تاریخی از `catalog` امروز بازیابی شود، survivor-bias در `diagnostics` صریح است و `CurveUniverseNoLookahead=False` می‌شود.

اختیار معامله

تحلیل‌های قیمت‌گذاری این بخش برای اختیار اروپایی هستند. مشخصات واقعی اعمال قرارداد، تعدیلات شرکتی و `dividend` از `yield` TSETMC حدس زده نمی‌شود.

ریاضی Black–Scholes

```
call = att.black_scholes_price(
    spot=100, strike=100, time_to_expiry=1,
    rate=0.05, volatility=0.20,
    option_type="call", dividend_yield=0.0,
)
greeks = att.black_scholes_greeks(100, 100, 1, 0.05, 0.20, "call")
bounds = att.option_price_bounds(100, 100, 1, 0.05, "call")
iv = att.implied_volatility(10.4505835722, 100, 100, 1, 0.05, "call")
```

خروجی واقعی و deterministic:

```
call = 10.4505835722

greeks = {
  'Delta': 0.6368306512,
  'Gamma': 0.0187620173,
  'Vega': 37.5240346917,
  'Vega1Pct': 0.3752403469,
  'ThetaPerYear': -6.4140275464,
  'ThetaPerDay': -0.0175726782,
  'Rho': 53.2324815454,
  'Rho100bp': 0.5323248155,
  'Status': 'ok'
}

bounds = (4.8770575499, 100.0)
iv = {'ImpliedVolatility': 0.1999999955, 'Status': 'ok', 'Iterations': 27}
```

واحدها مهم‌اند: **Vega** تغییر قیمت برای یک واحد volatility و **Vega1Pct** برای یک واحد درصد است؛ **ThetaPerYear/Day** و **Rho/Rho100bp** جدا گزارش می‌شوند. خروجی Greeks همیشه کلید **Status** دارد. **implied_volatility()** همواره **dict** می‌دهد و کلیدهای پایه آن **ImpliedVolatility**, **Status**, **Iterations** هستند؛ یکی از **ok**, **missing**, **expiry**, **out_of_bounds**, **no_bracket**, **non_converged** است. **out_of_bounds**, **no_bracket**, **non_converged** out-of-bounds/no-bracket ممکن است **LowerBound/UpperBound** و **non-converged** مقدار تشخیصی **CandidateVolatility** داشته باشد؛ عدم همگرایی exception نیست. فقط constraint های ورودی مانند **bracket/tolerance/style** نامعتبر **ValueError** می‌دهند.

snapshot اتمیک بازار اختیار

```
options = att.get_option_market(
    exchange=0,
    underlying="خودرو",
    max_requests=1,
    progress=False,
)
```

هر جفت call/put فقط وقتی metadata کافی و هویت سازگار داشته باشد وارد snapshot می‌شود. ستون‌های اصلی:

```
InsCode, PairID, PairSequence, ISIN, Symbol, Name, OptionType,
UnderlyingInsCode, UnderlyingSymbol, ContractSize, Strike,
BeginDate, EndDate, DaysToExpiry,
Last, Close, Yesterday, Volume, Value, TradeCount, NotionalValue,
OpenInterest, YesterdayOpenInterest, BidPrice, AskPrice, BidVolume, AskVolume,
UnderlyingLast, UnderlyingClose, Price, PriceSource,
AsOf, AsOfSource, SnapshotFreshnessKnown, PriceFreshnessKnown,
Stale, NoTrade, AnalyticsEligible, AnalyticsEligibilityReason,
MetadataConflict, Source
```

attrهای **atomic_snapshot**, **duplicate_pairs_dropped**, **incomplete_pairs_quarantined**, **exchange_event_freshness_known** و **malformed_pair_status** را توضیح می‌دهند snapshot کیفیت

API قدیمی `list_options(underlying=None)` فهرست قراردادها را می‌دهد و `get_options_chain(underlying, fetch_oi=False)` یک `dict` با کلیدهای دقیق `calls`, `puts`, `underlying_name`, `underlying_price`, `expiry_dates` و `market_time` برمی‌گرداند؛ کلید `price` وجود ندارد. برای `price` حرفه‌ای `analytics` وجود ندارد. پیشنهاد می‌شود `get_option_market()` استفاده شود.

IV، Greeks، parity و نقدشوندگی

```
analysis = att.analyze_option_chain(
    options=options,
    spot=None, # اگر معتبر باشد snapshot از
    risk_free_rate=0.30, # یا yield_curve=curve
    dividend_yield=0.0,
    valuation_date=None,
    exercise_style="european",
    parity_tolerance=None,
    allow_unverified_freshness=True,
)
```

ستون‌های تحلیلی افزوده‌شده:

```
TimeToExpiry, Spot, SpotSource, RiskFreeRate, RiskFreeRateSource,
DividendYield, DividendYieldSource,
ImpliedVolatility, ImpliedVolatilityBid, ImpliedVolatilityMid, ImpliedVolatilityAsk,
IVStatus, IVStatusBid, IVStatusMid, IVStatusAsk,
Delta, Gamma, Vega, Vega1Pct, ThetaPerYear, ThetaPerDay, Rho, Rho100bp,
PremiumContract, DeltaContract, GammaContract, VegaContract,
Vega1PctContract, ThetaPerYearContract, ThetaPerDayContract,
RhoContract, Rho100bpContract,
SpreadAbs, SpreadPct, QuotedDepth, LiquidityScore,
ParityResidual, ParityToleranceBand, ParityStatus, ImpliedForward,
GreeksStatus, AnalyticsReliability, AnalyticsWarning, AnalyticsComputed
```

خروجی نماینده:

Symbol	OptionType	Strike	Price	ImpliedVolatility	Delta	Vega1PctContract	SpreadPct	LiquidityScore
ParityStatus								
AnalyticsReliability								
ok verified_inputs	call	3000	420.0	0.41	0.62	18320.0	0.03	0.81
ok verified_inputs	put	3000	265.0	0.39	-0.38	17790.0	0.04	0.76

است و توصیهٔ آربیتراژ نیست؛ محدودیت وجه تضمین، سبک اعمال، کارمزد و امکان معامله را diagnostic فقط `ParityStatus` قابل اتکا ساخته نمی‌شود `analytics` معتبر نداشته باشید `curve` ندهید و `risk_free_rate` مدل نمی‌کند. اگر فقط زمانی ثبت می‌شود که کاربر مقدار را تعیین کرده باشد؛ مقدار صفر پیش‌فرض `DividendYieldSource='user_supplied'` نیست `DPS` به معنی کشف

PCR

```
pcr = att.option_put_call_ratios(analysis, group_by="underlying")
```

```
UnderlyingInsCode CallVolume PutVolume PCRVOLUME PCRVOLUMEStatus CallValue PutValue PCRValue
PCRValueStatus CallOpenInterest PutOpenInterest PCROpenInterest PCROpenInterestStatus
65883838195688438      20000      13000      0.65 ok          9.2e9      5.1e9      0.55      ok
40000          20000          0.50 ok
```

است. خروجی برای هر معیار علاوه بر `underlying_expiry` یا `expiry`، `underlying`، `market` دقیقاً یکی از `group_by` را می‌دهد؛ نسبت با `PCROpenInterestStatus` و `PCRValueStatus`، `PCRVOLUMEStatus` متناظر status، نسبت است. `zero_denominator` برابر status و nullable صفر denominator

تاریخچه اختیار و snapshot محلی

```
# opt-in قیمت قرارداد؛ ردیف امروز server-side تاریخچه
history = att.get_option_history(
    "..."، limit=10،
    include_today=True، max_requests=3، progress=False،
)

# snapshot بازار اختیار فقط با درخواست صریح کاربر روی فایل نوشته می‌شود
path = "option-snapshots.json"
att.save_option_snapshot(path، options=options)
saved = att.load_option_snapshots(path)
```

schema history:

```
Timestamp، InsCode، Symbol، Open، High، Low، Close، Last،
Volume، Value، TradeCount، OpenInterest،
BidPrice، AskPrice، BidVolume، AskVolume،
UnderlyingLast، UnderlyingClose، ContractSize، Strike، EndDate،
Price، PriceSource، Source، AsOf، Stale، NoTrade، AnalyticsEligible
```

attrهایی مانند `prices_no_lookahead`، `underlying_prices_no_lookahead`، `rates_no_lookahead`، `valuation_date_source`، `curve_applied` و `source_coverage` را برای backtest کنید. فایل `OPTION_SNAPSHOT_SCHEMA_VERSION` است dedupe و writer، write و قفل.

سایر API های بازار

این بخش قابلیت های قدیمی را در همان مرجع واحد نگه می دارد. API های legacy برای backward compatibility در دسترس اند و در بسیاری از خطاهای قدیمی `None` / پیام کنسول می دهند؛ API های جدید بیشتر از `typed exception` استفاده می کنند.

intraday

```
ticks_or_candles = att.get_intraday(
    "فملی",
    interval="1min",      # tick, 1min, 5min, 15min, 30min, 1h, 4h, 12h
    start="1403-05-01",   # برای امروز start/end حذف
    end="1403-05-03",
    progress=False,
)
```

خروجی candle معمولاً `Open, High, Low, Close, Volume` با `index` زمانی است. `tick` snapshot خام معاملات/قیمت را می دهد. `interval` بزرگتر از داده پایه `resample` می شود؛ تعطیلی و وقفه بازار را در محاسبه لحاظ کنید. نام های canonical بازه `1m`, `60min`, `4hour`, `240m`, `12h` و `tick`, `1min`, `5min`, `15min`, `30min`, `1h`, `4h` هستند؛ `alias` های عددی/کوتاه مانند `1m`, `60min`, `4hour`, `240m`, `12h` و `tick`, `1min`, `5min`, `15min`, `30min`, `1h`, `4h` و نیز `raw` / `ticks` پشتیبانی می شوند. این API رفتار legacy دارد: `interval` نامعتبر یا تاریخی که `validator` قدیمی نامعتبر تشخیص دهد را روی کنسول اعلام می کند و `None` برمی گرداند، نه اینکه عمدتاً `ValueError` قرار داده ای بدهد.

اطلاعات نماد و سهامدار

```
detail = att.get_detail("فملی")
info = att.get_info("فملی")
stats = att.get_stats("فملی")
shareholders = att.get_shareholders("فملی", include_id=True)
capital = att.get_capital_increase("فملی")
```

- می دهد `id` و ردیف `value` ستون، `key` برابر `index` کلید-مقدار با `DataFrame|None` یک `get_detail()`
- می دهند `key` برابر `index` کلید-مقدار با `DataFrame` `get_stats()` و `get_info()`
- شناسه را `include_id=True` تاریخچه روز را می دهد؛ `date` سهامداران فعلی و با `get_shareholders(date=None)` اضافه می کند.
- تاریخچه تغییر تعداد سهام/سرمایه را می دهد `get_capital_increase()`
- این توابع `ins_code` و `asset_type` keyword-only را نیز می پذیرند.

معرفی شرکت؛ API قدیمی unsupported

قدیمی مانده اند. معرفی ناشر داده `import/signature` فقط برای حفظ `stock_introduction()` و `get_introduction()` یا شبکه خطای زیر را می دهد `resolver` این پکیج قرار دارد؛ تابع همیشه پیش از `source boundary` کدال است و خارج از

```
try:
    att.get_introduction("فملی")
except att.UnsupportedDataSourceError as exc:
    print(exc)
```

UnsupportedDataSourceError: get_introduction/stock_introduction requires Codal data, which is outside algotik-tse's TSETMC market-data source boundary

برای اطلاعات TSETMC از `get_info()` و `get_detail()` استفاده کنید. helper افشای ناشر یا history آن در این پکیج وجود ندارد.

فهرست نمادها و ابزارها

```
symbols = att.get_symbols(
    bourse=True, farabourse=True, payeh=True,
    haghe_taqadom=False, sandogh=False, bonds=False, options=False,
    mortgage=False, commodity=False, energy=False,
    payeh_color=None, output="dataframe", progress=False,
)

etfs = att.list_etfs(progress=False)
bonds = att.list_bonds(progress=False)
funds = att.list_funds(fund_type="fixed_income", progress=False)
listed_funds = att.list_listed_funds(progress=False)
options = att.list_options(underlying="خودرو", progress=False)
indices = att.list_indices(progress=False)
members = att.get_index_companies("شاخص صنعت بانکها", progress=False)
industry_indices = att.list_industry_indices(progress=False)
industry_snapshot = att.get_industry_snapshot("بانک", progress=False)
```

بازار/نوع ابزار را نگه می‌دارد metadata `dataframe` فقط نام نمادها را می‌دهد؛ `get_symbols(output="list")` ها را صریح تنظیم flag asset-type، اشتباه universe است. برای جلوگیری از `قرمز`، `نارنجی`، `زرد` یکی از `payeh_color` کنید.

`list_bonds()` را می‌دهد NAV/discount برای سازگاری قبلی فقط صندوق‌های بورسی عمومی نوع ۳۰۵ و `list_etfs()` metadata `list_funds()` بالاست fixed-income های API دقیق اخزا در analytics اوراق و سررسید را فهرست می‌کند ولی `list_listed_funds()` دقیق و ابزارهای واقعا قابل معامله نوع ۳۰۵ و ۳۸۰ را با `list_listed_funds()` صندوق‌هاست؛ registry طبقه‌بندی قابل فیلتر می‌دهد.

شاخص‌ها

```
index_history = att.get_history("شاخص کل", limit=100, progress=False)
industry_history = att.get_industry_history("بانک", limit=100, progress=False)
members = att.get_industry_members("بانک", progress=False)
```

برای شاخص‌های عمومی همچنان `get_history()` را به‌کار ببرید. برای شاخص صنعت، API‌های اختصاصی فصل **شاخص‌ها و تحلیل صنایع** عضویت رسمی، تاریخچه اعضا، snapshot و intraday را یکدست ارائه می‌کنند. schema شاخص با سهام فرق دارد و volume جعلی ساخته نمی‌شود. `list_indices()` و `get_index_companies()` برای سازگاری با کد قدیمی حفظ شده‌اند.

ارز و سکه

این API legacy از TGJU استفاده می‌کند و برای backward compatibility حفظ شده است:

```
fx = att.get_currency(
    "dollar", start="1403-01-01",
    output_type="standard", date_format="jalali", progress=False,
)

coins = att.get_currency(["seke", "nim-seke"], limit=10, progress=False)
```

نام‌های انگلیسی دقیق:

```
dollar, euro, yuan, dirham, pound, lira,
dollar-sana-sell, dollar-sana-buy,
dollar-nima-buy, dollar-nima-sell,
dollar-sarafimelli-buy,
seke, seke-bahar-azadi, nim-seke, rob-seke, seke-gerami
```

نام‌های فارسی پشتیبانی‌شده شامل ربع، نیم سکه، سکه بهار آزادی، سکه، لیر، پوند، درهم، یوان، یورو، دلار، سکه گرمی، سکه و صورت‌های سنا/نیم در settings است. spelling کلیدها را دقیق رعایت کنید؛ نام سکه در API انگلیسی `seke` است، نه `sekke`.

خروجی standard ستون‌های `Open, High, Low, Close` دارد. multi-currency یک DataFrame با MultiIndex ستونی می‌دهد. `save_path` در اینجا نیز پوشه است.

تنظیمات و خطاها

تنظیمات singleton:

```

from algotik_tse import settings

settings.ssl_verify = True          # پیشفرض و توصیه شده
settings.timeout = 10              # ثانیه
settings.max_retries = 3
settings.retry_backoff_factor = 0.3
settings.rate_limit_delay = 0.3    # فاصله حداقل شروع درخواستها

settings.market_snapshot_freshness_seconds = 120.0
settings.market_clock_skew_tolerance_seconds = 5.0
settings.client_volume_consistency_tolerance = 0.05
settings.industry_membership_cache_ttl = 3600.0
settings.order_book_max_requests = 250
settings.trade_max_requests = 250

```

خراب CA فقط برای محیط کنترل شده با opt-out است. `True` پیشفرض TLS verification

```

from algotik_tse.http_client import safe_get

response = safe_get("https://cdn.tsetmc.com/...", verify=False)

```

این opt-out را سراسری نکنید. URLهای TSETMC همگی HTTPS هستند.

قرارداد دقیق : `safe_get(url, **kwargs)`

- نامجاز با URL/Codal path های مجاز باشد؛ provider و داخل HTTP(S) باید `url: str` `UnsupportedDataSourceError` از `rate-limit`، `session` و `I/O` پیش از
- فقط `headers=settings.headers`، `timeout=settings.timeout` و `verify=settings.ssl_verify` های default نداده باشد اعمال می شوند caller override وقتی
- هستند. نوع نادرست اولی/دومی wrapper پارامترهای خود `allow_redirects: bool=True` و `max_redirects: int=5` است `ValueError`، `max_redirects<0` و `TypeError`
- درخواست زیرین همیشه `allow_redirects=False` دارد. `redirect` فقط `(scheme,host,effective-` same-origin `port)` و `hop-by-hop` است؛ `UnsupportedDataSourceError` cross-origin و `loop` عبور از سقف `requests.exceptions.TooManyRedirects` می دهد.
- `redirect` در caller های header دوباره فرستاده نمی شود؛ `redirect` فقط روی درخواست اول اعمال می شود و روی `params` حفظ می شوند same-origin
- خطاهای transport خود `requests` بعد از `retry propagate` می شوند. `safe_get` به تنهایی روی `status 4xx/5xx` `raise_for_status()` نمی کند؛ API مصرف کننده باید پیش از `parse` آن را به خطای typed خود تبدیل کند.

سلسله مراتب خطا:

AlgotikTSEError

- └─ AmbiguousSymbolError
- └─ ConnectionError
- └─ DataParsingError
- └─ InvalidParameterError
- └─ StockNotFound
- └─ UnsupportedDataSourceError

legacy سطح بالای پکیج نیست. توابع export برای سازگاری داخلی وجود دارد ولی exceptions در مازول **RateLimitError** و پیام کنسول بدهند؛ قرارداد هر تابع را بررسی کنید **None** exception ممکن است به جای

الگوی امن:

```
try:
    df = att.get_price_adjustments(ins_code="35425587644337450")
except att.InvalidParameterError as exc:
    print("bad input", exc)
except att.AmbiguousSymbolError as exc:
    print("pass ins_code", exc)
except att.ConnectionError as exc:
    print("provider unavailable", exc)
except att.DataParsingError as exc:
    print("provider schema changed", exc)
except att.UnsupportedDataSourceError as exc:
    print("outside supported sources", exc)
```

مرجع تفصیلی همه توابع

در این بخش همه exportهای عمومی پوشش داده شده‌اند. هر ردیف API در کنار همان تابع، ورودی‌ها و گزینه‌های اختصاصی، خروجی، خطاهای اصلی و مثال را توضیح می‌دهد. جدول‌های پارامتر مشترک فقط تعریف اصطلاحات را یکسان نگه می‌دارند؛ signature دقیق هر تابع نیز بلافاصله پیش از جدول همان گروه آمده است.

نمای کلی exportها و نام‌های قدیمی

جدول زیر inventory کامل exportهای **algotik_tse.__all__** است. جزئیات خروجی در بخش موضوعی مربوط آمده است.

هویت، تنظیمات و خطا

کاربرد	Export
تنظیمات singleton شبکه/بازار	settings
هویت immutable ابزار	InstrumentRef

کاربرد	Export
نرمال سازی و حل دقیق هویت	<code>normalize_instrument_text</code> , <code>validate_ins_code</code> , <code>resolve_instrument</code>
خطاهای عمومی	<code>AlgotikTSEError</code> , <code>AmbiguousSymbolError</code> , <code>ConnectionError</code> , <code>DataParsingError</code> , <code>InvalidParameterError</code> , <code>StockNotFoundError</code> , <code>UnsupportedDataSourceError</code>

قیمت، trades، client type و اطلاعات نماد

عمومی Alias/legacy	Export canonical
<code>stock</code>	<code>get_history</code>
<code>stock_RI</code> , <code>stock_RL</code>	<code>get_client_type</code>
<code>stock_capital_increase</code>	<code>get_capital_increase</code>
<code>stock_intraday</code>	<code>get_intraday</code>
—	<code>get_trades</code> , <code>get_live_trades</code>
<code>stockdetail</code>	<code>get_detail</code>
<code>stock_information</code>	<code>get_info</code>
<code>stock_statistics</code>	<code>get_stats</code>
<code>stock_introduction</code> (unsupported همیشه)	<code>get_introduction</code> (unsupported همیشه)
<code>shareholders</code>	<code>get_shareholders</code>
<code>stocklist</code>	<code>get_symbols</code>
<code>currency_coin</code>	<code>get_currency</code>

پارامترهای قدیمی نیز پذیرفته می شوند: `stock` → `symbol` , `values` → `limit` , `tse_format` → `raw` , `multi_stock_drop` / `multi_currencies_drop` → `dropna` و `output_type="complete"` → `"full"` در مسیرهای مربوط. برای کد جدید نام canonical را به کار ببرید.

live، سفارش، watcher و history محلی

ها Export
<code>get_market_snapshot</code> , <code>get_market_client_type</code> , <code>get_order_book</code> , <code>get_live_market</code> , <code>get_live_symbol</code>
<code>get_order_book_history</code> , <code>get_orderbook_history</code> , <code>get_queue</code> , <code>get_queue_history</code>
<code>MarketEvent</code> , <code>MarketWatcher</code> , <code>watch_market</code>
<code>get_market_messages</code> , <code>get_instrument_state_changes</code> , <code>get_market_overview</code> , <code>get_market_breadth</code> , <code>get_sector_flow</code>
<code>MARKET_HISTORY_SCHEMA_VERSION</code> , <code>MARKET_HISTORY_APPLICATION_ID</code> , <code>check_market_history</code>

ها Export

`save_market_snapshot` , `load_market_snapshots` , `get_live_market_history`

`get_market_overview_history` , `get_market_snapshot_summary_history` , `get_market_breadth_history` ,
`get_sector_flow_history`

`record_market_event` , `get_market_event_history` , `archive_market_records`

`get_market_messages_history` , `get_instrument_state_changes_history`

`get_trading_calendar` , `get_market_activity` , `get_market_value_history` , `get_index_impact` ,
`get_market_trades`

`get_theoretical_opening_price` , `get_preopen_imbalance`

سه wrapper صفرآرگومان legacy نیز عمومی‌اند: `market_watch()` , `market_client_type()` , `market_data()` .
`market_data()` wrapper deprecated است؛ برای کد جدید `get_market_snapshot()` یا `get_live_market()` را انتخاب کنید.

fundamentals، تعدیل قیمت و ابزارها

ها Export

`get_market_fundamentals` , `get_market_fundamentals_history`

`get_price_adjustments` , `get_latest_price_adjustment`

`get_instrument_master` , `get_instrument_changes`

`list_options` , `get_options_chain` , `list_etfs` , `list_bonds` , `list_funds` , `list_listed_funds`

`list_indices` , `get_index_companies`

`list_industry_indices` , `get_industry_members` , `get_industry_snapshot` , `get_industry_membership_events`

`get_industry_history` , `get_industry_members_history` , `get_industry_membership_overlap` ,
`get_industry_membership_churn` , `get_industry_intraday` , `rank_industries`

درآمد ثابت

ها Export

`IRAN_TREASURY_FACE_VALUE` , `YieldCurve`

`parse_treasury_maturity` , `day_count_fraction` , `treasury_yield`

`bond_price` , `yield_to_maturity` , `bond_analytics` , `build_yield_curve`

`get_ifb_yield_table` , `get_treasury_yields`

`get_treasury_yield_history` , `get_treasury_yields_history`

`get_yield_curve` , `get_yield_curve_history`

اختیار معامله

ها Export

OPTION_SNAPSHOT_SCHEMA_VERSION

black_scholes_price, black_scholes_greeks, option_price_bounds, implied_volatility

get_option_market, analyze_option_chain, option_put_call_ratios

get_option_history, save_option_snapshot, load_option_snapshots

signature های پرکاربرد

```
get_live_market(symbol=None, *, strict=False)
get_live_symbol(symbol=None, *, ins_code=None, fallback="none")
get_order_book(symbol=None, *, selector_strict=False)
get_queue(symbol=None, side="both", strict=True, *, selector_strict=False)
get_trades(symbol=None, *, ins_code=None, start=None, end=None,
            include_canceled=False, max_requests=None, raw=False, progress=True)
get_market_messages(flow=0, top=20, since_id=None, *, archive_to=None)
get_instrument_state_changes(top=20, since_id=None, *, archive_to=None)
get_market_overview(flow=0, *, archive_to=None)
```

پارامترهایی که با `_` شروع می‌شوند seam داخلی تست‌اند و API کاربر محسوب نمی‌شوند، حتی اگر در `inspect.signature` دیده شوند.

روش خواندن مرجع تفصیلی

هر signature زیر عین خروجی پایدارشده `inspect.signature` است؛ فقط آدرس حافظه `safe_get` به خود نام `safe_get` نرمال شده است. نوع‌هایی که در signature قدیمی annotation ندارند در جدول پارامترها مشخص شده‌اند. مقدار `None` معمولاً یعنی «فیلتر/override اعمال نشود»، نه رشته `"None"`. پارامترهای مشترک فقط یک‌بار در واژه‌نامه زیر توضیح داده می‌شوند و هر مدخل API علاوه بر آن، `override` و `constraint` خاص خود را می‌گوید. «خطاها» خطاهای اصلی قرارداد است، نه فهرست همه خطاهای ممکن Python/pandas.

پارامترهای مشترک هویت، تاریخچه و خروجی

نام	معمول و default و type	معنا و constraint
<code>symbol</code> / <code>symbols</code>	<code>str</code> <code>Iterable[str]</code> <code>''</code> API <code>None</code> ؛ بسته به <code>None</code> یا	نماد فارسی، <code>InsCode</code> یا مجموعه آن‌ها؛ برای هویت مبهم <code>ins_code</code> بدهید. <code>symbols=None</code> یعنی کل universe.
<code>ins_code</code> / <code>inscode</code>	<code>str</code> <code>int</code> <code>None</code> = <code>None</code>	شناسه دقیق ۱ تا ۲۰ رقم ASCII؛ با <code>selector</code> متناقض مجاز نیست. فقط <code>inscode</code> spelling تاریخی reader وضعیت است.
<code>asset_type</code>	<code>str</code> = <code>'auto'</code>	تعیین provider نوع را از <code>auto</code> حل هویت؛ hint می‌کند.

نام	معمول default و type	معنا و constraint
<code>start , end , date</code>	<code>str date datetime None</code>	مرز شامل ابتدا/انتهای ISO شمسی یا میلادی در APIهای بازار. <code>date</code> سهامداران <code>None</code> یعنی آخرین مشاهده.
<code>limit</code>	ها reader یا <code>int = 0</code> یا <code>1000</code>	<code>0</code> در historyهای provider یعنی بدون محدودیت بعد از فیلتر؛ در SQLite حداکثر صفحه. منفی نامعتبر است.
<code>offset</code>	<code>int = 0</code>	بعد از فیلترها؛ نامنفی SQL offset.
<code>include_today</code>	<code>bool = False</code>	ردیف زنده امروز؛ ممکن است در ساعات opt-in بازار آخرین مشاهده همین لحظه باشد و provenance دارد.
<code>raw</code>	<code>bool = False</code>	ممکن order-book ؛ در provider نزدیک schema باشد delta/raw-mixed است.
<code>output_type</code>	<code>str = 'standard'</code>	خروجی؛ مقادیر دقیق تابعی اند schema (<code>standard / full</code> یا <code>long / wide</code>). است <code>full</code> قدیمی <code>alias complete</code> .
<code>date_format</code>	<code>str = 'jalali'</code>	شکل index/ستون تاریخ؛ <code>jalali</code> , <code>gregorian</code> , <code>both</code> در مسیرهای پشتیبانی شده.
<code>ascending</code>	<code>bool = True</code>	ترتیب زمانی خروجی بعد از فیلتر.
<code>progress</code>	<code>bool = True</code>	فقط پیام پیشرفت؛ در داده و schema اثر ندارد.
<code>save_to_file</code>	<code>bool = False</code>	ذخیره CSV opt-in.
<code>save_path</code>	<code>str Path None</code>	پوشه مقصد، نه نام فایل؛ بدون <code>save_to_file=True</code> نوشته نمی شود.
<code>dropna</code>	<code>bool = True</code>	حذف ردیف/ستون کاملاً تهی طبق قرارداد همان API؛ ردیف partial معنادار order-book حفظ می شود.
<code>return_type</code>	<code>str None</code>	history سازگاری برای انتخاب نوع خروجی در <code>alias</code> را <code>output_type</code> قیمت/ارز؛ برای کد جدید ترجیح دهید.
<code>max_requests</code>	<code>int None</code>	سقف سخت fan-out قبل/حین I/O؛ معنای دقیق شمارش در مدخل تابع آمده است.
<code>strict / selector_strict</code>	<code>bool</code>	را فعال match خطای داده/فیلتر بدون <code>strict</code> ؛ <code>selector_strict</code> انتخاب می کند؛ مبهم/گم شده را خطا می کند.
<code>allow_stale / include_stale</code>	<code>bool</code>	اجازه نگه داشتن snapshot/اوراق stale ؛ بودن همچنان در ستون/attrs گزارش می شود.
<code>archive_to , record_to , snapshot_path , path</code>	<code>str Path None</code>	فایل SQLite/JSON صریح؛ نوشتن فقط با opt-in. <code>path</code> در reader/writer اجباری است.

نام	معمول type و default	معنا و constraint
<code>kwargs</code>	های سازگاری keyword	فقط <code>alias</code> های مستند مانند <code>stock</code> , <code>values</code> , <code>tse_format</code> و نامهای انگلیسی keyword ; <code>stocklist</code> ناشناخته قرارداد عمومی نیست.
<code>_request</code> , <code>_snapshot</code> , <code>_client_type</code> , <code>_clock</code> , <code>_wait</code> , <code>_random</code> , <code>_recorded_at</code> , <code>_live_fetch</code> , <code>_request_budget_state</code>	داخلی seam	فقط تزریق deterministic در تست؛ برای مصرف عادی استفاده نشود و BC عمومی برای آن تضمین نمی‌شود.

پارامترهای مشترک live، watcher و SQLite

نام	type/default	معنا و constraint
<code>flow</code>	<code>int None = 0</code>	بازار/جریان provider: <code>None</code> در analytics یعنی بدون فیلتر.
<code>sector</code>	<code>str Iterable None</code>	فیلتر <code>SectorCode</code> .
<code>instrument_types</code>	<code>Iterable[int] None</code>	<code>300,303,309</code> ابزار؛ پیش‌فرض تحلیلی سهام universe
<code>traded_only</code>	<code>bool = False</code>	فقط ابزار دارای معامله را در denominator نگه می‌دارد.
<code>include_base_market</code>	<code>bool = True</code>	نوع 309 بازار پایه را در universe پیش‌فرض نگه می‌دارد.
<code>side</code>	<code>str = 'both'</code>	برای صف <code>both</code> یا <code>buy</code> , <code>sell</code>
<code>complete_only</code>	<code>bool = False</code>	فقط snapshotهای پنج‌سطح کامل.
<code>top</code>	<code>int = 20</code>	تعداد پیام/وضعیت در درخواست؛ مثبت و bounded.
<code>since_id</code>	<code>int str None</code>	فیلتر client-side رکوردهای پس از شناسه؛ cursor provider نیست.
<code>source</code>	<code>str = 'tsetmc'</code>	می‌تواند fixed-income ؛ در archive هویت/provenance باشد hybrid یا حالت مستند <code>tsetmc</code> .
<code>session_id</code>	<code>str None</code>	شناسه session watcher برای نوشتن/فیلتر replay.
<code>kind</code>	<code>str None</code>	kind یا <code>initial/delta/heartbeat/resync</code> های event پشتیبانی‌شده archive.
<code>recorded_at</code> , <code>as_of</code>	<code>timestamp-like</code> یا <code>None</code>	زمان مشاهده؛ <code>None</code> یعنی ساعت تهران/UTC داخلی معتبر تابع.
<code>max_records</code> , <code>record_max_records</code>	<code>int = 10000</code>	بر اساس تعداد؛ مثبت retention.
<code>retention_seconds</code> , <code>record_retention_seconds</code>	<code>float None</code>	یعنی غیرفعال <code>None</code> زمانی؛ retention.
<code>interval</code>	intraday: <code>str</code> در watcher <code>float=1.0</code> در	یا <code>tick/1min/5min/15min/30min/1h</code> candle interval بر حسب ثانیه polling فاصله
<code>max_updates</code>	<code>int None</code>	سقف eventهای iterator؛ <code>None</code> یعنی تا stop.

نام	type/default	معنا و constraint
<code>notifications</code>	پیش فرض، <code>iterable</code> ، (<code>'messages'</code> ، <code>'state'</code>)	فقط این دو notification پشتیبانی می‌شوند؛ Codal وجود ندارد.
<code>error_policy</code> ، <code>callback_error_policy</code> ، <code>storage_error_policy</code>	<code>str</code>	enum های دقیق: <code>error_policy ∈ {retry, raise, stop}</code> ، <code>callback_error_policy ∈ {raise, ignore, stop}</code> و <code>storage_error_policy ∈ {raise, ignore}</code> ؛ مقدار نامعتبر خطا است I/O پیش از
<code>max_backoff</code> ، <code>jitter</code> ، <code>request_timeout</code>	<code>float</code> <code>None</code>	هر درخواست؛ نامنفی/مثبت طبق <code>timeout</code> و <code>jitter</code> ، سقف <code>backoff</code> . کلاس.
<code>max_consecutive_retries</code> ، <code>max_retries</code>	<code>int</code> <code>None</code>	سقف <code>retry</code> ؛ <code>max_retries</code> نام قدیمی سازگار است.
<code>include_initial</code> ، <code>emit_heartbeats</code> ، <code>copy_snapshot</code> ، <code>record_heartbeats</code>	<code>bool</code>	کنترل <code>emission/copy/persistence event</code> ها.
<code>notification_top</code> ، <code>max_seen_notifications</code> ، <code>checkpoint_interval</code>	<code>int</code>	بین ۱ و ۱۰۰۰؛ دو مقدار دیگر مثبت <code>notification_top</code> ، بین ۱ و ۱۰۰۰۰ است <code>record_max_records</code>

پارامترهای درآمد ثابت و اختیار

نام	type/default	معنا و constraint
<code>settlement_date</code> ، <code>maturity_date</code> ، <code>issue_date</code> ، <code>valuation_date</code>	<code>None</code> یا <code>date-like</code>	تاریخ تسویه/سررسید/انتشار/ارزش‌گذاری؛ <code>None</code> در <code>live</code> یعنی امروز تهران.
<code>face_value</code>	<code>float</code> ؛ اخزا <code>1_000_000.0</code>	ارزش اسمی مثبت؛ <code>IRAN_TREASURY_FACE_VALUE</code> همین <code>default</code> است.
<code>day_count</code> / <code>convention</code>	<code>str='ACT/365F'</code>	پشتیبانی‌شده؛ تاریخ پایان باید بعد از شروع <code>convention</code> باشد.
<code>price</code> ، <code>annual_yield</code> ، <code>coupon_rate</code> ، <code>accrued_interest</code>	<code>float</code>	قیمت/بازده/کوپن/بهرهٔ تحقق‌یافته؛ <code>bounds</code> در تابع <code>math</code> اعتبارسنجی می‌شود.
<code>cashflows</code>	<code>iterable (date, amount)</code> یا <code>None</code>	جریان‌های نقدی؛ در حالت <code>None</code> از <code>maturity/face/coupon</code> ساخته می‌شود.
<code>compounding</code> ، <code>frequency</code> ، <code>price_type</code>	<code>str</code> ، <code>int</code> ، <code>str</code>	نوع مرکب، دفعات سالانه و <code>dirty/clean</code> .
<code>nodes</code>	<code>DataFrame/iterable mapping</code>	<code>rate/discount</code> و <code>maturity</code> شامل <code>curve</code> های <code>node</code> ؛ تعیین‌کنندهٔ تکرار است <code>duplicate policy</code>
<code>interpolation</code> ، <code>extrapolate</code> ، <code>duplicate_policy</code>	<code>log_discount</code> ، تابعی، <code>False</code>	سیاست <code>in-extrapolation discount</code> ؛ مستند <code>policy</code> یا <code>error duplicate</code>
<code>price_source</code>	<code>str='auto'</code>	انتخاب <code>Last/Close/Final/bid/ask</code> با <code>provenance</code> .

نام	type/default	معنا و constraint
maturity_map	None یا mapping	جلوگیری fuzzy صریح نماد→ سررسید؛ از حدس override می‌کند.
spot , strike , option_price , volatility	float	نامنفی volatility نامنفی و premium، مثبت spot/strike.
time_to_expiry , rate , dividend_yield	float	سال تا سررسید، نرخ بدون ریسک و yield پیوسته.
option_type , exercise_style	str='call' , str='european'	را می‌پذیرد European فعلی فقط math call/put ؛
lower_volatility , upper_volatility , tolerance , max_iterations	0.0 , 5.0 , تابعی، تابعی	و شمار lower < upper solver: bracket و همگرایی iteration مثبت.
risk_free_rate , yield_curve	None یا scalar/curve	نرخ ثابت یا YieldCurve ؛ اگر هر دو داده شوند قرارداد تحلیل آن‌ها را اعتبارسنجی می‌کند.
parity_tolerance , liquidity_weights	float mapping None	band parity scoring: یعنی None و وزن‌های band parity داخلی مستند.

پارامترهای کم‌تکرار نیز بخشی از قراردادند:

دامنه	پارامترها و معنا
قیمت history	متناظر volume تعدیل (bool=False) adjust_volume ؛ OHLC: تعدیل (bool=True) auto_adjust
فهرست بازار	bourse , farabourse , payeh (bool=True) و haghe_taqadom , sandogh , bonds , options , mortgage , commodity , energy (bool=False) inclusion: output (str='dataframe') format: نام/InsCode (str) index_name ؛ category: fund_type (str list None) نام ارز؛ (str list='') name ؛ شناسه سهامدار (bool=False) include_id ؛ شاخص؛
fundamentals	مثبت معتبر P/E فقط (bool=True) positive_pe ؛ مرزهای شامل؛ (float None) pe_min , pe_max
live/option	category کد بازار اختیاری؛ (int=0) exchange ؛ point opt-in: مسیر (str='none') fallback (str='treasury') دسته IFB: (float=10.0) lock_timeout و (float=300.0) stale_lock_seconds ؛ قفل فایل مثبت
fixed math	bump_size (float=0.0001) شوک DV01: rate_compounding (str='effective') ، rate_frequency (int=1) ؛ enforce_monotonic_discount (bool=True) guard curve.
storage	event (MarketEvent) رخداد ورودی و frame (DataFrame) batch archive.
resolver	selector (Any) الزام فعالیت (bool=True) require_active ؛ انتخاب ورودی؛

فیلدهای باقی‌مانده MarketEvent همگی constructor contract هستند: (int sequence) شماره افزایشی، (tuple[str,...] changed_incodes) و (tuple[tuple[str,int],...] delta، changed_order_levels) cursor (tuple[str,str,str] notification_tokens) ، market_state_changed (bool) های notification،

`state_changes` (`DataFrame|None`)، `cursor_before` / `cursor_after` (`int`)، `retry_count` (`int`)، `retry_error` (`persistence_status` / `persistence_error` (`str|None` و (`str|None`)، `notification_errors` (`tuple[str,...]`) هستند. فیلد `is_active` (`bool|None`) در `InstrumentRef` سه حالت فعال/غیرفعال/نامعلوم دارد.

نوع‌ها، ثابت‌ها، تنظیمات و exception‌های عمومی

```
InstrumentRef(ins_code: 'str', symbol: 'str | None' = None, name: 'str | None' = None, asset_type: 'str' = 'unknown', is_active: 'bool | None' = None, provenance: 'str' = '', selector: 'str | None' = None) -> None
```

نماد/نام، نوع دارایی، وضعیت فعال، منبع اثبات و canonical هویت است؛ فیلدها به ترتیب شناسه `dataclass immutable` ورودی هستند. ساخت مستقیم فقط برای داده از قبل اعتبارسنجی شده مناسب است؛ مسیر عادی `selector` گم‌شده/اضافی است `field` برای `TypeError` استاندارد `constructor` است. خطای `resolve_instrument()`

```
MarketEvent(kind: 'str', sequence: 'int', fetched_at: 'pd.Timestamp', trade_date: 'Optional[dt.date]', snapshot: 'dict[str, Any]', changed_incodes: 'tuple[str, ...]' = (), changed_order_levels: 'tuple[tuple[str, int], ...]' = (), market_state_changed: 'bool' = False, notification_tokens: 'tuple[str, str, str]' = ('', '', ''), messages: 'Optional[pd.DataFrame]' = None, state_changes: 'Optional[pd.DataFrame]' = None, cursor_before: 'int' = 0, cursor_after: 'int' = 0, retry_count: 'int' = 0, retry_error: 'Optional[str]' = None, notification_errors: 'tuple[str, ...]' = (), persistence_status: 'Optional[str]' = None, persistence_error: 'Optional[str]' = None) -> None
```

هستند؛ `mutable` کپی دفاعی ولی `copy_snapshot=True` با `snapshot/messages/state_changes` است. `event watcher` را نگه می‌دارند. مثال ساخت دستی لازم نیست؛ نمونه `provenance` / `delta` / `cursor` های `tuple` ها `watch_market()` موضوعی بالاتر است.

```
YieldCurve(settlement_date: datetime.date, maturities: tuple, times: tuple, discount_factors: tuple, continuous_zero_rates: tuple, day_count: str = 'ACT/365F', interpolation: str = 'log_discount', extrapolate: bool = False, node_metadata: tuple = <factory>, diagnostics: dict = <factory>) -> None
```

بسازید. `build_yield_curve()` است؛ آن را با `metadata/diagnostics` محاسباتی با آرایه‌های هم‌طول و `curve immutable` می‌دهند `ValueError`، `extrapolate=False` خارج از محدوده با `interpolation` متدهای

معنا	type/value	ثابت export
نسخهٔ SQLite market history schema.	<code>int = 2</code>	<code>MARKET_HISTORY_SCHEMA_VERSION</code>
برای رد فایل نامرتب SQLite application id.	<code>int = 1096045381</code>	<code>MARKET_HISTORY_APPLICATION_ID</code>
ارزش اسمی پیش‌فرض اخزا، نه override اجباری همهٔ اوراق.	<code>float = 1_000_000.0</code>	<code>IRAN_TREASURY_FACE_VALUE</code>
نسخهٔ سند snapshot JSON اختیاری؛ reader mismatch را رد می‌کند.	<code>int = 1</code>	<code>OPTION_SNAPSHOT_SCHEMA_VERSION</code>

`settings` یک singleton از `Settings` تنظیمات mutable است. عمومی مهم `ssl_verify: bool=True` , `timeout: int|float=10` , `max_retries: int=3` , `retry_backoff_factor: float=0.3` , `rate_limit_delay: float=0.3` , `market_snapshot_freshness_seconds: float=120.0` , `market_clock_skew_tolerance_seconds: float=5.0` , `client_volume_consistency_tolerance: float=0.05` , `industry_membership_cache_ttl: float=3600.0` , `order_book_max_requests: int=250` , `trade_max_requests: int=250` و `order_book_discovery_lookback_days: int=10` هستند؛ provider قرارداد تنظیم `url_*: str` های روز/ارز/صندوق/بازار پایه و `mapping` , `headers: dict` هستند. را نقض کند و برای مصرف عادی توصیه نمی‌شود source-boundary می‌تواند URL تغییر

دارند `AlgotikTSEError` مشترک base و `(*args)` ارث‌بردهٔ constructor های عمومی همگی exception برای `DataParsingError` , `HTTP/provider` برای `ConnectionError` , برای چند هویت هم‌رتبه `AmbiguousSymbolError` برای نبود هویت و `StockNotFound` , برای ورودی نامعتبر `InvalidParameterError` , نامن schema/identity در فصل تنظیمات است catch برای خروج از مرز منبع. نمونهٔ `UnsupportedDataSourceError`

قرارداد	constructor
base exception: <code>args: tuple[Any,...]</code> را مانند context/پیام <code>Exception</code> نگه می‌دارد؛ ندارد return خودش.	<code>AlgotikTSEError</code>
چند هویت exact هم‌رتبه؛ راه‌حل ارائهٔ <code>ins_code</code> است.	<code>AmbiguousSymbolError</code>
HTTP/status/redirect/provider failure با cause اصلی <code>(raise ... from exc)</code> .	<code>ConnectionError</code>
کود معمولاً مناسب نیست retry هویت/فایل ناسازگار؛ payload/schema/.	<code>DataParsingError</code>
I/O. های جدید پیش از API ورودی؛ در constraint	<code>InvalidParameterError</code>
انجام نمی‌شود fuzzy-first-hit پیدا نشده؛ selector exact	<code>StockNotFound</code>
است fail-before-I/O نمونهٔ <code>get_introduction</code> boundary؛ نیازمند منبع خارج API	<code>UnsupportedDataSourceError</code>
نامعتبر field/shape و خطای object curve بالا؛ خروجی constructor dataclass <code>build_yield_curve</code> . <code>TypeError/ValueError</code> ؛ مثال	<code>YieldCurve</code>

قرارداد API: هویت، قیمت، معاملات و API‌های قدیمی

```
normalize_instrument_text(value: 'Any') -> 'str'
resolve_instrument(selector=None, *, ins_code=None, asset_type='auto', snapshot=None,
require_active=True) -> 'InstrumentRef'
validate_ins_code(value: 'Any') -> 'str'
get_history(symbol='', start=None, end=None, limit=0, raw=False, auto_adjust=True,
output_type='standard', date_format='jalali', progress=True, save_to_file=False, dropna=True,
adjust_volume=False, return_type=None, ascending=True, save_path=None, include_today=False, *,
ins_code=None, asset_type='auto', **kwargs)
get_client_type(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
get_capital_increase(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_intraday(symbol='شتران', interval='1min', start=None, end=None, progress=True, **kwargs)
get_trades(symbol=None, *, ins_code=None, start=None, end=None, include_canceled=False,
max_requests=None, raw=False, progress=True)
get_live_trades(symbol=None, *, ins_code=None, include_canceled=False, max_requests=None, raw=False,
progress=True)
get_detail(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_info(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_stats(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_introduction(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
get_symbols(bourse=True, farabourse=True, payeh=True, haghe_taqadom=False, sandogh=False, bonds=False,
options=False, mortgage=False, commodity=False, energy=False, payeh_color=None, output='dataframe',
progress=True, **kwargs)
get_shareholders(symbol='', date=None, include_id=False, *, ins_code=None, asset_type='auto', **kwargs)
get_currency(name='', start=None, end=None, limit=0, output_type='standard', date_format='jalali',
progress=True, save_to_file=False, dropna=True, return_type=None, ascending=True, save_path=None,
**kwargs)
get_market_snapshot(*args, **kwargs)
get_market_client_type(*args, **kwargs)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>normalize_instrument_text</code>	به رشته تهی و <code>value: Any None</code> ؛ متن با یکسان‌سازی ی/ک، فاصله و به کلید مقایسه تبدیل می‌شود ZWNJ	<code>mutate</code> ؛ ورودی را <code>canonically</code> نمی‌کند	خطای ویژه ندارد؛ <code>normalize_instrument_text("گل") == "گل"</code>
<code>validate_ins_code</code>	مثبت یا <code>integer value: str int</code> ؛ فاصله ابتدا/انتهای ASCII رشته ۰۰۱ رقم رقم فارسی، <code>float</code> ، <code>bool</code> ، (می‌شود <code>strip</code> داخلی، صفر و طول <code>space</code> ، <code>sign</code> ، عربی بیش از ۲۰ رد می‌شوند	معتبر دقیقاً به <code>ASCII: integer str</code> رشته تبدیل می‌شود	<code>InvalidParameterError</code> ؛ مثال <code>resolver</code> بخش
<code>resolve_instrument</code>	<code>snapshot:</code> <code>dict DataFrame None</code> منبع حتی اگر تهی؛ <code>authoritative</code> <code>require_active: bool=True</code> ابزار غیرفعال را حذف می‌کند. اولویت آمده است <code>resolver</code> در فصل <code>exact</code>	<code>InstrumentRef</code> ؛ مسیر اثبات <code>provenance</code> (<code>explicit_ins_code</code> ، <code>snapshot/search/index registry</code> و مشابه) را ثبت می‌کند	<code>InvalidParameterError</code> ، <code>StockNotFoundError</code> ، <code>AmbiguousSymbolError</code> ، <code>DataParsingError</code> ، <code>ConnectionError</code> ؛ مثال بالاخر <code>exact/ambiguity</code> .

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
get_history	symbol/ins_code/asset_type هویت؛ start/end/limit/ascending raw schema TSE؛ auto_adjust OHLC؛ adjust_volume تعدیل حجم؛ output_type∈{standard,full}؛ date_format∈{jalali,gregorian,both}؛ return_type∈{simple,log,both,th,list} include_today؛ progress/dropna ردیف زنده؛ save_to_file/save_path نمایش/ترکیب؛ alias فقط kwarg؛ ذخیره‌های CSV؛ مستند.	برای سهم و auto_adjust=True ، output_type='standard' دقیقاً Open,High,Low,Close,Volume ستون‌های me است؛ Final,No.,Value و Ticker را اضافه می‌کند. با auto_adjust=False ، standard Adj Close ستون هم index/industry schema دارد. محدودتر خود را دارند؛ چند نماد MultilIndex. attrs پیش‌فرض تهی و با include_today=True شامل include_today_appended/include_today_warning است.	ورودی‌های ناسازگار legacy معمولاً ValueError / None و ConnectionError ;provider مثال فصل قیمت.
get_client_type	symbol/ins_code/asset_type هویت؛ start/end/limit/ascending raw schema provider؛ output_type∈{standard,full}؛ date_format∈{jalali,gregorian,both} include_today؛ progress/dropna ردیف امروز؛ save_to_file/save_path نمایش/چندنمادی؛ alias قدیمی؛ kwarg؛ CSV؛	روزانه DataFrame Buy_I/N_Count , Buy_I/N_Volume , Sell_I/N_Count , Sell_I/N_Volume , قدرت/ سرانه‌های مشتق؛ چند نماد MultilIndex. opt-in ردیف امروز.	خطای selector/provider یا legacy None ; مثال فصل حقیقی/حقوقی.
get_capital_increase	stock= . قدیمی alias و selector	index با DataFrame None و date old_shares_amount,new_shares_amount .	در provider/نماد گم‌شده/index None ؛ قرارداد قدیمی پیام و att.get_capital_increase ("فملی") .
get_intraday	interval canonical یکی از tick,1min,5min,15min,30min,1h,4h,12h های _INTERVAL_MAP مانند 1m,60min,4hour,240m,12hour,720,ticks,raw بدون تاریخ start snapshot معاملات امروز، با تاریخی.	شامل tick ; DataFrame None شامل candle زمان/قیمت/حجم و Open,High,Low,Close,Volume با me, TradeCount DatetimeIndex.	نامعتبر یا تاریخی که interval قدیمی نامعتبر تشخیص validator None دهد پیام چاپ می‌کند و provider/مسیر/نماد نیز در مسیر/legacy ؛ ValueError None ؛ نیست؛ API خطای عمدی قرارداد این مثال att.get_intraday ("فملی", interval="5min") .

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>get_trades</code>	بدون تاریخ امروز تهران؛ Thu/Fri قبل از budget حذف؛ <code>include_canceled</code> ; فقط <code>max_requests</code> Trade endpoint: <code>raw</code> ستون‌های provider را اضافه می‌کند.	standard: <code>DataFrame[InsCode, Symbol, GregorianDate, JalaliDate, TradeNo, Time, Timestamp, Price, Volume, Value, Canceled, Source]</code> ; attrs شامل request/provenance/failures. raw نیز دارد audit ستون‌های	<code>InvalidParameterError</code> , resolver errors, <code>ConnectionError</code> , <code>DataParsingError</code> ; مثال: فصل معاملات
<code>get_live_trades</code>	هویت: <code>symbol/ins_code</code> رکورد ابطالی: <code>include_canceled</code> سقف درخواست: <code>max_requests</code> audit schema: <code>raw</code> progress نمایش. تاریخ به‌صورت خودکار امروز تهران است.	دقیق attrs و schema معاملات امروز با: <code>get_trades</code> زمان، قیمت، حجم، ارزش، وضعیت provenance. ابطال و	<code>InvalidParameterError</code> , resolver errors, <code>ConnectionError</code> , <code>DataParsingError</code> ; <code>att.get_live_trades(ins_code="...")</code> .
<code>get_detail</code>	قدیمی API HTML دقیق؛ selector	index یا <code>DataFrame None</code> به همراه <code>value</code> و ستون <code>key</code> row <code>id</code> .	در رفتار index/no-match/HTTP مثال: legacy <code>None</code> ; <code>att.get_detail("فملی")</code> .
<code>get_info</code>	دقیق selector	از <code>DataFrame None</code> key/value کامل <code>instrumentInfo</code> . flatten	یا legacy <code>None</code> ؛ مثال فصل connection/parsing اطلاعات نماد
<code>get_stats</code>	دقیق selector	<code>DataFrame None</code> key/value عددی value آمار با کلید فارسی و	یا legacy <code>None</code> connection/parsing: <code>att.get_stats("فملی")</code> .
<code>get_introduction</code>	هیچ پارامتر BC فقط برای signature نمی‌شود I/O باعث	هرگز خروجی موفق ندارد.	همیشه <code>UnsupportedDataSourceError</code> پیش از I/O؛ جایگزین market-data: <code>. get_info/get_detail</code>
<code>get_symbols</code>	market/asset، boolean <code>payeh_color:</code> <code>str list None</code> ; output: های <code>str='dataframe'</code> ; alias <code>kwargs</code> انگلیسی در	فهرست ابزارها یا <code>DataFrame</code> قدیمی انتخابی؛ ستون‌های format ؛ خروجی تهی type هویت/نام/بازار و پایدار دارد schema	فیلتر/output نامعتبر ; legacy <code>ValueError</code> یا <code>None</code> مثال فصل فهرست ابزارها.
<code>get_shareholders</code>	<code>include_id: bool=False</code> ; آخرین و تاریخ مشخص آن روز snapshot	<code>DataFrame None[share_holder_name, number_of_shares, percentage_of_shares, change_state, change_amount, date]</code> و با <code>opt-in</code> <code>share_holder_id</code> .	پیام و legacy در provider/selector مثال فصل اطلاعات نماد: <code>None</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی، schema و attrs	خطاهای اصلی و مثال
<code>get_currency</code>	نام ارز/سکه: <code>name: str list</code> <code>start/end/limit/ascending</code> schema: <code>output_type</code> بازه؛ <code>date_format∈{jalali,gregorian,both}</code> بازه؛ <code>return_type</code> نمایش/چندارزی؛ <code>progress/dropna</code> <code>save_to_file/save_path</code> های قدیمی. منبع alias: <code>kwargs</code> CSV: است TGLU.	تک ارز <code>DataFrame[Open,High,Low,Close]</code> ؛ چند ارز ستون <code>MultiIndex</code> ؛ index تاریخ.	نام نامعتبر/provider ممکن است <code>ValueError / None</code> ؛ مثال فصل ارز.
<code>get_market_snapshot</code>	به تابع BC برای <code>*args/**kwargs</code> <code>market_watch</code> صفرآرگومان می‌شود؛ در عمل آرگومان غیرتهی forward می‌دهد <code>TypeError</code> .	با کلیدهای دقیق <code>dict</code> <code>stocks,order_book,market_time,index_value,migration,trade_date,market_state,exchange_time,fetched_at,snapshot_age_seconds,is_today_trade_date,is_history_eligible,is_realtime_fresh,is_previous_trade_date,is_stale,is_partial</code> .	<code>ConnectionError</code> , مثال: <code>DataParsingError</code> ; live.
<code>get_market_client_type</code>	<code>*args/**kwargs</code> wrapper <code>market_client_type</code> .	<code>DataFrame[InsCode,Buy_I_Count,...,Net_I_Volume,Net_N_Volume]</code> .	<code>ConnectionError</code> , مثال: <code>DataParsingError</code> ; live.

قرارداد live: API، سفارش، watcher و تحلیل بازار

```

get_order_book(symbol=None, *, selector_strict=False)
get_live_market(symbol=None, *, strict=False)
get_live_symbol(symbol=None, *, ins_code=None, fallback='none')
get_order_book_history(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, complete_only=False, *, max_requests=None, _request_budget_state=None, **kwargs)
get_orderbook_history(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, complete_only=False, *, max_requests=None, _request_budget_state=None, **kwargs)
get_queue(symbol=None, side='both', strict=True, *, selector_strict=False)
get_queue_history(symbol='', start=None, end=None, limit=0, date_format='jalali', progress=True,
save_to_file=False, dropna=True, ascending=True, save_path=None, include_today=False,
complete_only=False, side='both', strict=True, *, max_requests=None, **kwargs)
MarketWatcher(symbol=None, interval=1.0, max_updates=None, include_initial=True, emit_heartbeats=True,
notifications=('messages', 'state'), notification_top=50, stop_event=None, error_policy='retry',
max_backoff=30.0, jitter=0.1, callback_error_policy='raise', max_consecutive_retries=5,
max_retries=None, max_seen_notifications=10000, copy_snapshot=True, request_timeout=None, *,
record_to=None, record_heartbeats=False, checkpoint_interval=100, record_max_records=10000,
record_retention_seconds=None, storage_error_policy='raise', _request=safe_get, _clock=None,
_wait=None, _random=None)
watch_market(*args, **kwargs)
get_market_messages(flow=0, top=20, since_id=None, *, archive_to=None, _recorded_at=None,
_request=safe_get)
get_instrument_state_changes(top=20, since_id=None, *, archive_to=None, _recorded_at=None,
_request=safe_get)
get_market_overview(flow=0, *, archive_to=None, _recorded_at=None, _request=safe_get)
get_market_breadth(symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True,
instrument_types=None, *, _snapshot=None)
get_sector_flow(symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True,
instrument_types=None, *, _snapshot=None, _client_type=None)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_order_book</code>	scalar/list/ <code>None</code> ; فقط <code>selector_strict</code> انتخاب را سخت می‌کند resolution.	با <code>long DataFrame</code> <code>InsCode, Symbol, Name, Level, BidOrderCount, BidVolume, BidPrice, AskPrice, AskVolume, AskOrderCount</code> و metadata <code>trade_date, market_state, exchange_time, fetched_at, is_*</code> response پنج سطح از همان. snapshot.	<code>InvalidParameterError</code> , <code>StockNotFoundError</code> در مثال: <code>strict. connection/parsing</code> فصل سفارش.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_live_market</code>	های <code>selector</code> <code>strict=False</code> گم‌شده را در <code>attrs['missing_selectors']</code> آن‌ها را خطا می‌کند <code>strict</code> ثبت می‌کند؛ <code>]</code>	با <code>DataFrame</code> <code>STOCK_COLUMNS</code> ، <code>client</code> <code>columns</code> ، <code>wide</code> سطح‌های <code>BidPrice1..5/AskPrice1..5</code> از جمله <code>metrics</code> و <code>EstimatedNetIndividualFlow</code> <code>row-wise</code> ستون <code>freshness</code> ؛ <code>attrs</code> دقیق است: <code>field_validity, source_schema_presence, migration, missing_selectors</code> . <code>Last</code> پایانی است <code>Close</code> آخرین معامله و	<code>InvalidParameterError</code> ، <code>StockNotFoundError</code> در <code>strict</code> ، <code>ConnectionError/DataParsingError</code> ؛ <code>quickstart</code> و مثال <code>attrs</code> بالا.
<code>get_live_symbol</code>	<code>fallback: 'none' 'point'</code> ؛ <code>point</code> فقط در غیاب <code>MarketWatch</code> . <code>fallback='none'</code> برای <code>no-match</code> <code>frame</code> نمی‌دهد.	exception؛ دقیقاً یک‌ردیفی یا <code>frame</code> <code>provenance</code> <code>SnapshotSource='market_watch' 'closing_price_info'</code> و ستون‌های <code>_fallback'</code> <code>PresentInMarketWatch, MarketStateTitle, has_trade_today, PriceActionable, fallbackReason, IdentityVerified</code> .	<code>no-match</code> با <code>fallback none:</code> <code>StockNotFoundError</code> ؛ همچنین <code>InvalidParameterError</code> ، <code>AmbiguousSymbolError</code> ، <code>DataParsingError</code> ؛ فصل <code>live</code> .
<code>get_order_book_history</code>	<code>raw</code> ؛ <code>output_type='standard'</code> <code>long</code> و <code>'wide'</code> ؛ <code>complete_only</code> ؛ <code>budget</code> این <code>calls</code> همه <code>workflow</code> .	<code>long/wide/raw</code> <code>DataFrame</code> ؛ <code>identity/time/5</code> سطح و <code>is_partial, is_complete, market_partial_status, is_reconstructed, is_stale, record_type, source</code> ؛ <code>attrs</code> <code>schema, failed_requests, request_count, max_requests</code> .	<code>InvalidParameterError</code> ، <code>DataParsingError</code> ، <code>resolver/connection: failure</code> ؛ مثال فصل <code>attrs</code> + <code>warning</code> جزئی تاریخچه سفارش.
<code>get_orderbook_history</code>	کاملاً یکسان با <code>signature</code> هویتی و <code>alias</code> <code>get_order_book_history</code> .	دقیقاً همان <code>.object/return/schema/attrs</code>	همان خطاها و همان مثال؛ <code>att.get_orderbook_history</code> <code>is</code> <code>att.get_order_book_history</code> . <code>ry</code>
<code>get_queue</code>	فقط <code>side</code> ؛ <code>strict=True</code> های قطعی را نگه می‌دارد <code>queue</code> .	<code>DataFrame[InsCode, Symbol, Name, ..., Side, QueuePrice, QueueVolume, QueueOrders, QueueValue, PriceLimit, is_queue, book_state, ...]</code> .	نامعتبر <code>side</code> <code>InvalidParameterError</code> ؛ مثال <code>selector/provider errors</code> صف.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_queue_history</code>	<code>symbol/start/end/limit/ascending</code> ؛ تاریخ: <code>date_format</code> ؛ بازه: <code>include_today</code> ؛ <code>live</code> ؛ <code>complete_only</code> ؛ snapshot؛ کامل: <code>side∈{buy,sell,both}</code> ؛ فقط صف قطعی: <code>strict</code> ؛ بودجه: <code>max_requests</code> ؛ نمایش/تهی: <code>progress/dropna</code> ؛ <code>save_to_file/save_path</code> ؛ از CSV: <code>kwargs</code> ؛ <code>alias</code> ؛ <code>threshold</code> ؛ است. as-of snapshot همان تاریخ و	<code>QUEUE_COLUMNS</code> ؛ <code>is_queue</code> ؛ nullable. <code>crossed book</code> ؛ <code>false</code> ؛ <code>attrs budget/failures</code> .	<code>InvalidParameterError</code> ، <code>resolver/connection/parsing</code> ؛ مثال فصل صف.
<code>MarketWatcher</code>	فقط <code>notifications</code> ؛ <code>messages/state</code> ؛ policy enum؛ <code>glossary</code> ؛ دقیقاً در <code>interval,max_backoff>=0</code> ، نامنفی <code>retry limit</code> ؛ <code>0<=jitter<=1</code> ، <code>request_timeout/record_reten</code> ؛ <code>tion_seconds>0</code> ؛ دارای <code>stop_event</code> ؛ <code>is_set/wait</code> ؛ <code>record_to</code> ؛ opt-in.	<code>MarketEvent</code> ؛ سنکرون <code>iterator</code> ؛ <code>kind</code> ؛ <code>initial/delta/heartbeat/resync</code> ؛ <code>constructor</code> ؛ I/O خود؛ نمی‌کند.	نامعتبر <code>policy/range</code> ؛ <code>InvalidParameterError</code> ؛ <code>iteration</code> ؛ runtime؛ پیش از <code>ConnectionError/DataParsingError</code> ؛ مثال <code>policy</code> طبق <code>watcher</code> .
<code>watch_market</code>	تمام <code>*args/**kwargs</code> بدون تغییر به <code>MarketWatcher</code> می‌روند.	نه، <code>MarketWatcher</code> ؛ <code>DataFrame</code> .	همان <code>constructor/runtime</code> ؛ مثال <code>for event in att.watch_market(max_updates=3): ...</code>
<code>get_market_messages</code>	local؛ فیلتر <code>since_id</code> ؛ <code>archive_to</code> ؛ <code>opt-in</code> ؛ نوشتن اتمیک.	<code>DataFrame[message_id,date,time,timestamp,title,description,flow]</code> ؛ <code>attrs provenance/archive</code> .	<code>InvalidParameterError</code> ، <code>ConnectionError</code> ، <code>DataParsingError</code> ، <code>storage error</code> ؛ مثال فصل پیام‌error.
<code>get_instrument_state_changes</code>	<code>since_id</code> ؛ <code>bounded</code> ؛ تعداد <code>top</code> ؛ فقط رکوردهای بعد از شناسه؛ <code>archive_to</code> ؛ <code>SQLite</code> ؛ <code>opt-in</code> ؛ مسیر <code>_recorded_at/_request</code> ؛ فقط <code>seam</code> ؛ تست.	<code>DataFrame[event_id,date,time,timestamp,InsCode,Symbol,Name,state_code,state_real_time,under_supervision,state_title]</code> .	<code>InvalidParameterError</code> ، <code>ConnectionError</code> ، <code>DataParsingError</code> ؛ یا خطای <code>storage</code> ؛ مثال فصل وضعیت <code>storage</code> .
<code>get_market_overview</code>	تهی. صفر ردیف است <code>payload</code> ؛ <code>flow</code> ؛	با <code>DataFrame</code> ؛ <code>provider overview</code> ؛ <code>payload/fast-view</code> ؛ <code>attrs source/time/archive</code> ؛ <code>flow</code> ؛ و فیلدهای	<code>parameter/connection/parsing/storage</code> ؛ مثال <code>overview</code> .
<code>get_market_breadth</code>	فیلترهای <code>universe</code> ؛ <code>denominator</code> ؛ ابرار انتخاب‌شده.	یک‌ردیف <code>DataFrame</code> ؛ <code>counts/percentages</code> ؛ <code>A/D</code> ؛ <code>volume/value</code> ؛ <code>limit counts</code> ؛ <code>freshness</code> ؛ <code>attrs analytics/source</code> .	<code>InvalidParameterError</code> ، <code>DataParsingError</code> ؛ مثال <code>breadth</code> .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
get_sector_flow	فیلتر: <code>symbol/flow/sector</code> <code>traded_only</code> universe معامله‌شده؛ بازار پایه؛ <code>include_base_market</code> کد نوع ابزار؛ <code>instrument_types</code> فقط <code>_snapshot/_client_type</code> فقط برای ردیف client feed. تست شده به کار می‌رود reconcile	یک ردیف در هر <code>SectorCode</code> با <code>breadth +</code> <code>client_coverage, net_individual_volume, estimated_net_individual_value, value_available, value_method</code> و <code>freshness</code>	parameter/connection/parsing؛ مثال sector.

قرارداد API: تقویم، نمای کلان، مستر ابزار و پیش‌گشایش

```

get_trading_calendar(start=None, end=None, market='all', include_closed=False, limit=90)
get_market_activity(start=None, end=None, market='all', max_requests=30, progress=True)
get_market_value_history(start=None, end=None, market='all', limit=90)
get_index_impact(date=None, market='all', top=10, direction='both')
get_market_trades(date=None, market='all', progress=True)
get_instrument_master(active=None, market=None, asset_type=None, progress=True)
get_instrument_changes(previous, current=None, progress=True)
get_theoretical_opening_price(*, username=None, password=None, market='all', timeout=None,
progress=True)
get_preopen_imbalance(top_data=None, *, username=None, password=None, market='all', timeout=None,
progress=True)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
get_trading_calendar	تاریخ شمسی/میلادی؛ <code>start/end</code> یا بورس/فرا بورس؛ <code>market='all'</code> فقط <code>include_closed=False</code> در نبود بازه، <code>limit=90</code> مثبت و حداکثر ۵۰۰۰ session	<code>DataFrame[Market, Flow, GregorianDate, JalaliDate, IsTradingDay, ReferenceIndex, ReferenceInsCode, Source]</code> شامل attrs <code>calendar_semantics='published_reference_index_sessions'</code>	ترتیب/نوع تاریخ، بازار، bool یا limit نامعتبر ؛ <code>InvalidParameterError</code> مثال فصل payload/شبهه typed؛ تقویم.
get_market_activity	session؛ بازه <code>start/end</code> ؛ <code>market</code> ; <code>max_requests=30</code> بودجه سخت ۳۶۶.۱؛ بدون بازه آخرین جلسه را می‌گیرد	<code>DataFrame[Market, GregorianDate, JalaliDate, InstrumentCount, TradedInstrumentCount, TradeCount, Volume, Value, AverageTradeValue, Source]</code> bulk daily و attrs request count.	بازه/بودجه market نامعتبر ؛ <code>InvalidParameterError</code> ؛ <code>provider typed</code> ؛ مثال فصل نمای کلان.
get_market_value_history	؛ <code>start/end</code> ; <code>market</code> ; start مثبت تا ۵۰۰۰؛ با <code>limit=90</code> داخلی ۵۰۰۰ است fetch سقف	<code>DataFrame[Market, Flow, GregorianDate, JalaliDate, MarketValue, Change, ChangePercent, Source]</code> ؛ واحد <code>attrs['unit']='rial'</code>	پارامتر/تاریخ ؛ <code>InvalidParameterError</code> ؛ <code>connection/schema typed</code> ؛ مثال فصل نمای کلان.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_index_impact</code>	آخرین جلسه؛ <code>date=None</code> مثبت تا ۱۰۰۰؛ <code>top=10</code> <code>market ; top=10</code> <code>direction∈{both,positive,negative}</code> .	<code>DataFrame[Market,Flow,GregorianDate,JalaliDate,Rank,InsCode,Symbol,Name,Close,Impact,Direction,AbsSharePct,Source]</code> attrs. درصد در denominator	<code>direction/top/date/market</code> نامعتبر : <code>InvalidParameterError</code> مثال <code>provider typed</code> : <code>att.get_index_impact(top=10)</code> .
<code>get_market_trades</code>	آخرین جلسه؛ <code>date=None</code> فیلتر بازار با <code>progress</code> . است ISIN پیشوند.	یک خلاصه روزانه برای هر ابزار با هویت، OHLC، معامله/حجم/ارزش و تغییر؛ <code>attrs['transaction_level']=False</code> و شمار طبقه‌بندی نامعلوم.	<code>parameter/connection/schema</code> typed؛ برای معاملات ریز از <code>get_trades</code> استفاده شود؛ مثال <code>bulk</code> . فصل
<code>get_instrument_master</code>	<code>active: bool None ; market: str None ; asset_type: str iterable None</code> از ۱۰ نوع None غیر <code>active</code> . <code>progress</code> مستند؛ اضافه می‌گیرد MarketWatch یک	<code>DataFrame[InsCode,ISIN,Symbol,Name,EnglishName,CompanyNameCode,CompanyISIN,Market,MarketGroup,Industry,AssetType,InstrumentStatus,Active,Source]</code> identity key در attrs.	نامعتبر <code>filter/type</code> : <code>InvalidParameterError</code> HTML/schema/network typed؛ Parquet مثال ذخیره.
<code>get_instrument_changes</code>	اجباری و <code>previous</code> <code>current=None</code> ؛ هر کدام DataFrame یا CSV/Parquet/JSON؛ <code>InsCode</code> کلید اجباری. <code>progress</code> .	long-form <code>DataFrame[ChangeType,InsCode,Field,OldValue,NewValue,Symbol,Source]</code> و <code>added/removed/changed</code> count در snapshot در attrs.	مسیر/پسوند/ستون نامعتبر : <code>InvalidParameterError</code> current ممکن است backfill بدون <code>provider error</code> تاریخی.
<code>get_theoretical_opening_price</code>	keyword-only اشتراک <code>username/password</code> <code>market ; timeout ;</code> package در <code>credentials</code> . ذخیره نمی‌شوند	رسمی با قیمت/حجم تئوریک TOP، remainder، buy/sell؛ تغییر؛ <code>attrs</code> <code>official=True,subscriber_service=True,credentials_stored=False</code> .	ناموجود <code>credentials</code> : <code>InvalidParameterError</code> SOAP/network یا <code>ConnectionError</code> مثال <code>DataParsingError</code> ؛ فصل TOP.
<code>get_preopen_imbalance</code>	<code>top_data: DataFrame None ;</code> از None در <code>username/password/market/timeout/progress</code> برای TOP استفاده می‌کند.	همه ستون‌های TOP + <code>ImbalanceVolume,ImbalanceRatio,ImbalanceSide</code> فرمول دقیق در <code>attrs</code> .	نامعتبر <code>type/bool/ستون</code> : <code>InvalidParameterError</code> ؛ مثال <code>fetch</code> در حالت TOP خطاهای فصل TOP.

قرارداد API: تاریخچه محلی و fundamentals

```

get_market_fundamentals(symbols=None, *, pe_min=None, pe_max=None, positive_pe=True, instrument_types=(300, 303, 309), strict=False, allow_stale=False, archive_to=None, max_requests=1, progress=True)
get_market_fundamentals_history(path, start=None, end=None, symbols=None, *, pe_min=None, pe_max=None, positive_pe=True, instrument_types=(300, 303, 309), strict=False, allow_stale=True, limit=1000, offset=0)
get_price_adjustments(symbol=None, *, ins_code=None, start=None, end=None, progress=True)
get_latest_price_adjustment(symbol=None, *, ins_code=None, start=None, end=None, progress=True)
check_market_history(path)
save_market_snapshot(path, snapshot=None, *, as_of=None, _live_fetch=None)
load_market_snapshots(path, start=None, end=None, symbol=None, *, limit=1000, offset=0)
get_live_market_history(path, start=None, end=None, symbol=None, *, limit=1000, offset=0)
get_market_overview_history(path, start=None, end=None, flow=0, *, source='tsetmc', limit=1000, offset=0)
get_market_snapshot_summary_history(path, start=None, end=None, flow=None, *, limit=1000, offset=0)
get_market_breadth_history(path, start=None, end=None, symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True, instrument_types=None, *, limit=1000, offset=0)
get_sector_flow_history(path, start=None, end=None, symbol=None, flow=None, sector=None, traded_only=False, include_base_market=True, instrument_types=None, *, limit=1000, offset=0)
record_market_event(path, event, *, session_id=None, include_snapshot=True, max_records=10000, retention_seconds=None)
get_market_event_history(path, start=None, end=None, kind=None, *, session_id=None, limit=1000, offset=0)
archive_market_records(path, kind, frame, *, source='tsetmc', recorded_at=None)
get_market_messages_history(path, start=None, end=None, flow=0, since_id=None, *, source='tsetmc', limit=1000, offset=0)
get_instrument_state_changes_history(path, start=None, end=None, symbol=None, since_id=None, *, inscode=None, source='tsetmc', limit=1000, offset=0)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
get_market_fundamentals	pe_min/pe_max: float None positive_pe=True شامل مرز؛ archive_to؛ مثبت؛ P/E فقط max_requests=1 bulk.	دقیق attrs ثابت بالا؛ schema request_count,max_requests,missing_selectors,strict,stale_rejected,source,price_source,eps_source,no_lookahead,no_backfill,archive_path.	در bounds/filter/no-match strict: InvalidParameterError /resolver error: stale با allow_stale=False frame exception: نه attr تهی و provider typed errors مثال: fundamentals.
get_market_fundamentals_history	path SQLite؛ start/end/limit/offset فیلتر؛ symbols صفحه و بازه؛ pe_min/pe_max/positive_pe instrument_types غربال P/E؛ strict no-match؛ allow_stale snapshot نگه‌داشتن قدیمی.	attrs زنده با schema missing_selectors,strict,source,price_source,eps_source,current_eps_used,no_lookahead,no_backfill,coverage_start,coverage_end; current_eps_used=False.	file/schema/filter InvalidParameterError یا DataParsingError؛ مثال: backtest.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_price_adjustments</code>	شامل date bounds دقیق و selector	<code>DataFrame[InsCode, Symbol, GregorianCalendar, JalaliDate, AdjustedClosingPrice, UnadjustedClosingPrice, AdjustmentAmount, CorporateTypeCode, CorporateActionType, IsConfirmedDPS, IdentityVerified, Source, FetchedAt] ;</code> <code>IsConfirmedDPS=False ,</code> <code>attrs dps_available=False .</code>	resolver/parameter/connection/parsing مثال فصل تعدیل
<code>get_latest_price_adjustment</code>	هویت: <code>symbol/ins_code</code> فیلتر تاریخ: <code>start/end</code> نمایش. پس از اعمال فیلتر <code>progress</code> فقط جدیدترین رخداد انتخاب می‌شود	schema <code>get_price_adjustments ,</code> attrs و typed صفر یا یک ردیف provenance.	<code>InvalidParameterError ,</code> resolver errors, <code>ConnectionError ,</code> مثال: <code>DataParsingError</code> فصل تعدیل
<code>check_market_history</code>	موجود باشد SQLite باید <code>path</code>	دقیقاً <code>dict[str, bool int]</code> با <code>ok=True, SchemaVersion, ApplicationID</code> DataFrame نیست.	فایل گم‌شده مثال: <code>InvalidParameterError</code> غیر SQLite/نسخه ناسازگار مثال: <code>DataParsingError</code> <code>att.check_market_history(db)</code>
<code>save_market_snapshot</code>	<code>snapshot:</code> <code>DataFrame dict None</code> ; اگر fetch: فقط یک <code>as_of</code> مشاهده override	<code>str , SHA-256 snapshot_id ;</code> هم‌زمان و live/client/order با DB داخل attrs اتمیک؛ transaction بازیابی می‌شود reader	path/type/schema/storage <code>InvalidParameterError/DataParsingError</code> ; network مثال: <code>snapshot=None</code> فقط هنگام SQLite.
<code>load_market_snapshots</code>	فیلتر inclusive زمان و symbol ; pagination SQL	ردیف‌های <code>DataFrame</code> با <code>STOCK_COLUMNS</code> observation و meta <code>SnapshotID, AsOf, Source, SchemaVersion, NoBackfill, CoverageStart, SnapshotAtomic, PersistenceAtomic, SourceAtomic, PriceSourceAsOf, ClientSourceAsOf, NoLookahead .</code>	file/schema/filter typed مثال: SQLite.
<code>get_live_market_history</code>	SQLite: <code>path</code> <code>start/end/symbol/limit/offset/ascending/include_stale</code> معنایی alias. برای فیلتر و صفحه‌بندی reader rich live نه alias identity.	سازگار با frame/meta <code>load_market_snapshots .</code>	<code>InvalidParameterError</code> و file/schema: خطاهای <code>att.get_live_market_history(db, symbol="فملی") .</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_market_overview_history</code>	<code>flow/source</code> ، مستقل archive	payload exact provider قبلی + meta archive <code>ArchiveIdentity, Version, FirstObservedAt, ObservedAt, ProviderTimestamp, AsOf, Source, SchemaVersion, NoBackfill, ...</code>	نامعتبر <code>kind/source/schema</code> ؛ مثال <code>archive_to</code> ، <code>typed</code> .
<code>get_market_snapshot_summary_history</code>	atomic snapshot مشتق از <code>summary</code> ؛ همه <code>flow=None</code>	<code>DataFrame[flow, instrument_count, trade_count, total_volume, total_value, market_cap, trade_date, exchange_time, fetched_at, is_realtime_fresh]</code> + meta.	؛ مثال <code>file/filter/schema errors</code> ، <code>SQLite</code> .
<code>get_market_breadth_history</code>	همان فیلترهای <code>live</code> روی هر snapshot. <code>.persisted</code>	<code>BREADTH_COLUMNS</code> + meta؛ بدون snapshot-by-snapshot، look-ahead.	؛ مثال <code>file/filter/schema errors</code> ، <code>breadth history</code> .
<code>get_sector_flow_history</code>	همان فیلترهای <code>client</code> و <code>live</code> ، <code>prices</code> همان <code>.observation</code> .	<code>SECTOR_FLOW_COLUMNS</code> + meta/no-lookahead.	؛ مثال <code>file/filter/schema errors</code> ، <code>sector history</code> .
<code>record_market_event</code>	<code>event: MarketEvent</code> ؛ <code>include_snapshot</code> ؛ <code>retention/session</code> .	<code>event_id</code> ، SHA-256 <code>str</code> ؛ در <code>notification/snapshot</code> و <code>transaction</code> .	<code>type/kind/storage</code> ؛ مثال <code>InvalidParameterError/DataParsingError</code> ؛ مثال <code>record_to</code> watcher.
<code>get_market_event_history</code>	<code>kind=None initial delta heartbeat resync</code> ؛ <code>session/pagination</code> .	<code>DataFrame[event_id, SessionID, kind, SnapshotMode, sequence, fetched_at, trade_date, changed_incodes, changed_order_levels, market_state_changed, notification_tokens, cursor_before, cursor_after, retry_count, retry_error, notification_errors, snapshot, messages, state_changes]</code> + meta.	ناشناخته <code>kind</code> ؛ مثال <code>InvalidParameterError</code> ؛ <code>schema</code> حتی برای فایل گم‌شده؛ <code>replay</code> ؛ مثال <code>errors</code> .
<code>archive_market_records</code>	یکی از <code>kind</code> ؛ <code>messages, state_changes, overview</code> ؛ <code>frame: DataFrame</code> ؛ <code>recorded_at/source</code> هویت archive.	های تاریخچه <code>version</code> تعداد <code>int</code> ؛ صفر <code>true duplicate</code> نوشته‌شده؛ <code>transaction atomic</code> .	؛ مثال <code>kind/type/storage errors</code> ، <code>archive_to</code> ؛ <code>helper</code> های.
<code>get_market_messages_history</code>	و <code>flow/since_id/source</code> ؛ <code>time/page filters</code> .	schema پیام + archive meta؛ <code>dedup provider ID</code> .	؛ مثال <code>file/schema/filter errors</code> ، <code>archive</code> .
<code>get_instrument_state_changes_history</code>	قدیمی <code>spelling</code> و/یا <code>symbol</code> ؛ <code>inscode</code> ؛ <code>since_id/source</code> .	schema state + archive meta.	<code>selector</code> متناقض/فایل <code>schema</code> ؛ مثال <code>errors</code> ، <code>archive</code> .

قرارداد API: درآمد ثابت

```

parse_treasury_maturity(symbol)
day_count_fraction(start, end, convention='ACT/365F')
treasury_yield(price, maturity_date, settlement_date=None, face_value=1000000.0, day_count='ACT/365F')
bond_price(annual_yield, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None)
yield_to_maturity(price, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None, tolerance=1e-12, max_iterations=300)
bond_analytics(price, cashflows=None, settlement_date=None, day_count='ACT/365F',
compounding='nominal', frequency=1, price_type='dirty', accrued_interest=None, maturity_date=None,
face_value=None, coupon_rate=None, issue_date=None, annual_yield=None, bump_size=0.0001)
build_yield_curve(nodes, settlement_date, interpolation='log_discount', extrapolate=False,
duplicate_policy='error', day_count='ACT/365F', rate_compounding='effective', rate_frequency=1,
enforce_monotonic_discount=True)
get_ifb_yield_table(category='treasury')
get_treasury_yields(symbol=None, settlement_date=None, face_value=1000000.0, include_stale=False,
min_volume=0, price_source='auto', day_count='ACT/365F', strict=False, face_value_source=None,
source='tsetmc', maturity_date=None, maturity_map=None, allow_no_trade=False)
get_treasury_yield_history(symbol=None, start=None, end=None, limit=0, settlement_date=None,
face_value=1000000.0, include_today=False, date_format='jalali', price_source='auto',
day_count='ACT/365F', ascending=True, progress=True, strict=False, face_value_source=None,
max_requests=250, maturity_date=None, maturity_map=None, use_ifb_reference=False)
get_treasury_yields_history(symbol=None, start=None, end=None, limit=0, settlement_date=None,
face_value=1000000.0, include_today=False, date_format='jalali', price_source='auto',
day_count='ACT/365F', ascending=True, progress=True, strict=False, face_value_source=None,
max_requests=250, maturity_date=None, maturity_map=None, use_ifb_reference=False)
get_yield_curve(symbol=None, settlement_date=None, face_value=1000000.0, include_stale=False,
min_volume=0, min_nodes=3, price_source='auto', day_count='ACT/365F', interpolation='log_discount',
extrapolate=False, duplicate_policy='volume_weighted', enforce_monotonic_discount=True,
source='tsetmc')
get_yield_curve_history(symbol=None, start=None, end=None, limit=0, face_value=1000000.0,
include_today=False, date_format='jalali', price_source='auto', day_count='ACT/365F', min_nodes=3,
interpolation='log_discount', duplicate_policy='volume_weighted', enforce_monotonic_discount=True,
ascending=True, progress=True, max_requests=250, maturity_map=None)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>parse_treasury_maturity</code>	<code>symbol: Any</code> ; فقط full-match پس از تبدیل رقم YYMMDD اخذ ZWNJ: فارسی/عربی و حذف و 00..79→1400..1479 80..99→1380..1399 .	موفق ; dict None <code>maturity_jalali: str</code> , <code>maturity_gregorian: datetime.date</code> , <code>maturity_source='user_confirmed_symbol_jalali_ymmd'</code> .	<code>None</code> ، تاریخ نامعتبر/ non-match deterministic ؛ مثال exception نه بالاتر.
<code>day_count_fraction</code>	<code>start/end: date-like</code> ; <code>convention</code> .	<code>float</code> year fraction.	نامعتبر date order/convention <code>ValueError</code> ; <code>day_count_fraction(date(2025,1,1), date(2026,1,1)) == 1.0</code> .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>treasury_yield</code>	zero-coupon price/face/maturity/settlement.	شامل <code>dict</code> <code>DiscountFactor, EffectiveAnnualYield, ContinuousYield, SimpleAnnualYield, BankDiscountYield, MacaulayDuration, ModifiedDuration, Convexity, DV01, DaysToMaturity/Tenor</code> .	قیمت/face/date/day-count/ نامعتبر <code>ValueError</code> ؛ مثال .deterministic
<code>bond_price</code>	یا <code>cashflows</code> صریح، یا پارامترهای ساخت <code>schedule</code> ؛ <code>annual_yield</code> ؛ <code>price_type</code> .	<code>float</code> clean یا dirty price.	cashflow/rate/frequency/date مثال: <code>ValueError</code> نامعتبر <code>bond_price(0.2, [(date(...), amount)], ...)</code> .
<code>yield_to_maturity</code>	قیمت مشاهده‌شده؛ <code>price</code> صریح یا <code>cashflows</code> <code>settlement_date/maturity_date/face_value/coupon_rate/frequency/issue_date</code> برای <code>schedule</code> ؛ <code>accrued_interest/price_type</code> clean/dirty؛ <code>compounding</code> <code>tolerance/max_iterations</code> solver.	با <code>float</code> annual yield یا انتخابی <code>compounding</code> .	price bounds/عدم bracket <code>ValueError</code> ؛ یا عدم همگرایی فصل درآمد ثابت round-trip مثال.
<code>bond_analytics</code>	ورودی <code>schedule</code> مانند ؛ <code>yield_to_maturity</code> <code>annual_yield=None</code> یعنی حل از YTM <code>price</code> ؛ <code>day_count/compounding/price_type</code> convention <code>bump_size=0.0001</code> شوک DV01.	با <code>dict</code> price/yield. <code>MacaulayDuration, ModifiedDuration, Convexity, DV01</code> و <code>metadata schedule</code> .	cashflow/rate/date/price نامعتبر <code>ValueError</code> ؛ ناموفق solver یا مثال deterministic.
<code>build_yield_curve</code>	rate/discount. های node <code>compounding</code> ، <code>duplicate</code> و <code>monotonic guard</code> .	<code>YieldCurve</code> ؛ و <code>node_metadata</code> با کلیدهای دقیق <code>diagnostics</code> <code>input_node_count, node_count, duplicate_count, duplicate_policy, monotonic_discount_count_enforced</code> .	node کم/duplicate/discount غیرمثبت/non-monotonic <code>ValueError</code> ؛ مثال curve.
<code>get_ifb_yield_table</code>	<code>category: str='treasury'</code> IFB. دسته جدول صفحه	<code>DataFrame[Symbol, Price, LastTradeJalali, LastTradeDate, PublishJalali, PublishDate, MaturityJalali, Maturity, Volume, ReferenceYTM, ReferenceSimpleYield, ReferenceSource]</code> ؛ attrs <code>provenance</code> URL/time.	category/HTML/schema/connection مثال: <code>typed</code> ؛ comparison IFB.

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_treasury_yields</code>	live universe: <code>min_volume</code> ; <code>face_value_source</code> ; maturity override/map: <code>allow_no_trade</code> ; source.	هویت: <code>TREASURY_COLUMNS</code> ; price provenance/سررسید/تسویه yield، چهار duration/convexity/DV01، stale/status: attrs source/universe/fetch time.	<code>InvalidParameterError</code> , <code>StockNotFoundError/AmbiguousSymbolError</code> , <code>ConnectionError/DataParsingError</code> ; مثال live مثال.
<code>get_treasury_yield_history</code>	history OHLC: <code>max_requests</code> فقط <code>use_ifb_reference</code> reference: settlement per row مگر override.	<code>TREASURY_HISTORY_COLUMNS</code> با <code>TradeDate, JalaliDate, Open, High, Low, Close, Final, Volume...</code> analytics: attrs failures/request/provenance.	parameter/resolver/provider/parsing: مثال attrs/warning در partial history.
<code>get_treasury_yield_history</code>	کامل یکسان با signature alias identity	object/schema/attrs همان <code>get_treasury_yield_history</code>	همان؛ <code>att.get_treasury_yields_history</code> is <code>att.get_treasury_yield_history</code>
<code>get_yield_curve</code>	حداقل <code>min_nodes</code> ; duplicate :default volume-weighted extrapolation opt-in	از <code>YieldCurve</code> calibrated snapshot live: diagnostics و metadata nodeها را ثبت provenance/no-lookahead می‌کند.	node/invalid curve provider typed: <code>ValueError</code> و مثال curve live.
<code>get_yield_curve_history</code>	trade date: جدا برای هر curve بدون budget, hard <code>maturity_map</code> ; عمومی extrapolate	با <code>DataFrame</code> panel <code>CURVE_HISTORY_COLUMNS</code> , <code>CurveID, CurveStatus, CurveError, CurveNodeCount</code> و flags no-lookahead: attrs failures/request.	parameter/budget/provider/curve errors: status/attrs: روز ناموفق در مثال curve history.

قرارداد API: شاخص‌های صنعت

```
list_industry_indices(progress=True, include_member_count=False, refresh=False, max_workers=6)
get_industry_members(industry, include_live=True, include_client_type=False, include_orderbook=False,
progress=True, refresh=False)
get_industry_snapshot(industries=None, include_client_type=True, include_orderbook=False,
include_empty=False, progress=True, refresh=False, max_workers=6)
get_industry_history(industry, start=None, end=None, limit=0, ascending=True, progress=True)
get_industry_members_history(industry, days=30, ascending=True, progress=True, refresh=False)
compare_industries(industries, start=None, end=None, limit=0, metric="close", ascending=True,
progress=True, max_workers=6)
get_industry_relative_strength(industries, benchmark, start=None, end=None, limit=0, metric="close",
ascending=True, progress=True, max_workers=6)
get_industry_correlation(industries, start=None, end=None, limit=0, ascending=True, progress=True,
max_workers=6)
get_industry_membership_overlap(industries, progress=True, refresh=False, max_workers=6)
get_industry_membership_churn(industries, days=0, progress=True, refresh=False, max_workers=6)
get_industry_membership_events(industries, days=30, progress=True, refresh=False, max_workers=6)
get_industry_concentration(industries, top_n=10, include_weights_by_market_value=True, progress=True,
refresh=False, max_workers=6)
get_industry_momentum_profile(industries, windows=(5, 20, 60), start=None, end=None, progress=True,
max_workers=6)
get_industry_correlation_neighborhood(industry, industries=None, top=5, min_correlation=0.0,
by_absolute=False, start=None, end=None, limit=0, ascending=False, progress=True, max_workers=6)
get_industry_health_score(industries=None, start=None, end=None, top_concentration=10,
momentum_windows=(20, 60), progress=True, refresh=False, max_workers=6)
get_industry_intraday(industry, interval='1min', progress=True)
rank_industries(metric='IndexChangePct', top=None, ascending=False, include_client_type=True,
include_orderbook=False, progress=True, refresh=False, max_workers=6)
```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>list_industry_i ndices</code>	پیام: <code>progress: bool=True</code> <code>include_member_count:</code> عضویت ۴۵ صنعت را <code>bool=False</code> واکشی می‌کند؛ <code>refresh:</code> را دور می‌زند؛ <code>cache</code> در بازهٔ <code>max_workers: int=6</code> 1..16.	<code>DataFrame[IndustryName,I ndustryNameEn,IndustryGro upCode,IndexInsCode,Index Value,IndexPreviousValue, DayHigh,DayLow,IndexChang e,IndexChangePct,MemberCo unt,HasMembers,ExchangeTi me]</code> attrs source/cache/count.	نامعتبر <code>bool/worker</code> ؛ <code>InvalidParameterError</code> provider <code>ConnectionError/DataPar</code> singError؛ مثال فصل صنعت
<code>get_industry_me mbers</code>	نام/alias/ <code>industry: str int</code> ؛ <code>include_live=True</code> MarketWatch؛ <code>include_client_type=False</code> client metrics؛ پنج <code>include_orderbook=False</code> دو. <code>refresh=False</code> سطح/صف؛ نیاز دارند به live enrichment گزینه	<code>INDUSTRY_MEMBER_COLUMNS</code> ثابت با هویت، قیمت، معامله market cap تخمینی، client، order book و freshness: attrs exact membership/current/cache/coverage request.	صنعت گم‌شده ؛ <code>StockNotFoundError</code> ترکیب/نوع نامعتبر ؛ <code>InvalidParameterError</code> typed provider؛ مثال <code>get_industry_members("ب ل‌انک")</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
get_industry_snapshot	<code>industries: ;None str int iterable=None</code> <code>include_client_type=True</code> <code>include_orderbook=False</code> <code>include_empty=False</code> <code>refresh=False</code> <code>max_workers=6</code>	یک ردیف در هر شاخص با <code>INDUSTRY_SNAPSHOT_COLUMN</code> S: وضعیت شاخص، breadth، equal-weight/median/dispersion معامله، flow/coverage، queue/coverage و freshness attrs overlap/current membership/cache	selector/bool/worker یا <code>InvalidParameterError</code> ؛ <code>StockNotFoundError</code> snapshot/schema/provider .؛ مثال فصل صنعت typed
get_industry_history	<code>industry</code> اجباری؛ <code>start/end:</code> شمسی/میلادی؛ <code>limit:</code> <code>ascending:</code> پس از فیلتر؛ <code>bool=True</code> ؛ <code>progress</code>	<code>DataFrame[IndustryName, IndustryIndexCode, TradeDate, JalaliDate, High, Low, Close, Change, ChangePct]</code> ؛ attrs <code>volume_available=False</code> ، <code>completed_sessions_only=True</code>	تاریخ/limit/order نامعتبر ؛ <code>InvalidParameterError</code> صنعت/typed/provider/schema .؛ مثال تاریخچه typed
get_industry_members_history	<code>industry</code> ؛ <code>days: int=30</code> تاریخ آخر؛ <code>1..30</code> دقیقاً <code>ascending=True</code> ؛ <code>refresh=False</code>	long-form <code>INDUSTRY_MEMBER_HISTORY_COLUMNS</code> ؛ attrs برای اعضای فعلی؛ <code>point_in_time_membership=False</code> ، <code>survivorship_bias_possible=True</code>	days/bool/selector typed؛ provider/schema typed؛ مثال فصل صنعت
compare_industries	<code>industries (str</code> <code>start/end:</code> <code>limit: int=0</code> ، <code>metric:</code> <code>close price change_pct log_return</code> ، <code>ascending=True</code> ، <code>progress=True</code> ، <code>max_workers=6</code>	int	iterable)؛ <code>start/end:</code> <code>str None</code> ، <code>limit: int=0</code> ، <code>metric:</code> <code>close price change_pct log_return</code> ، <code>ascending=True</code> ، <code>progress=True</code> ، <code>max_workers=6</code>
get_industry_relative_strength	<code>industries</code> ، <code>benchmark</code> ، <code>start/end: str None</code> ، <code>limit: int=0</code> ، <code>metric:</code> <code>close price change_pct log_return</code> ، <code>ascending=True</code> ، <code>progress=True</code> ، <code>max_workers=6</code>	long-form <code>TradeDate, JalaliDate, BenchmarkIndexCode, BenchmarkName, IndustryIndexCode, IndustryName, IndustryReturn, BenchmarkReturn, RelativeStrength</code> ؛ attrs شامل <code>analysis='industry_relative_strength'</code> ، <code>benchmark_index_code</code> ، <code>industry_count</code>	benchmark/generic نامعتبر یا داده worker metrics/worker ناکافی ؛ <code>InvalidParameterError</code> ؛ مثال <code>relative_strength = ...</code>
get_industry_correlation	<code>industries (str</code> <code>start/end:</code> <code>limit: int=0</code> ، <code>ascending=True</code> ، <code>progress=True</code> ، <code>max_workers=6</code>	int	iterable)؛ <code>start/end:</code> <code>str None</code> ، <code>limit: int=0</code> ، <code>ascending=True</code> ، <code>progress=True</code> ، <code>max_workers=6</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
get_industry_membership_overlap	industries (str)	int	iterable), progress=True , refresh=False , max_workers=6 .
get_industry_membership_events	industries (str)	int	iterable), days: int=1..30 , progress=True , refresh=False , max_workers=6 .
get_industry_membership_churn	industries (str)	int	iterable), days: int=1..30 , progress=True , refresh=False , max_workers=6 .
get_industry_concentration	industries (str)	int	iterable), top_n: int=10 , include_weights_by_market_value =True, progress=True , refresh=False , max_workers=6 .
get_industry_momentum_profile	industries (str)	int	iterable), windows=(5,20,60) , start/end: str None , progress=True , max_workers=6 .
get_industry_correlation_neighborhood	هدف industry (str)	int), industries , top=5 , min_correlation=0.0 , by_absolute=False , start/end: str None , limit=0 , ascending=False , progress=True , max_workers=6 .	DataFrame[IndustryName, IndustryIndexCode, NeighborName, NeighborIndexCode ,Correlation,CorrelationAbs,CorrelationRank] ; شامل attrs analysis='industry_correlation_neighborhood' , target_industry_code , target_industry_name , top , min_correlation , by_absolute , start_date , end_date .
get_industry_health_score	industries=None (str)	int	iterable), start/end: str None , top_concentration=10 , momentum_windows=(20,60) , progress=True , refresh=False , max_workers=6 .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_industry_intraday</code>	<code>industry</code> : <code>interval</code> : یکی از <code>'1min'</code> , <code>raw</code> , <code>1min</code> , <code>5min</code> , <code>15min</code> , <code>30min</code> , <code>60min</code> , <code>1h</code> ; <code>progress</code> .	<code>DataFrame[IndustryName, IndustryIndexCode, Timestamp, JalaliDate, Interval, Open, High, Low, Close, Change, ChangePct]</code> ; <code>timezone</code> : تهران؛ <code>attrs</code> : مصنوعی و بدون <code>volume</code> و <code>latest-day</code> .	<code>interval/selector</code> <code>InvalidParameterError/S</code> <code>tockNotFoundError</code> مثال: <code>provider/schema typed</code> <code>get_industry_intraday("minخودرو", "5")</code> .
<code>rank_industries</code>	<code>metric</code> : مستند؛ <code>canonical/alias</code> <code>top</code> : <code>int None</code> ; <code>ascending=False</code> ; <code>client/order</code> <code>switches</code> : <code>refresh</code> ; <code>max_workers</code> .	<code>Rank</code> + <code>snapshot</code> : تمام ستون‌های <code>attrs</code> : ردیف <code>metric</code> تهی حذف؛ <code>ranking_metric, ranking_ascending</code> و <code>provenance</code> <code>snapshot</code> .	نامعتبر <code>metric/top/bool/worker</code> <code>InvalidParameterError</code> مثال: <code>provider typed</code> <code>rank_industries(metric="breadth", top=5)</code> .

قرارداد API: اختیار معامله و فهرست ابزارها

```

list_options(underlying=None, progress=True)
get_options_chain(underlying, fetch_oi=False, progress=True)
black_scholes_price(spot, strike, time_to_expiry, rate, volatility, option_type='call',
dividend_yield=0.0, exercise_style='european')
black_scholes_greeks(spot, strike, time_to_expiry, rate, volatility, option_type='call',
dividend_yield=0.0, exercise_style='european')
option_price_bounds(spot, strike, time_to_expiry, rate, option_type='call', dividend_yield=0.0,
exercise_style='european')
implied_volatility(option_price, spot, strike, time_to_expiry, rate, option_type='call',
dividend_yield=0.0, exercise_style='european', lower_volatility=0.0, upper_volatility=5.0,
tolerance=1e-08, max_iterations=200)
get_option_market(exchange=0, underlying=None, progress=True, max_requests=1)
analyze_option_chain(options=None, underlying=None, spot=None, risk_free_rate=None, yield_curve=None,
dividend_yield=0.0, valuation_date=None, exercise_style='european', parity_tolerance=None,
liquidity_weights=None, progress=True, *, allow_unverified_freshness=True)
option_put_call_ratios(options, group_by='market')
get_option_history(symbol, start=None, end=None, limit=0, include_today=False, snapshot_path=None,
progress=True, max_requests=3)
save_option_snapshot(path, options=None, exchange=0, progress=True, *, lock_timeout=10.0,
stale_lock_seconds=300.0)
load_option_snapshots(path)
list_etfs(progress=True)
list_bonds(progress=True)
list_funds(fund_type=None, progress=True, *, listed_only=False, strategy=None,
commodity_underlying=None, classification_status=None)
list_listed_funds(progress=True, *, fund_category=None, strategy=None, commodity_underlying=None,
classification_status=None, include_unknown=True)
list_indices(progress=True)
get_index_companies(index_name, progress=True)

```

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>list_options</code>	<code>underlying: str None</code> filter نام underlying.	قراردادها با هویت <code>DataFrame</code> ، <code>OptionType, Underlying*, Strike, BeginDate, EndDate, DaysToExpiry, ContractSize</code> و قیمت/حجم.	یا <code>provider parse/connection</code> یا <code>frame</code> مثال فصل اختیار.
<code>get_options_chain</code>	اجباری؛ <code>underlying</code> fan-out OI از <code>fetch_oi=False</code> جلوگیری می‌کند.	<code>calls:</code> دقیقاً شامل <code>dict DataFrame, puts: DataFrame, underlying_name, underlying_price, expiry_dates, market_time</code> با <code>fetch_oi</code> ستون‌های <code>OpenInterest, ContractSize, BeginDate, EndDate</code> .	؛ مثال <code>underlying/provider errors</code> ؛ فصل اختیار <code>chain</code> .
<code>black_scholes_price</code>	فقط European: scalar math params.	<code>float</code> premium.	نامعتبر <code>bounds/type/style</code> ؛ مثال <code>ValueError</code> ؛ <code>deterministic</code> .
<code>black_scholes_greeks</code>	ورودی‌های <code>spot/strike/time_to_expiry/rate/volatility</code> عددی؛ <code>option_type∈{call,put}</code> ؛ <code>dividend_yield</code> نرخ پیوسته؛ <code>exercise_style='european'</code> تنها سبک پشتیبانی‌شده.	<code>dict[Delta, Gamma, Vega, Vega1Pct, ThetaPerYear, ThetaPerDay, Rho, Rho100bp, Status]</code> یا <code>Status='ok'</code> یا <code>undefined_at_expiry_or_zero_volatility</code> .	نامعتبر <code>constraint</code> ؛ مثال <code>ValueError</code> ؛ <code>Greeks</code> .
<code>option_price_bounds</code>	بدون <code>volatility</code> ؛ <code>no-arbitrage bound</code> .	<code>tuple (lower: float, upper: float)</code> .	مثال <code>ValueError</code> ؛ <code>deterministic</code> با خروجی <code>(4.87705755, 100.0)</code> .
<code>implied_volatility</code>	<code>premium + bracket/tolerance/iterations: tolerance>0</code> ، عدد صحیح مثبت <code>max_iterations</code> و <code>0<=lower<upper</code> .	با <code>dict ImpliedVolatility, Status, Iterations</code> ؛ <code>status</code> های <code>ok/missing/expiry/out_of_bounds/no_bracket/non_converged</code> و کلیدهای تشخیصی اختیاری.	و <code>bounds</code> ناموجود/خارج <code>premium</code> هستند، نه <code>status</code> عدم همگرایی؛ فقط <code>exception</code> ؛ نامعتبر <code>constraint/style/bracket</code> ؛ مثال <code>ValueError</code> ؛ <code>IV</code> .

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_option_mark</code> <code>et</code>	<code>exchange: int=0 ; underlying</code> <code>filter: max_requests=1 bulk.</code>	<code>DataFrame[InsCode, PairID</code> <code>, PairSequence, ISIN, Symbol</code> <code>, Name, OptionType, Underlyi</code> <code>ngInsCode, UnderlyingSymbo</code> <code>l, UnderlyingName, Contract</code> <code>Size, Strike, BeginDate, End</code> <code>Date, DaysToExpiry, Last, Cl</code> <code>ose, Yesterday, Volume, Valu</code> <code>e, TradeCount, NotionalValu</code> <code>e, OpenInterest, YesterdayO</code> <code>penInterest, BidPrice, AskP</code> <code>rice, BidVolume, AskVolume,</code> <code>UnderlyingLast, Underlying</code> <code>Close, Price, PriceSource, A</code> <code>sOf, AsOfSource, SnapshotFr</code> <code>eshnessKnown, PriceFreshne</code> <code>ssKnown, Stale, NoTrade, Ana</code> <code>lyticsEligible, AnalyticsE</code> <code>ligibilityReason, Metadata</code> <code>Conflict, Source] ; attrs</code> <code>snapshot provenance.</code>	<code>exchange/budget/filter</code> <code>InvalidParameterError ;</code> <code>provider typed</code> مثال: <code>snapshot.</code>
<code>analyze_option_</code> <code>chain</code>	می‌کند؛ <code>fetch</code> <code>options=None</code> <code>spot/rate/curve overrides:</code> <code>allow_unverified_freshnes</code> <code>s</code> explicit risk switch.	<code>input columns +</code> <code>TimeToExpiry, Spot, RiskFr</code> <code>eeRate, DividendYield, Impl</code> <code>iedVolatility* , Greeks per</code> <code>unit/contract.</code> <code>spread/depth/liquidity.</code> <code>ParityResidual, ParitySta</code> <code>tus, ImpliedForward, Analyt</code> <code>icsReliability ; attrs</code> <code>assumptions/warnings.</code>	<code>input type TypeError ;</code> <code>math/freshness/rate/column</code> <code>problems ValueError ;</code> <code>provider errors if fetch</code> مثال: <code>حرفه‌ای.</code>
<code>option_put_call</code> <code>_ratios</code>	از یک <code>options: DataFrame</code> دقیقاً یکی <code>group_by</code> <code>AsOf</code> <code>atmیک</code> ؛ از <code>market, underlying, expiry, u</code> <code>nderlying_expiry .</code>	<code>DataFrame</code> با <code>call/put</code> <code>volume/value/OI.</code> <code>PCRVOLUME, PCRValue, PCROp</code> <code>enInterest</code> و <code>status</code> دقیق <code>PCRVOLUMEStatus, PCRValue</code> <code>Status, PCROpenInterestSta</code> <code>tus ; attrs coverage/source</code> ورودی	ستون/ <code>AsOf</code> یا <code>group</code> یا <code>قرارداد تکراری</code> نامعتبر <code>ValueError</code> و نوع غیر <code>TypeError</code> <code>DataFrame</code> ؛ مثال <code>PCR.</code>

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>get_option_history</code>	قرارداد دقیق؛ + <code>server history</code> <code>include_today</code> ; فقط <code>snapshot_path</code> join <code>snapshot</code> های <code>opt-in</code>: <code>budget</code>.	<code>DataFrame[Timestamp, InsCode, Symbol, Open, High, Low, Close, Last, Volume, Value, TradeCount, OpenInterest, BidPrice, AskPrice, BidVolume, AskVolume, UnderlyingLast, UnderlyingClose, ContractSize, Strike, EndDate, Price, PriceSource, Source, AsOf, Stale, NoTrade, AnalyticsEligible]</code> ; <code>attrs</code> failures/no-lookahead.	<code>selector/date/budget/provider/snapshot</code> ؛ مثال تاریخچه <code>schema errors</code> اختیار.
<code>save_option_snapshot</code>	<code>fetch</code>: lock یک <code>options=None</code> مثبت stale lock نامنفی و <code>timeout</code>.	شده با <code>dedupely</code> ترکیب <code>DataFrame</code> <code>OPTION_COLUMNS</code> ; <code>attrs</code> <code>schema_version, source, path</code> و lock با <code>JSON versioned</code> فایل . اتمیک نوشته می‌شود <code>replace</code>.	گم‌شده <code>parent</code> <code>FileNotFoundError</code> , <code>DataFrame</code> غیر <code>options</code> مقدار نامعتبر , <code>TypeError</code> قفل , <code>ValueError</code> اگر <code>provider</code> <code>TimeoutError</code> ; <code>snapshot</code> ; مثال <code>fetch</code>.
<code>load_option_snapshots</code>	<code>JSON versioned</code>. <code>path</code>	با <code>DataFrame</code> ؛ فایل <code>OPTION_COLUMNS</code> و <code>attrs</code> تهی با <code>frame</code> گم‌شده خطا نیست و <code>attrs['status']='missing'</code> می‌دهد .	خراب <code>JSON</code> <code>version</code> و <code>JSONDecodeError</code> مثال ; <code>ValueError</code> ناسازگار <code>history</code>.
<code>list_etfs</code>	فقط <code>progress</code> .	<code>DataFrame[InsCode, ISIN, Symbol, Name, Last, Close, Yesterday, Volume, Value, TradeCount, Low, High, NAV, NAV_Discout, Change, ChangePct, MarketCode]</code> .	؛ مثال <code>provider typed/empty</code> <code>list_etfs().query("NAV_Discout < -1")</code> .
<code>list_bonds</code>	فقط <code>progress</code> .	<code>DataFrame[InsCode, ISIN, Symbol, Name, BondType, Ticker, MaturityJalali, MaturityGregorian, DaysToMaturity, Last, Close, Yesterday, Volume, Value, TradeCount, Change, ChangePct]</code> .	نامعلوم <code>maturity</code>: <code>provider</code>: <code>parse</code> ؛ مثال فصل ابزارها <code>nullable</code>.
<code>list_funds</code>	از <code>fund_type: str list None</code> ۱۵ دسته‌بندی؛ ؛ فیلترهای <code>listed_only=False</code> keyword-only: <code>strategy</code> , <code>commodity_underlying</code> , <code>classification_status</code> ; نام است <code>venture</code> <code>alias</code> قدیمی.	با <code>registry rich funds</code> <code>NAV/returns/composition/manager</code> و محورهای <code>registry_category, asset_exposure, strategy_tags</code> و شواهد طبقه‌بندی؛ بدون تضمین <code>InsCode</code>.	<code>type/category/combo/filter</code> نامعتبر یا <code>InvalidParameterError</code> <code>legacy</code>: <code>provider</code> رفتار سازگار خطا دار کنار گذاشته می‌شود؛ مثال فصل صندوق

تابع	ورودی‌ها و گزینه‌ها	خروجی/schema/attrs	خطا و مثال
<code>list_listed_funds</code>	یک MarketWatch bulk؛ <code>fund_category</code> , <code>strategy</code> , <code>commodity_underlying</code> , <code>classification_status</code> ; <code>.include_unknown=True</code>	شامل <code>LISTED_FUND_COLUMNS</code> و تمام <code>InstrumentType</code> ؛ هر دو نوع <code>taxonomy</code> ۳۰۵ و <code>attrs</code> ۳۸۰؛ <code>no_fuzzy_join=True, registry_joined=False, taxonomy_version .</code>	فیلتر نامعتبر : <code>InvalidParameterError</code> <code>connection/parsing</code> ؛ مثال <code>list_listed_funds(commodity_underlying="gold")</code>
<code>list_indices</code>	فقط <code>progress</code> .	<code>DataFrame[Name, InsCode, Value, High, Low, Change, ChangePct]</code> حرکت از <code>gePct</code> 1.2.0؛ <code>ChangePct</code> درصد واحد شاخص و است.	مسیر legacy در خطای provider frame تهی/پیام؛ برای صنعت <code>list_industry_indices</code> پیشنهاد می‌شود.
<code>get_index_companies</code>	فارسی یا <code>index_name: str</code> <code>InsCode</code> .	<code>DataFrame[Symbol, Name, InsCode, Close, Yesterday, Last]</code> .	در مسیر provider/گم‌شده index تهی/پیام؛ مثال فصل legacy frame شاخص.

قرارداد alias‌ها و wrapperهای backward-compatible

قدیمی و تفاوت رفتاری مخفی نماند. تمام پارامترها signature های این جدول مدخل مستقل دارند تا alias است override را دارند؛ فقط تفاوت صریح جدول target تابع type/default/constraint

```

stock(symbol='', start=None, end=None, limit=0, raw=False, auto_adjust=True, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, adjust_volume=False,
return_type=None, ascending=True, save_path=None, include_today=False, *, ins_code=None,
asset_type='auto', **kwargs)
stock_RI(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
stock_RL(symbol='', start=None, end=None, limit=0, raw=False, output_type='standard',
date_format='jalali', progress=True, save_to_file=False, dropna=True, ascending=True, save_path=None,
include_today=False, *, ins_code=None, asset_type='auto', **kwargs)
stock_capital_increase(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_intraday(symbol='شتران', interval='1min', start=None, end=None, progress=True, **kwargs)
stockdetail(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_information(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_statistics(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stock_introduction(symbol='', *, ins_code=None, asset_type='auto', **kwargs)
stocklist(bourse=True, farabourse=True, payeh=True, haghe_taqadom=False, sandogh=False, bonds=False,
options=False, mortgage=False, commodity=False, energy=False, payeh_color=None, output='dataframe',
progress=True, **kwargs)
shareholders(symbol='', date=None, include_id=False, *, ins_code=None, asset_type='auto', **kwargs)
currency_coin(name='', start=None, end=None, limit=0, output_type='standard', date_format='jalali',
progress=True, save_to_file=False, dropna=True, return_type=None, ascending=True, save_path=None,
**kwargs)
market_watch()
market_client_type()
market_data()

```

alias/wrapper	و تفاوت target	خروجی/schema/attrs	خطا و مثال
stock	همان ; <code>get_history</code> target signature و implementation history.	همان DataFrame/attrs.	همان خطاها؛ <code>att.stock("فملى", .limit=10)</code>
stock_RI	. <code>get_client_type</code> target	همان DataFrame/attrs.	همان؛ <code>att.stock_RI("فملى", .limit=10)</code>
stock_RL	که به wrapper compatibility identity می‌کند؛ delegate <code>stock_RI</code> alias signature نیست ولی برابر است.	همان DataFrame/attrs.	همان؛ برای کد جدید . <code>get_client_type</code>
stock_capital_i ncrease	target canonical . <code>get_capital_increase</code>	همان frame/None قدیمی.	همان.
stock_intraday	target <code>get_intraday</code> ; signature برابر.	همان tick/candle frame.	همان.
stockdetail	. <code>get_detail</code> target	همان key/value frame یا None .	همان.
stock_informati on	. <code>get_info</code> target	همان key/value frame یا None .	همان.
stock_statistic s	. <code>get_stats</code> target	همان key/value frame یا None .	همان.
stock_introduc tion	target <code>get_introduction</code> ; wrapper legacy unsupported.	هیچ خروجی موفق.	همیشه <code>UnsupportedDataSourceEr ror</code> پیش از I/O.
stocklist	همان ; <code>get_symbols</code> target signature و alias های kwargs.	همان فهرست.	همان.
shareholders	. <code>get_shareholders</code> target	همان frame/None.	همان.
currency_coin	. <code>get_currency</code> target	همان frame/MultiIndex و source .TGJU	همان.
market_watch	تابع صفرآرگومان canonical قدیمی target :snapshot مفهومی . <code>get_market_snapshot</code>	<code>dict[stocks,order_book,m arket_time,...]</code> ستون‌های legacy <code>Yesterday/BaseVolume/Cha nge</code> حفظ شده و ستون‌های corrected additive هستند.	<code>ConnectionError/DataPar singError ; snapshot=att.market_wat ch()</code>
market_client_t ype	تابع صفرآرگومان bulk :target مفهومی . <code>get_market_client_type</code>	CLIENT_COLUMNS DataFrame.	typed provider errors.
market_data	صفرآرگومان به deprecated wrapper <code>market_watch</code> ; identity alias نیست.	همان dict.	همان؛ برای کد جدید <code>get_market_snapshot/get _live_market</code>

دو alias identity غیر legacy نیز قبلاً مدخل دارند: `get_orderbook_history is get_order_book_history` و `get_treasury_yields_history is get_treasury_yield_history`. تفاوت signature یا خروجی ندارند.

ها نیز دقیقاً چنین است signature constructor exception

```
AlgotikTSEError(*args)
AmbiguousSymbolError(*args)
ConnectionError(*args)
DataParsingError(*args)
InvalidParameterError(*args)
StockNotFoundError(*args)
UnsupportedDataSourceError(*args)
```

الگوهای کاربردی

فیلتر قدرت خریدار حقیقی با نقدشوندگی

```
live = att.get_live_market()
screen = live.loc[
    (live["IndividualPower"] > 1.5)
    & (live["Value"] > 50_000_000_000)
    & live["is_realtime_fresh"].fillna(False)
].sort_values("EstimatedNetIndividualFlow", ascending=False)

print(screen[[
    "Symbol", "Last", "ChangePct", "IndividualPower",
    "EstimatedNetIndividualFlow", "SpreadBps", "L5Imbalance",
]])
```

look-ahead بدون backtest

```
db = "research.sqlite"
att.save_market_snapshot(db)

fund = att.get_market_fundamentals_history(db, symbols="فملی")
assert fund.attrs["no_lookahead"] is True
assert fund.attrs["current_eps_used"] is False

curves = att.get_yield_curve_history(
    start="1403-01-01", end="1403-03-31", max_requests=100, progress=False
)
safe_curves = curves.loc[curves["CurveNoLookahead"].fillna(False)]
```

مانیتور بازار و archive مستقل

```
db = "monitor.sqlite"

for event in att.watch_market(
    interval=3,
    max_updates=20,
    record_to=db,
    notifications=("messages", "state"),
):
    if event.kind in {"initial", "delta"}:
        print(event.sequence, len(event.changed_inscodes))

# archive_to مستقل فعال شود helper باید روی
att.get_market_messages(flow=0, top=50, archive_to=db)
messages = att.get_market_messages_history(db, flow=0)
```

منابع داده

منبع	استفاده
های subdomain و tsetmc.com (TSETMC رسمی)	قیمت، client type، market watch، سفارش، trades، پیام، وضعیت، ابزار، صندوق و اطلاعات بازار
فراپورس ایران (ifb.ir/ytm.aspx)	جدول مرجع YTM برای مقایسه/دسته‌بندی اوراق؛ منبع مجاز و با provenance جدا
TGJU (api.tgju.org)	فقط API legacy ارز و سکه

کدال منبع این پکیج نیست؛ حتی endpointهای proxy شده آن زیر host دیگر در boundary شبکه رد می‌شوند.

تست و مشارکت

تست پیش‌فرض کاملاً آفلاین است:

```
python -m pytest -m "not online"
```

یا:

```
make test
```

است: opt-in آنلاین محدود و smoke

```
python -m pytest -m online --timeout=30
```

تست آنلاین به وضعیت بازار/provider وابسته است و جایگزین تست deterministic آفلاین نیست. پیش از PR:

```
python -m pytest tests/test_release_offline.py -q
python -m pytest -m "not online" -q
```

برای مشارکت، issue یا pull request در [GitHub](#) باز کنید. انتشار PyPI فقط با مسیر دستی و تأیید صریح انجام می‌شود؛ target عادی [release](#) صرفاً artifact محلی می‌سازد و upload نمی‌کند.

مجوز و ارتباط

این پروژه تحت [GNU General Public License v3](#) منتشر می‌شود.

- وبسایت: [algotik.com](#)
- تلگرام: [t.me/algotik](#)
- نویسنده: Mohsen Alipour — [alipour@algotik.ir](#)

تاریخچه تغییرات

این بخش در هر انتشار به‌روزرسانی می‌شود؛ جزئیات کامل‌تر در [HISTORY.rst](#) ثبت شده است.

- **1.4.0 – 2026-09-12**: تجمیع تقویم و نمای کلان بازار با ابزارهای پیش‌گشایش؛ تقویم رسمی بورس/فراپورس، ارزش بازار تاریخی، فعالیت روزانه، اثر نمادها بر شاخص، خلاصه bulk روز، مستر کامل ابزارها و diff مبتنی بر TOP، [InsCode](#) رسمی مشترکان و عدم تعادل پیش‌گشایش.
- **1.3.0 – 2026-09-12**: طبقه‌بندی چندمحوره صندوق‌ها، اصلاح شناسه‌های registry، افزودن پنج دسته جدید، پوشش صندوق‌های بورسی نوع ۳۸۰، فیلتر طلا/نقره/زعفران و شواهد قابل ممیزی.
- **1.2.4 – 2026-09-02**: رویدادهای افزوده/حذف‌شده اعضای شاخص صنعت.
- **1.2.3 – 2026-09-01**: churn، همبستگی و امتیاز سلامت صنایع.
- **1.2.2 – 2026-09-01**: سنجش همپوشانی اعضای رسمی صنایع.
- **1.2.1 – 2026-09-01**: مقایسه، قدرت نسبی و همبستگی صنایع.
- **1.2.0 – 2026-09-01**: intraday، کامل شاخص‌های صنعت، اعضا، تاریخچه API.
- **1.1.3 – 2026-08-28**: بازطراحی راهنمای PDF و هماهنگ‌سازی badge نسخه.
- **1.1.2 – 2026-08-28**: بازطراحی README و تکمیل قرارداد ورودی/خروجی توابع.
- **1.1.1 – 2026-08-25**: بهبود نمایش فارسی و تکمیل badgeهای پروژه.
- **1.1.0 – 2026-08-24**: داده زنده/تاریخی، SQLite، watcher، اخزا، منحنی بازده و اختیار معامله.
- **1.0.3 – 2026-07-12**: جست‌وجوی املای canonical نماد با نرمال‌سازی فارسی/عربی.
- **1.0.2 – 2026-03-24**: گسترش شاخص‌های صنعت و افزودن فهرست شاخص‌ها و اعضا.
- **1.0.1 – 2026-02-19**: تکمیل وابستگی‌ها و اصلاح نوع داده [NAV_Discount](#).

- **1.0.0 – 2025-07-12**: انتشار پایدار با HTTP client مرکزی، تنظیمات و exception های اختصاصی.
- **0.3.12 تا 0.2.8 – 2023 تا 2024**: نسخه های آغازین قیمت، سهامداران، افزایش سرمایه، اطلاعات نماد، ارز و فهرست ابزارها.